Hands-on Labs

Create a Bedrock Custom Model with Fine-tuning and inference

Test this lab: if you're a machine learning engineer, data scientist, or AI enthusiast, this lab will empower you with the knowledge and skills needed to leverage AWS Bedrock and unlock the full potential of Foundation Models.

Real-Time Scenario:

You are a Machine Learning Engineer working for a company that runs an online gardening and plant-care platform. Your task is to fine-tune a Foundation Model (FM) using Amazon Bedrock so it can accurately answer domain-specific gardening questions - topics like plant care, composting, pest control, and soil management - that the base model doesn't handle well out of the box. After fine-tuning the model on your curated gardening dataset, you will deploy it for on-demand inference and test it in the Bedrock Playground to confirm it generates relevant, accurate gardening advice for your users.

In addition to the primary gardening scenario, this lab also includes **six optional industry scenarios** - covering Support Ticket Triage, IT/DevOps Helpdesk, Pharmacovigilance, Patient Message Triage, Banking, and E-Commerce - so you can explore how the same fine-tuning workflow applies across a wide range of real-world business domains.

.

Description:

In this hands-on lab, you will learn how to fine-tune a Foundation Model (FM) in Amazon Bedrock to cater to a specific business use case. Amazon Bedrock provides you with a seamless platform to work with various Foundation Models and customize them based on your own data.

You will start with a primary use case - a gardening knowledge assistant - and walk through the complete workflow: uploading training data to S3, running a supervised fine-tuning job, deploying the model for on-demand inference, and validating it in the Bedrock Playground.

Once the core workflow is clear, the lab expands into **six optional industry scenarios** that many of our learners have specifically requested. Each scenario follows the exact same process but applies it to a different domain - from IT helpdesk automation to pharmaceutical adverse-event triage - so you can see how fine-tuning translates into practical value across industries. By the end of this lab, you will have a solid, repeatable framework for customizing Foundation Models for any specialized business application.

Contents

1. Introduction

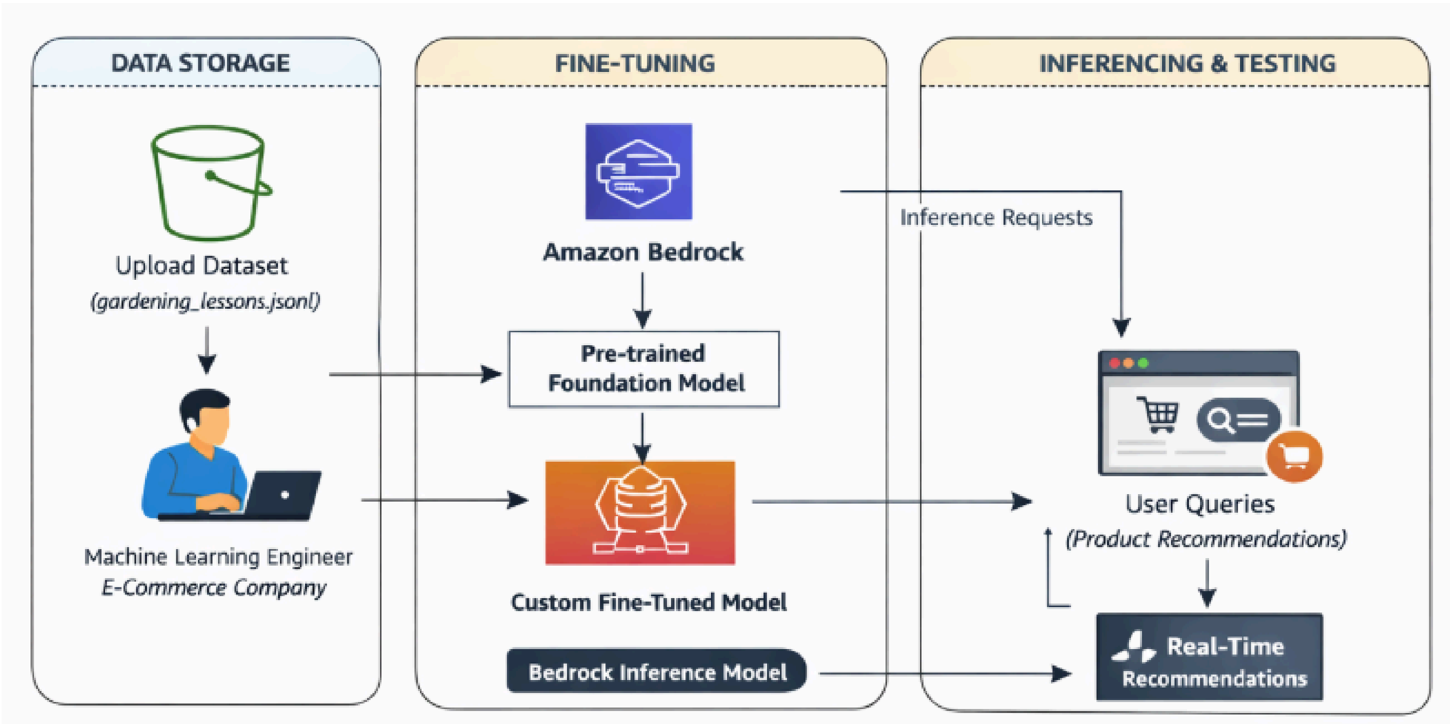
In today's AI-driven landscape, **Foundation Models (FMs)** enable organizations to build intelligent applications quickly without training models from scratch. While these pre-trained models are highly capable, many real-world business use cases require domain-specific knowledge to deliver accurate and relevant responses. This is where **fine-tuning** and **inference** become essential.

Amazon Bedrock provides a fully managed platform for accessing, customizing, and deploying Foundation Models using your own datasets, making it easier to create AI applications tailored to specific industries and use cases.

Fine-tuning allows you to adapt a pre-trained Foundation Model using a curated dataset so it performs better on specialized tasks. **Inference** is the process of using the fine-tuned model to generate accurate responses for new user queries in real time.

In this lab, you will fine-tune an Amazon Bedrock Foundation Model using a curated **gardening knowledge dataset** containing information about plant care, composting, pest control, and soil management. Once the model has been customized, you will deploy it for **on-demand inference** and validate its performance using the **Amazon Bedrock Playground** to ensure it provides accurate, domain-specific gardening advice.

By the end of this lab, you will understand how to customize Foundation Models for specialized business applications, deploy them for inference, and evaluate their performance in a production-ready environment. These skills will help you build intelligent AI assistants that deliver accurate, context-aware responses for industry-specific use cases using Amazon Bedrock.



This activity guide covers steps for:

1. Create an S3 Bucket to Store Data
2. Create and Customize a Foundation Model (FM) in Amazon Bedrock
3. Access and Test the Customized Foundation Model
4. Delete Resources (Clean Up)

2. Overview & Key Concepts

In this section, we'll cover key concepts used in this guide, including an overview of what's covered in this guide.

a. What is Amazon Bedrock?

Amazon Bedrock is a fully managed service that provides access to high-performance foundation models (FMs) from leading AI companies such as AI21 Labs, Anthropic, Cohere, Meta, Stability AI, and Amazon through a single API.

It offers extensive features for building generative AI applications, making development easier while ensuring privacy and security with Amazon Bedrock, you can experiment with various top FMs, customize them privately with your data using methods like fine-tuning and retrieval augmented generation (RAG), and create managed agents for complex business tasks such as booking travel, processing insurance claims, creating ad campaigns, and managing inventory—all without needing to write any code.

Being serverless, Amazon Bedrock eliminates the need to manage infrastructure, allowing you to securely integrate and deploy generative AI capabilities into your applications using familiar AWS services.

b. Amazon S3 (Simple Storage Service)

Amazon S3 is a scalable object storage service designed to store and retrieve large amounts of data.

In the Bloodbank project, S3 is used to store files such as images, documents, and backups.

The service provides durability and scalability, which makes it ideal for storing static assets like donor photographs, medical records, and reports, ensuring they are readily available and secure.

The S3 service also integrates with other AWS tools like Lambda and CloudFront, enabling automation of tasks such as data processing or distributing content globally via CDN.

Its pay-as-you-go pricing ensures cost-effectiveness even as the amount of data grows.

c. Bedrock Custom Model

A **custom model in Amazon Bedrock** is a machine learning model that is built and fine-tuned from a pre-trained foundation model (FM) to meet the specific needs of a given use case.

By uploading your own data, you can fine-tune the foundation model to enhance its accuracy and relevance for your specific task. After fine-tuning, the model can be tested and deployed for real-time inference or batch processing.

This allows businesses to create specialized AI solutions tailored to their unique requirements, while leveraging Amazon Bedrock's scalable infrastructure.

3. Documentation Links

1. Getting Started with AWS Amazon Bedrock (Enable Foundation Model)
   https://docs.aws.amazon.com/bedrock/latest/userguide/getting-started.html

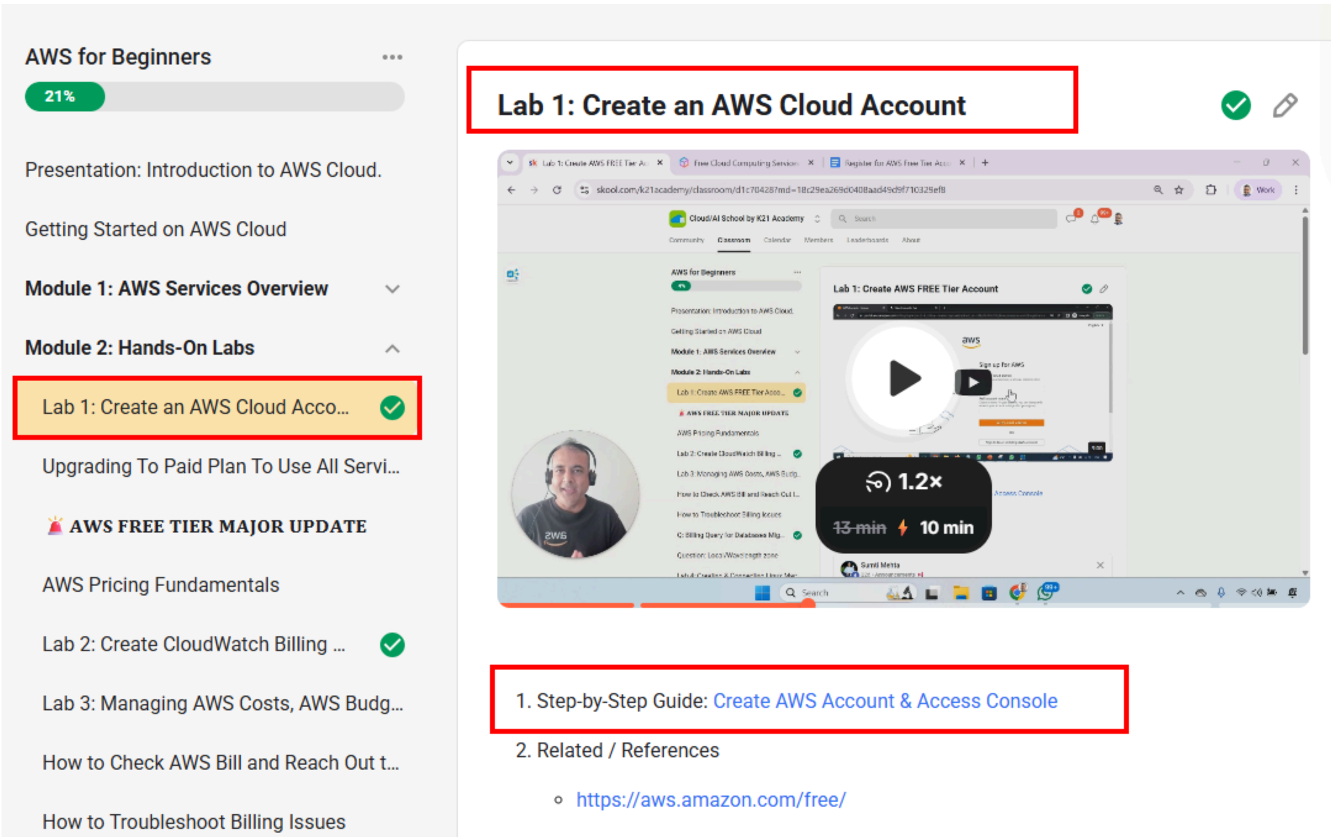
2. BedRock Supported Regions
   https://docs.aws.amazon.com/bedrock/latest/userguide/bedrock-regions.html

3. Amazon BedRock Pricing
   https://aws.amazon.com/bedrock/pricing/

   (**Note:** There is no charge to perform this lab.)
   Basic Concepts or Key Definitions: Foundation model (FM), Base model, Model inference, Prompt, Token, Model parameters, Inference parameters, Playground, Embedding, etc…
   https://docs.aws.amazon.com/bedrock/latest/userguide/key-definitions.html

4. Prerequisite

1. An AWS Account: If you don't have an AWS Account, then please check Activity ***Guide: Create AWS Account [Here](#)***.



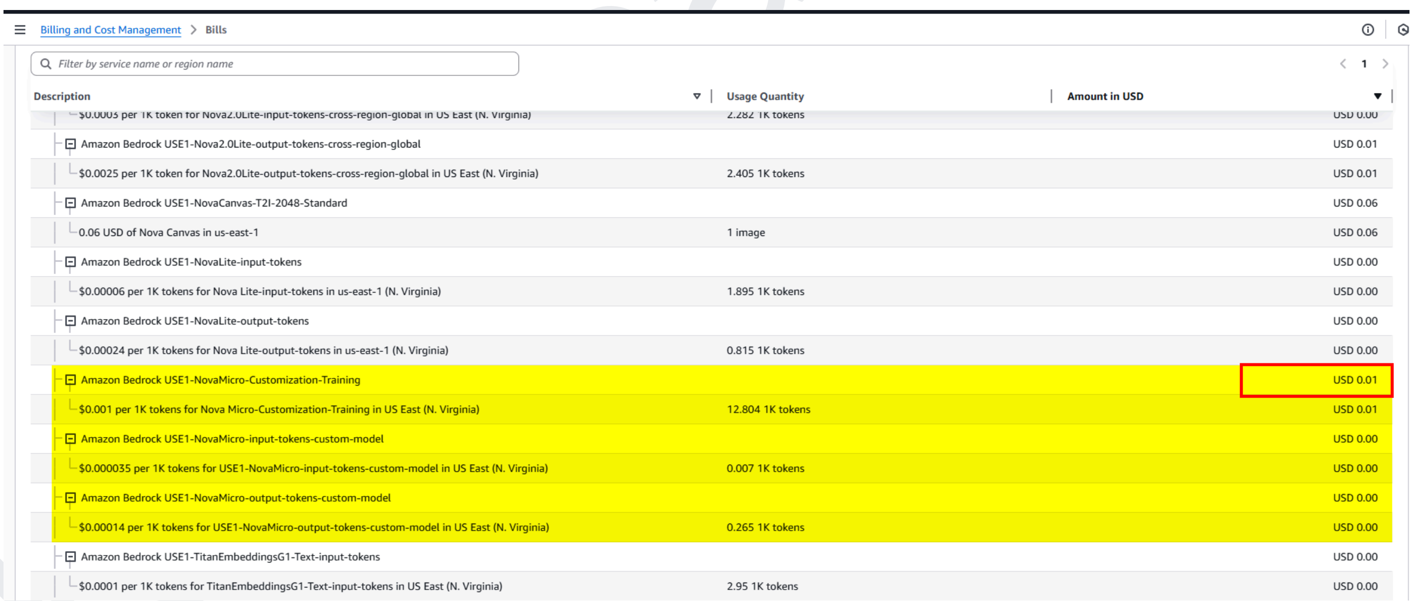
5. Cost To Perform This Lab

The **estimated cost** for this lab is **$0.01**, as long as usage stays within the Free Tier limits. Please note that if you run more prompts, you may incur higher charges.

📌 **Important Note:** However, don't worry; any charges will be deducted from the **free credits** you received when you created your AWS account. As long as you have remaining credits, you won't need to pay out of pocket for these charges.

AWS Free Tier Credits Breakdown

- **$100 Available Immediately**: When you sign up for an AWS Free Tier account, you receive $100 in credits that are available for use right away.
- **$100 After Completing Specific Activities**: After fulfilling certain conditions, such as completing AWS labs or certain tasks, you'll receive an additional $100 in credits.



Billing and Cost Management > Bills

Description	Usage Quantity	Amount in USD
$0.0003 per 1K token for Nova2.0Lite-input-tokens-cross-region-global in US East (N. Virginia)	2.282 1K tokens	USD 0.00
Amazon Bedrock USE1-Nova2.0Lite-output-tokens-cross-region-global		USD 0.01
$0.0025 per 1K token for Nova2.0Lite-output-tokens-cross-region-global in US East (N. Virginia)	2.405 1K tokens	USD 0.01
Amazon Bedrock USE1-NovaCanvas-T2I-2048-Standard		USD 0.06
0.06 USD of Nova Canvas in us-east-1	1 image	USD 0.06
Amazon Bedrock USE1-NovaLite-input-tokens		USD 0.00
$0.00006 per 1K tokens for Nova Lite-input-tokens in us-east-1 (N. Virginia)	1.895 1K tokens	USD 0.00
Amazon Bedrock USE1-NovaLite-output-tokens		USD 0.00
$0.00024 per 1K tokens for Nova Lite-output-tokens in us-east-1 (N. Virginia)	0.815 1K tokens	USD 0.00
Amazon Bedrock USE1-NovaMicro-Customization-Training		USD 0.01
$0.001 per 1K tokens for Nova Micro-Customization-Training in US East (N. Virginia)	12.804 1K tokens	USD 0.01
Amazon Bedrock USE1-NovaMicro-input-tokens-custom-model		USD 0.00
$0.000035 per 1K tokens for USE1-NovaMicro-input-tokens-custom-model in US East (N. Virginia)	0.007 1K tokens	USD 0.00
Amazon Bedrock USE1-NovaMicro-output-tokens-custom-model		USD 0.00
$0.00014 per 1K tokens for USE1-NovaMicro-output-tokens-custom-model in US East (N. Virginia)	0.265 1K tokens	USD 0.00
Amazon Bedrock USE1-TitanEmbeddingsG1-Text-input-tokens		USD 0.00
$0.0001 per 1K tokens for TitanEmbeddingsG1-Text-input-tokens in US East (N. Virginia)	2.95 1K tokens	USD 0.00

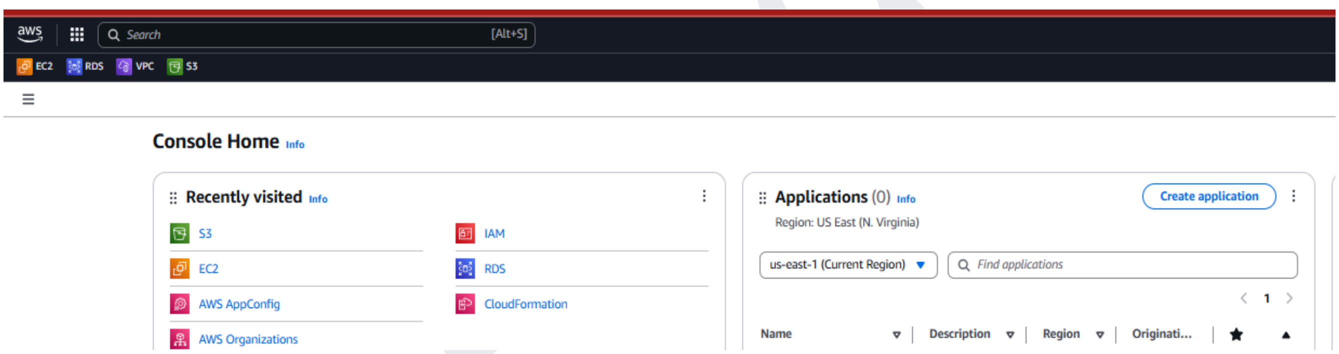
6. Create an S3 Bucket and upload the code

Creating an **S3 bucket** is a crucial step in this lab because it provides a secure and scalable location for storing your dataset, which is essential for fine-tuning a Foundation Model in Amazon Bedrock.

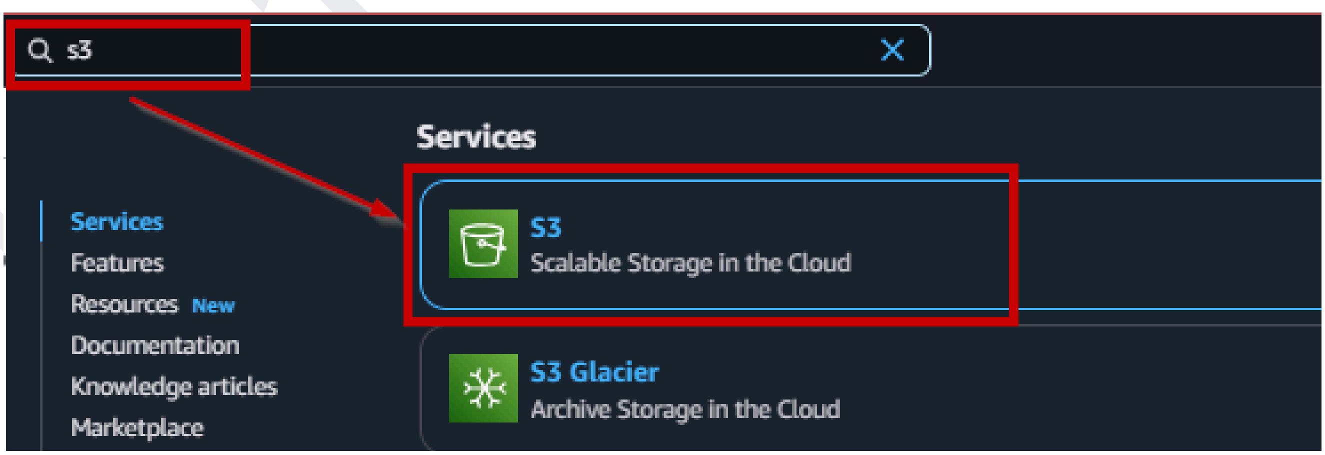
The S3 bucket allows for easy access to your data, ensuring it can be integrated seamlessly with Amazon Bedrock for model training. It also offers flexibility in handling different data types (e.g., text, images), and ensures the data remains secure and well-managed.

This step is foundational for preparing the data needed to train and test your customized model.

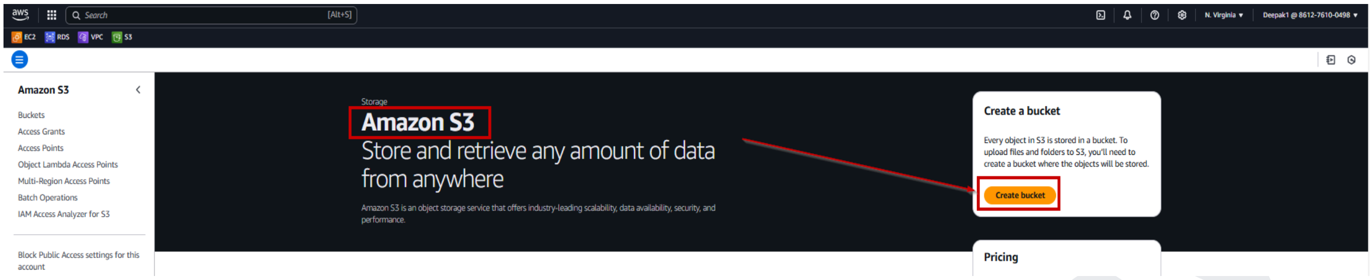
1. Open your AWS Console.



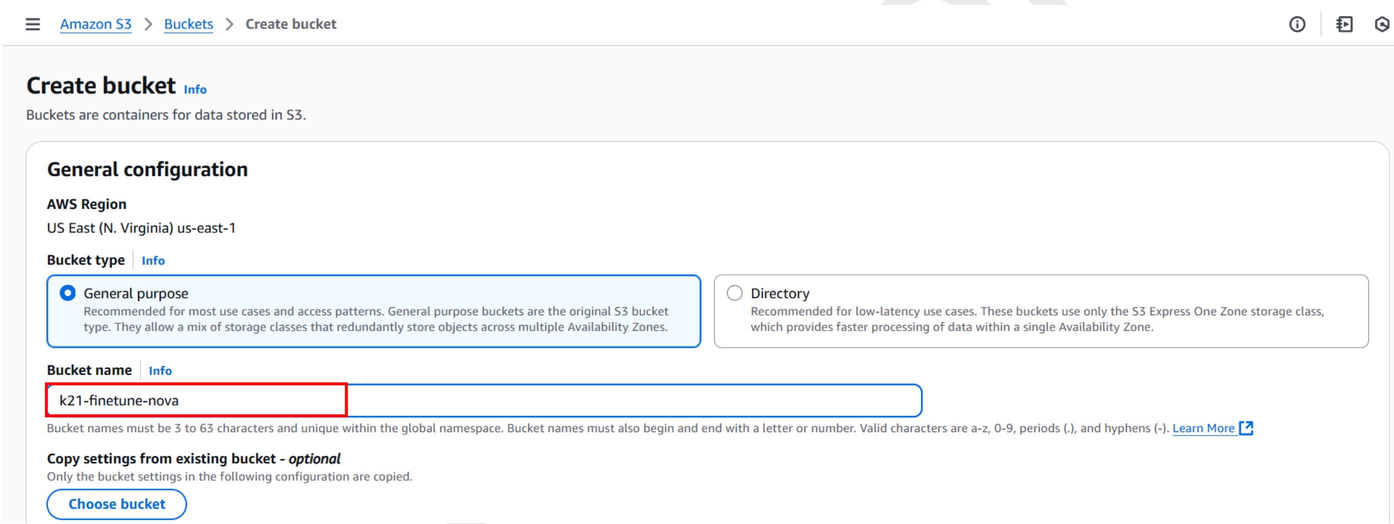
2. Click on the search bar, type **S3,** and click on **S3**.



3. Click on the Create **Bucket** button.



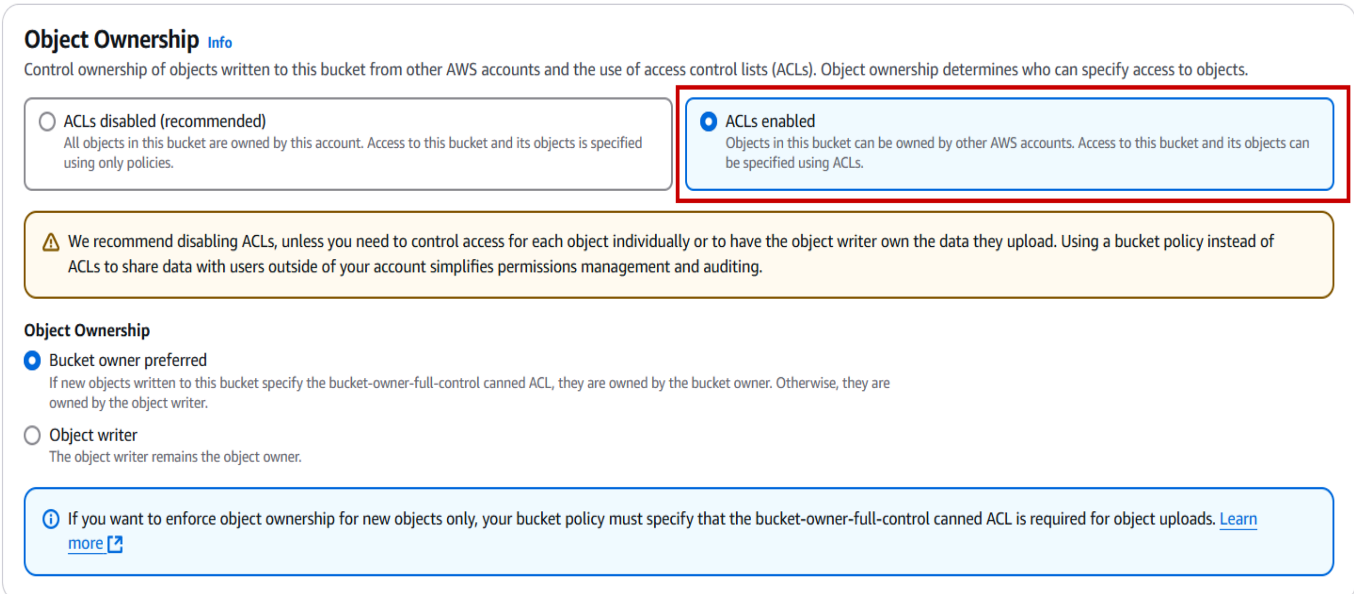
4. In the Name and Region screen, provide a unique **Bucket Name** (k21-finetune-nova) to your bucket and select a **Region** of your choice, then click **Next**.



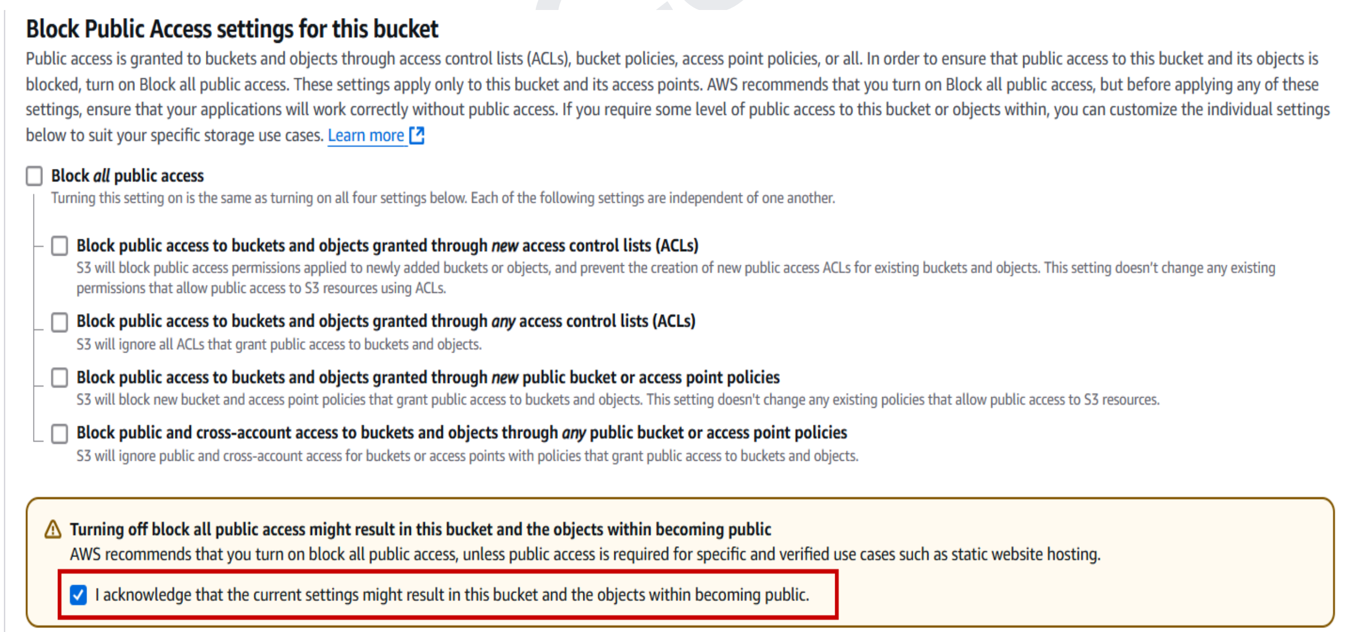
Note: An Amazon S3 bucket name is **globally unique,** and the namespace is shared by all AWS accounts. This means that after a bucket is created, the name of that bucket cannot be used by another AWS account in any AWS Region until the bucket is deleted.

Note: Now you shall see in the S3 console it shows **'Global',** which means S3 is not region-specific. However, you must then ask why we are selecting a region in the above step. S3 is a Global service, which means that you can access any bucket from any region, but if you want that access to be fast, you should have the bucket in the same region. That is, you shall choose any region for your bucket that is geographically close to you to optimize latency and minimize costs.

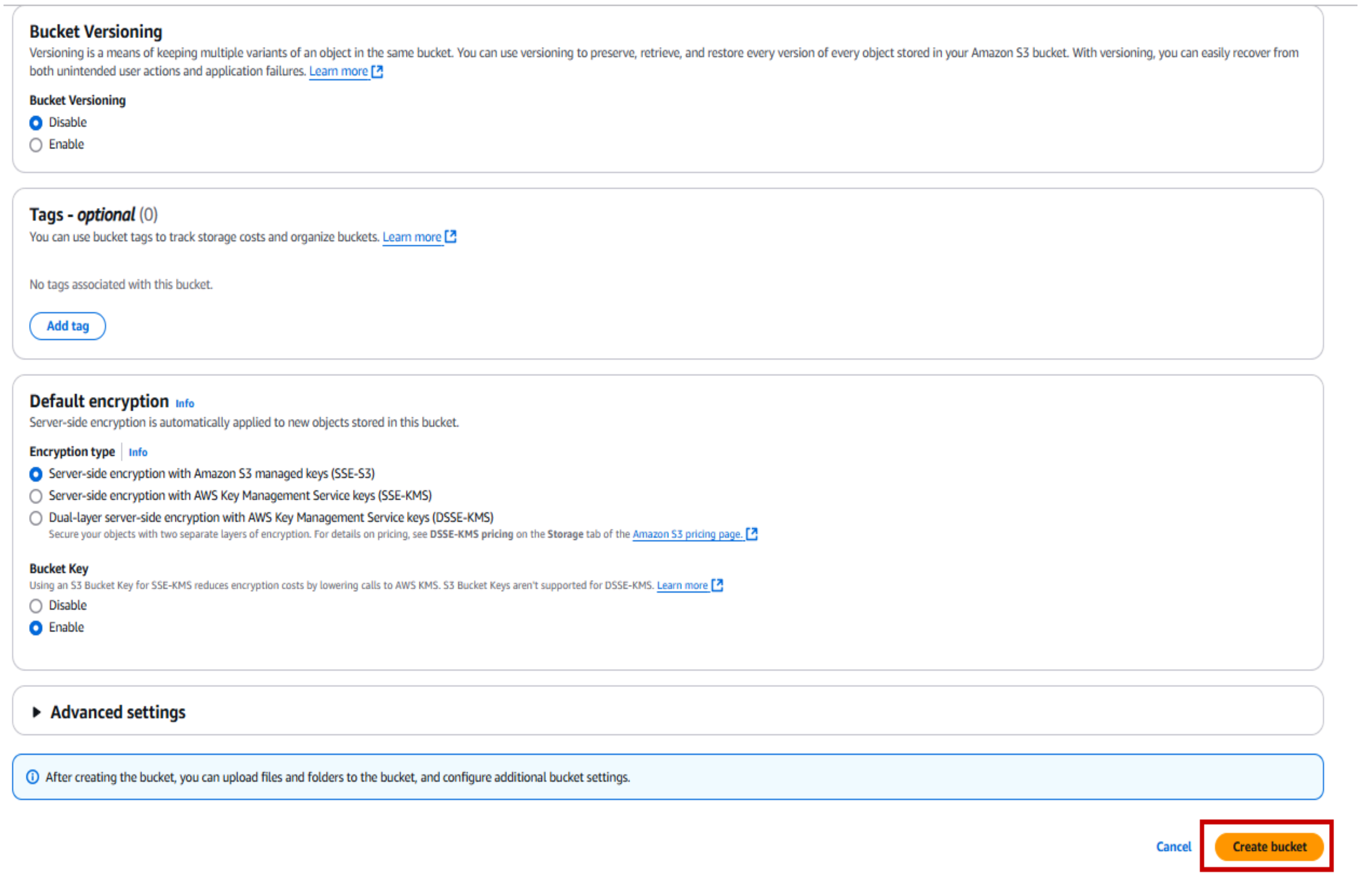
5. Now, under object ownership, choose the **ACLs enabled** option and scroll down to the page.



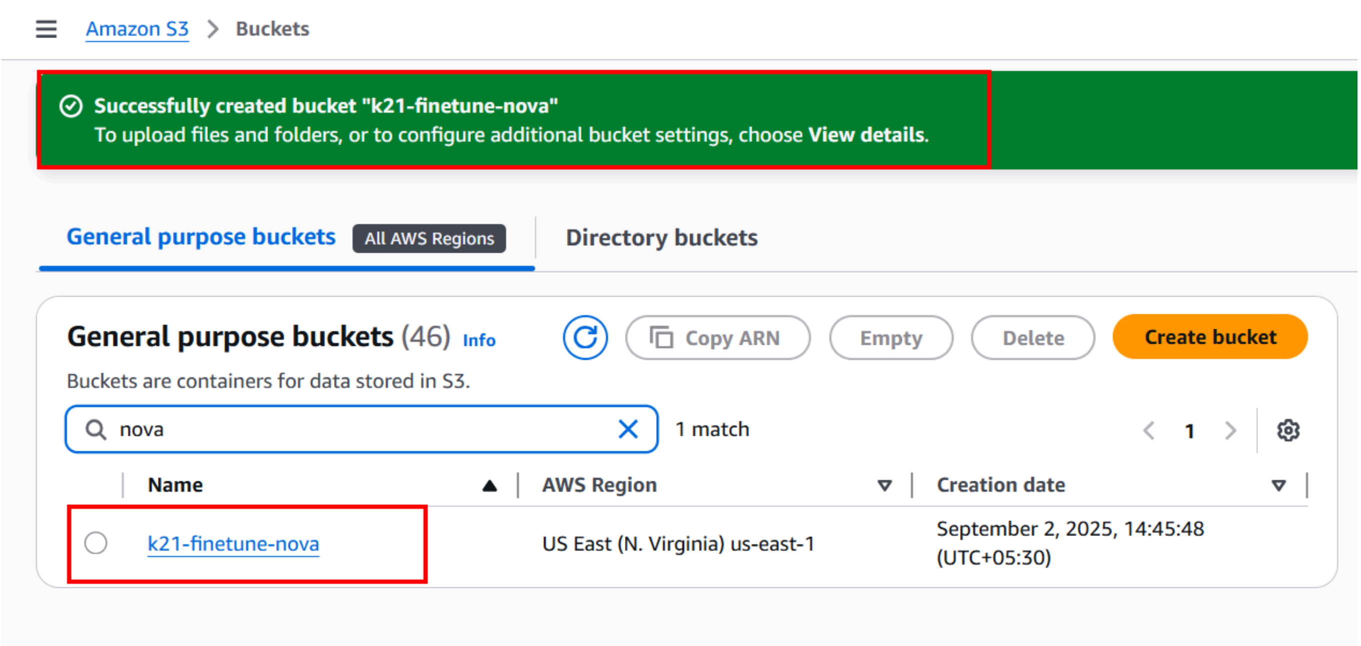
6. Here, we allow public access to our bucket and tick the acknowledge option.



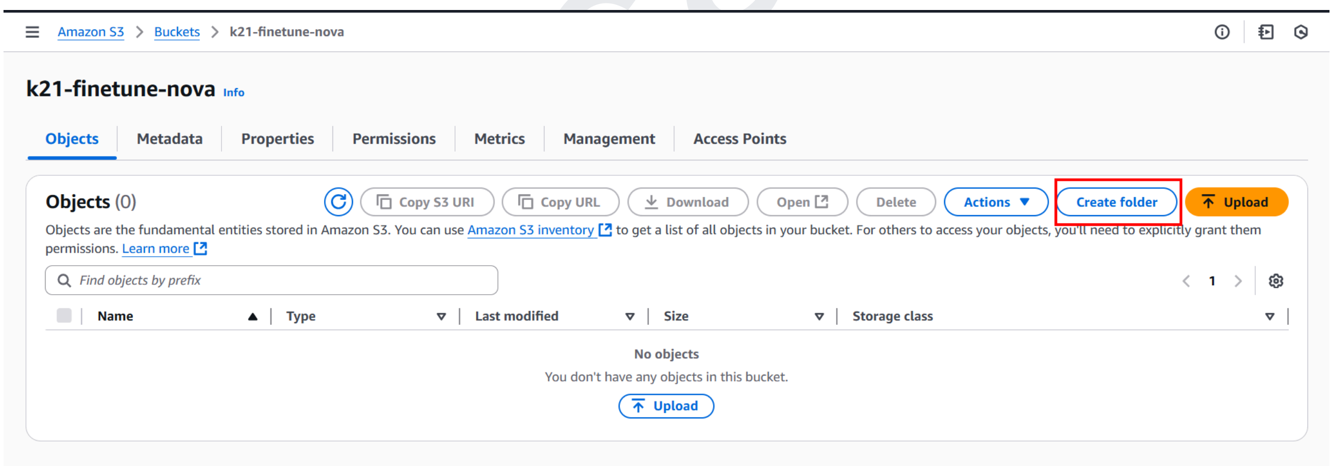
7. Scroll down, let everything rest by default, and click on **Create Bucket.**



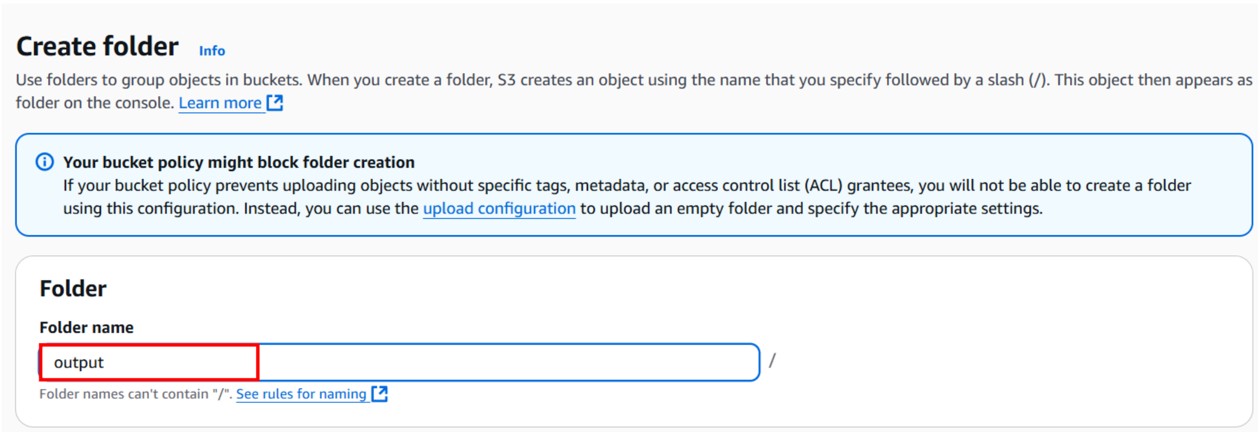
8. In this screen, you shall see that the bucket has been created and is available for use.

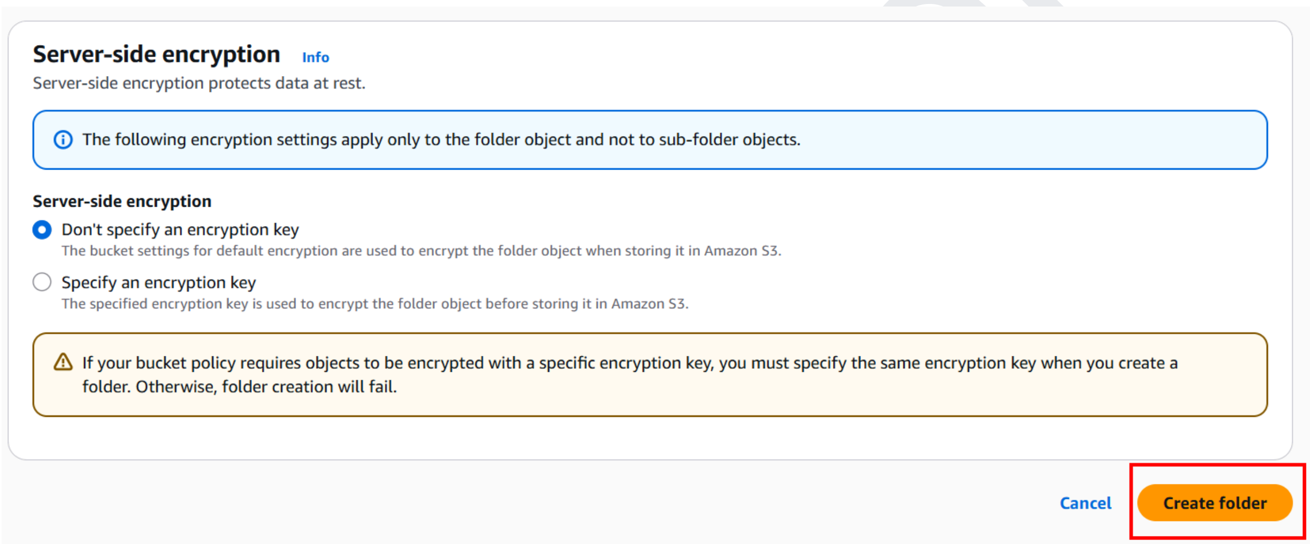


9. Here, we will create a Folder. You can click on the **bucket** → create folder, named as output.

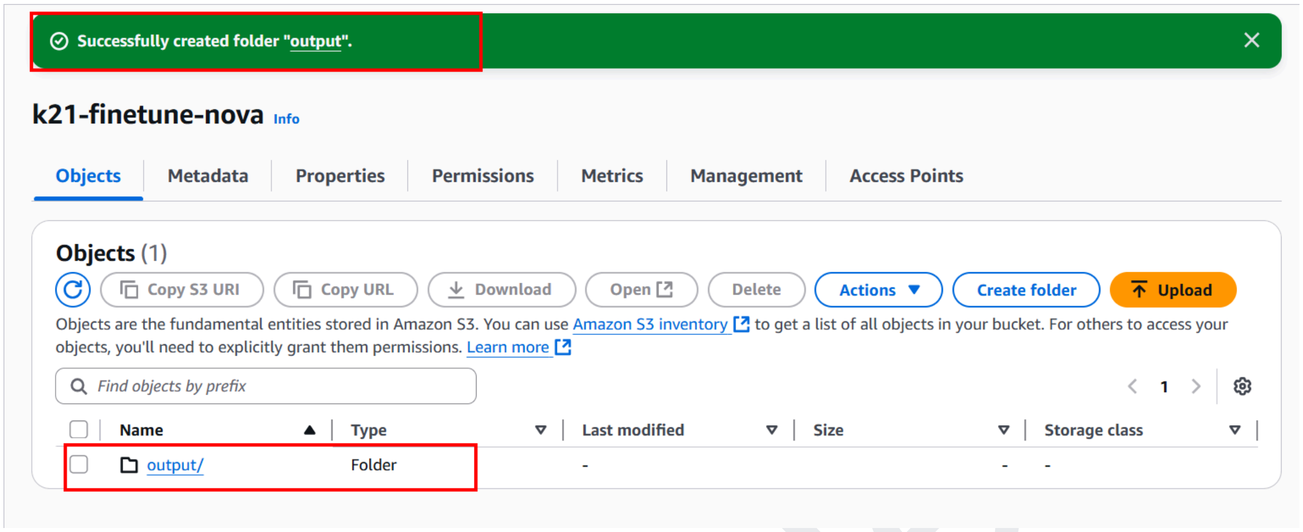


10. Now, provide the Name of the folder as output and click on create folder.

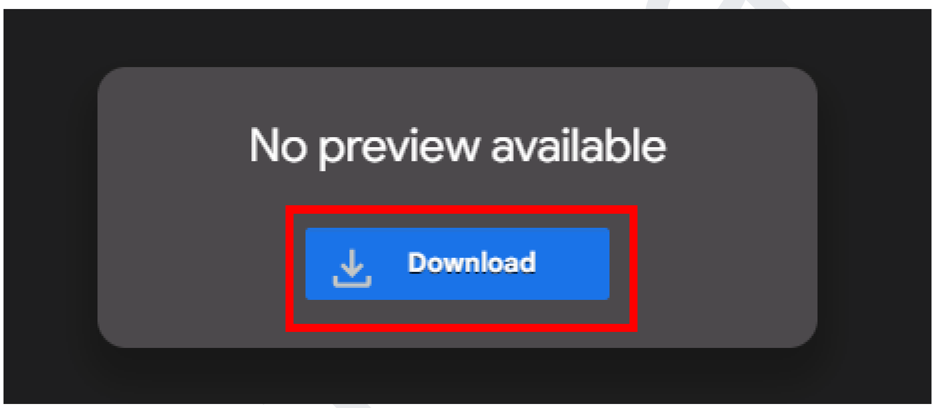




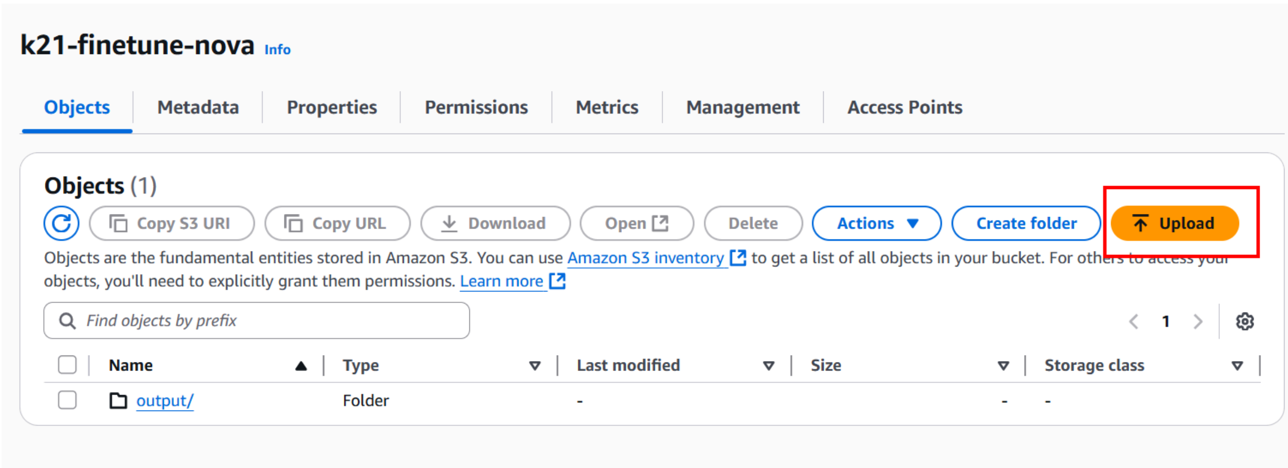
11. We have successfully created a folder inside our bucket to store output.



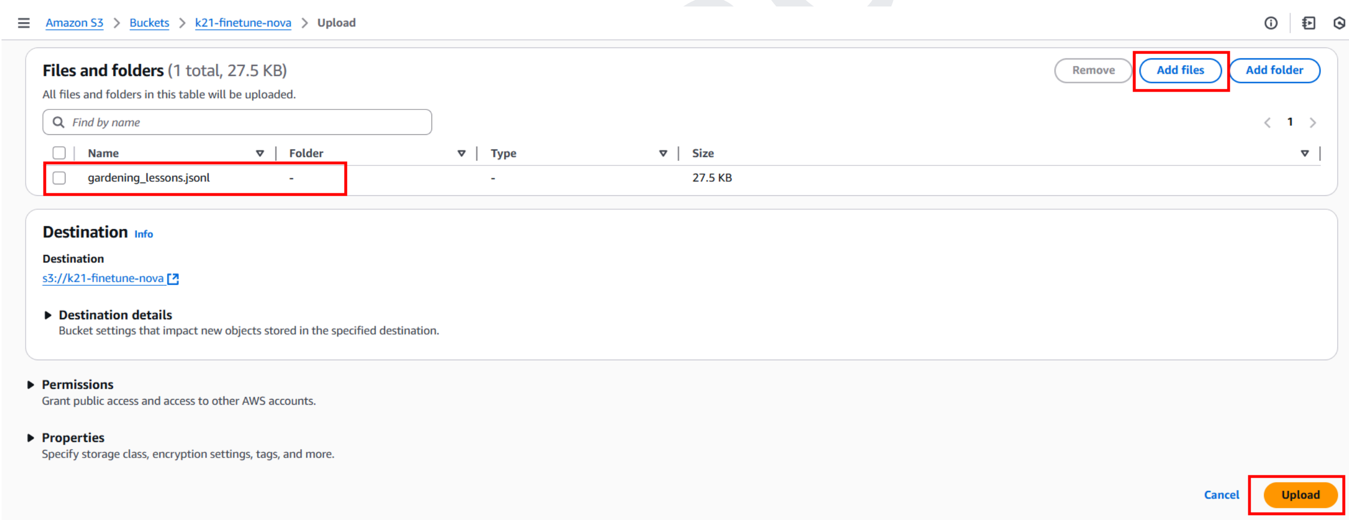
12. Now let's upload a file "gardening_lessons.jsonl" inside our bucket. Download the file here: [gardening_lessons.jsonl]



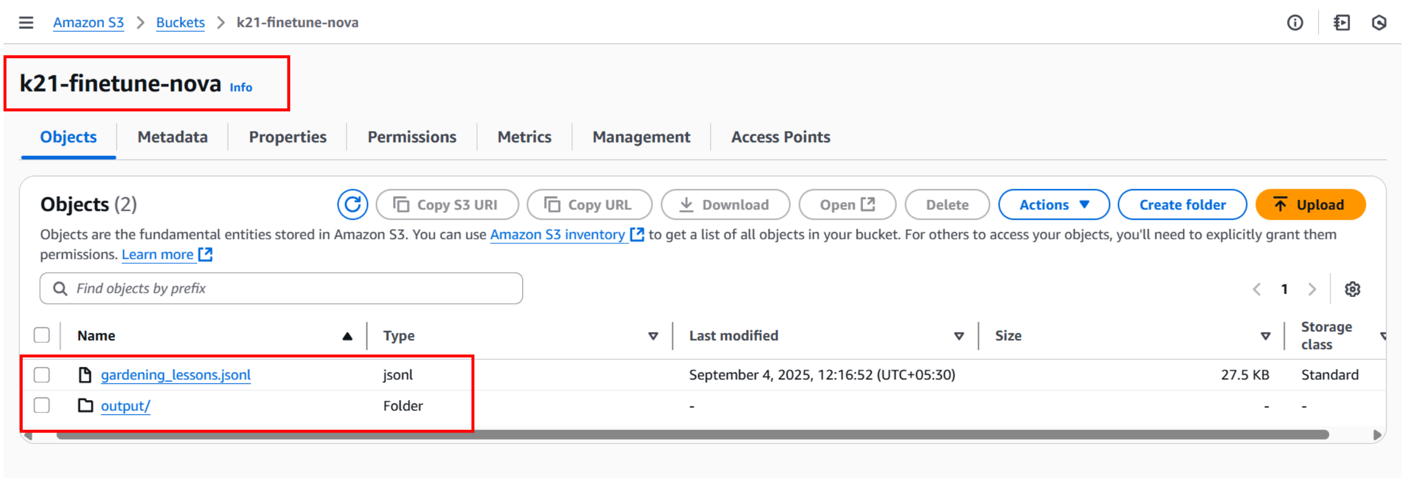
13. Now go to the bucket and click on Upload.



14. Now, we will upload the object gardening lesson to our bucket. Click on Add file, select the same file we downloaded earlier, and select upload.



15. You can see the file successfully uploaded to your bucket.



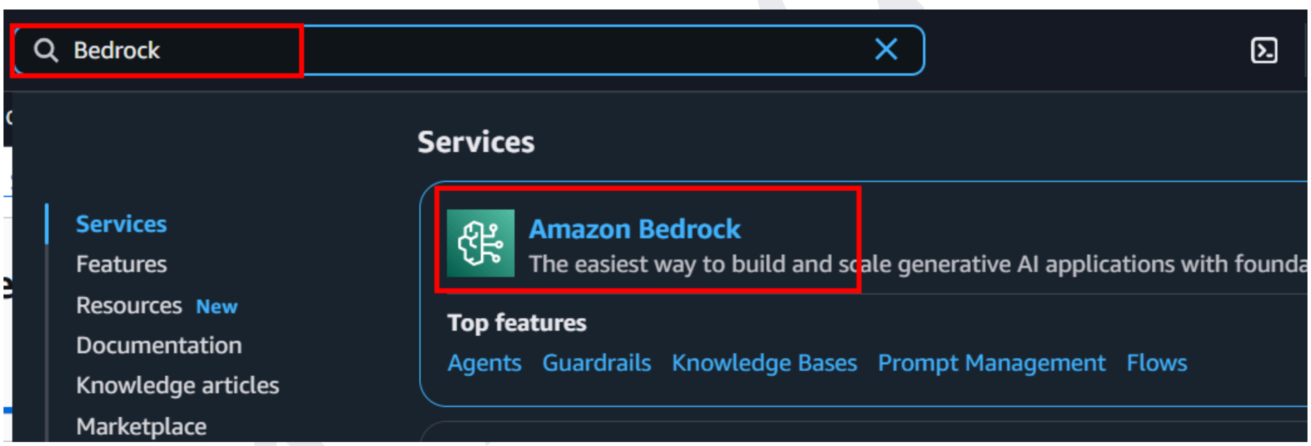
We have successfully uploaded an object and a folder to the bucket we will be using in the next steps.

7. Create a Bedrock Custom Model with Fine-Tune

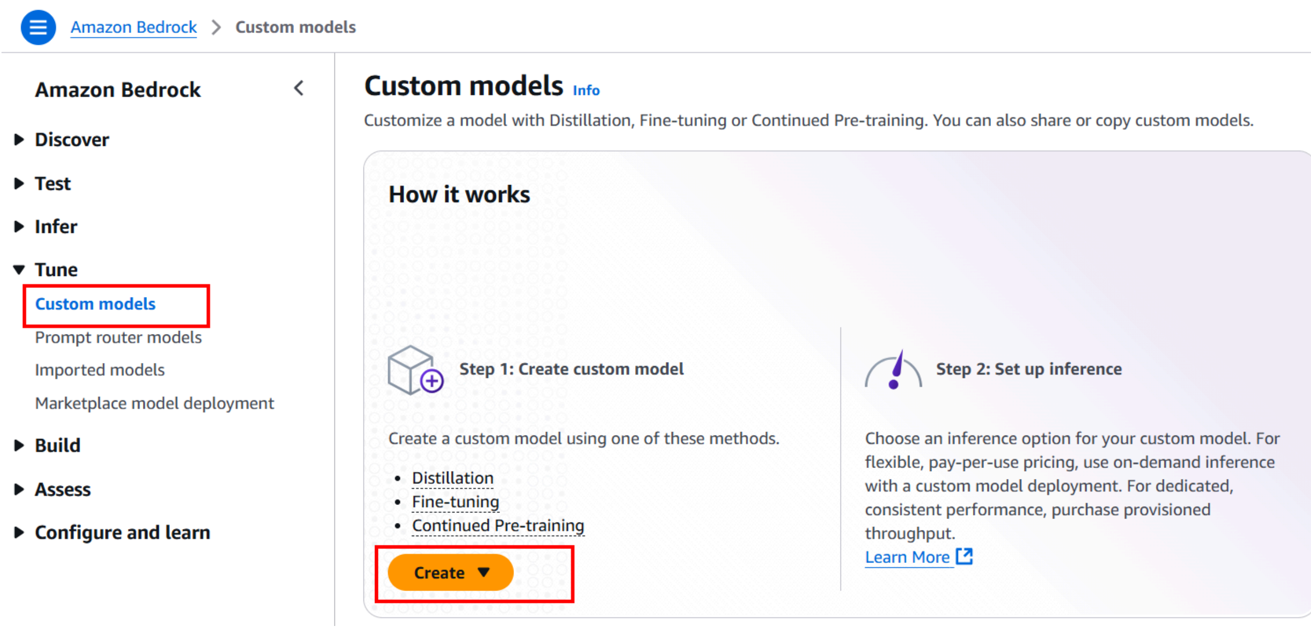
In this step, we'll fine-tune a custom model in Amazon Bedrock using a gardening example. This step will allow us to adapt a pre-trained foundation model (FM) to recognize and respond to specific queries related to gardening, such as plant care, pest control, and soil management.

Fine-tuning is essential because the **generic pre-trained models** may not be optimized to understand or provide accurate answers about gardening. By fine-tuning the model with **gardening-specific data**, we ensure it can provide more relevant and accurate responses tailored to gardening-related tasks, improving its usefulness for real-world applications.

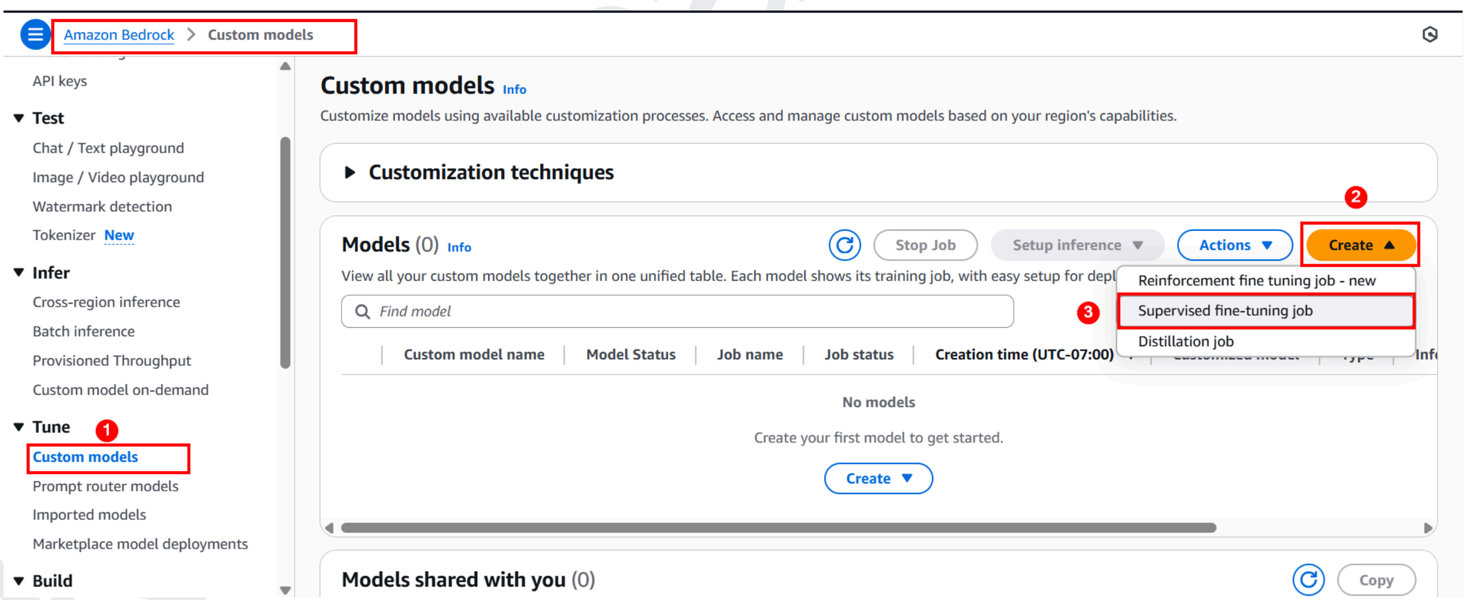
1. In the **AWS Console**, in the search bar, type "**Amazon Bedrock**" and select it. (We are working in the Northern Virginia region)



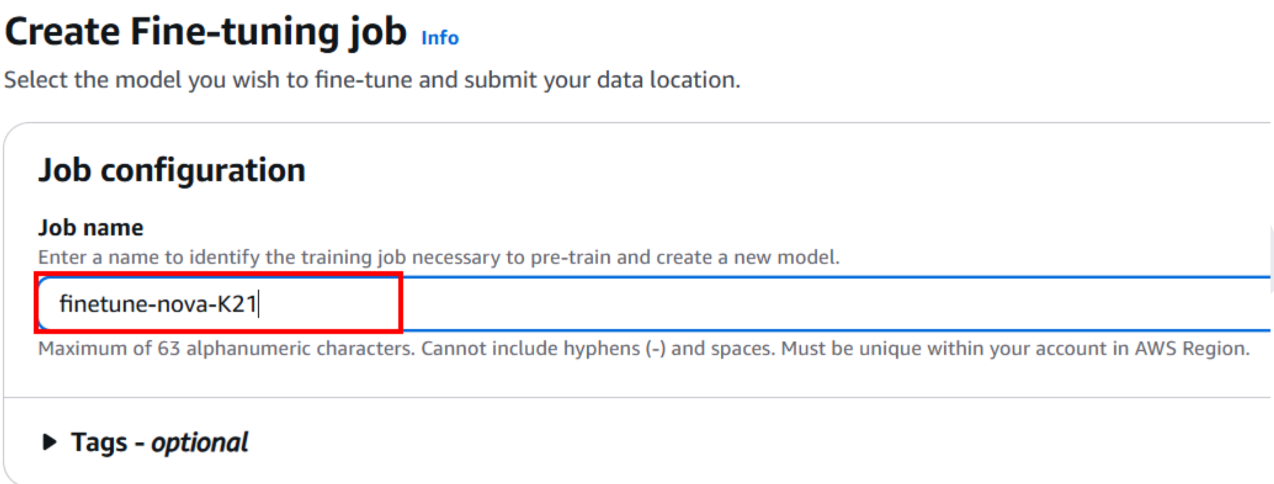
2. From the left navigation pane, choose Custom models under Foundation models.



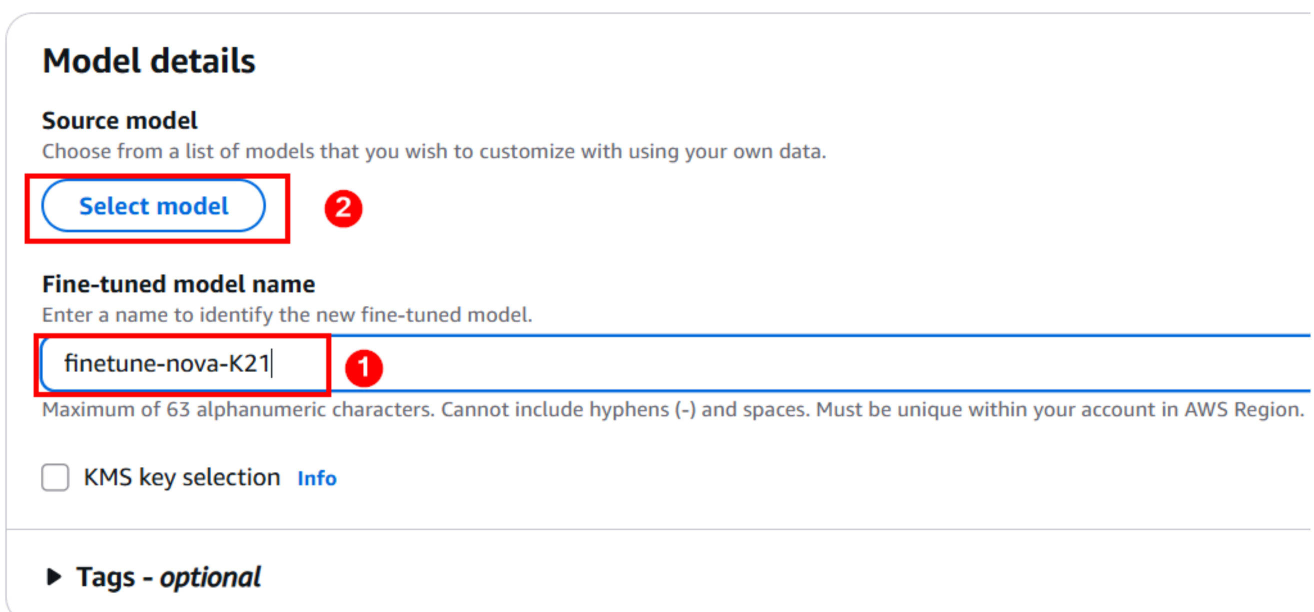
3. In the Models tab, choose a **Supervised fine-tunin job** depending on the type of model you want to train, and then click on Create.



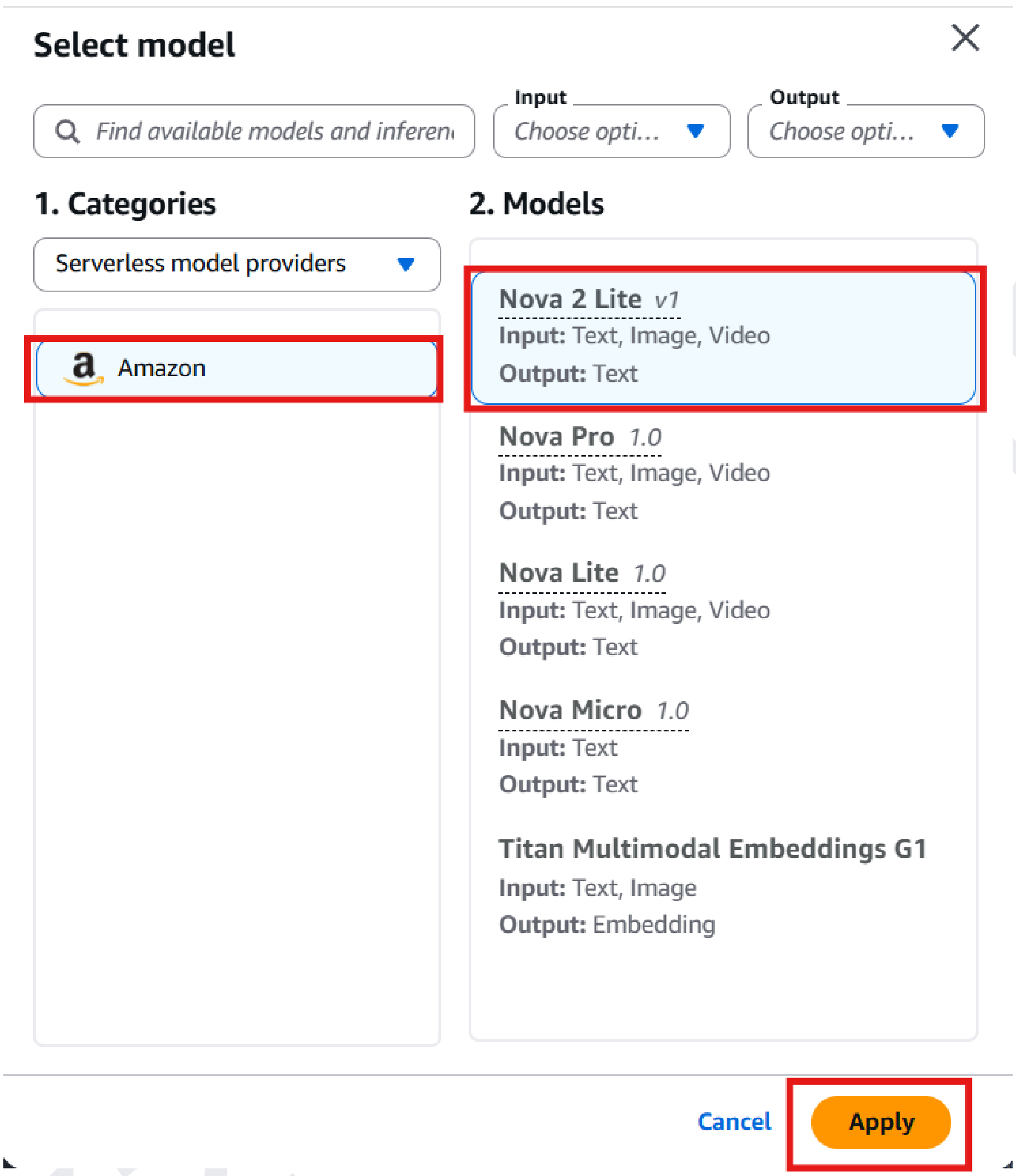
4. Now provide the name of the Job, whatever you want, we are giving it the name “finetune-nova-K21”.



5. In the Model details section, we will choose the model that you want to customize with your own data and give your resulting model a name.



6. From the list of AWS models, we will be selecting the Nova 2 Lite model and clicking on Apply.



7. In the Hyperparameters section, keep everything as it is.

Hyperparameters Info

Epochs

The maximum number of iterations through the entire training dataset.

2

Enter an integer between 1 and 5.

Batch size

The number of samples processed before updating model parameters.

1

Learning rate

Multiplier that influences the learning rate at which model parameters are updated after each batch.

0.00001

Enter a float value between 0.000001 and 0.0001

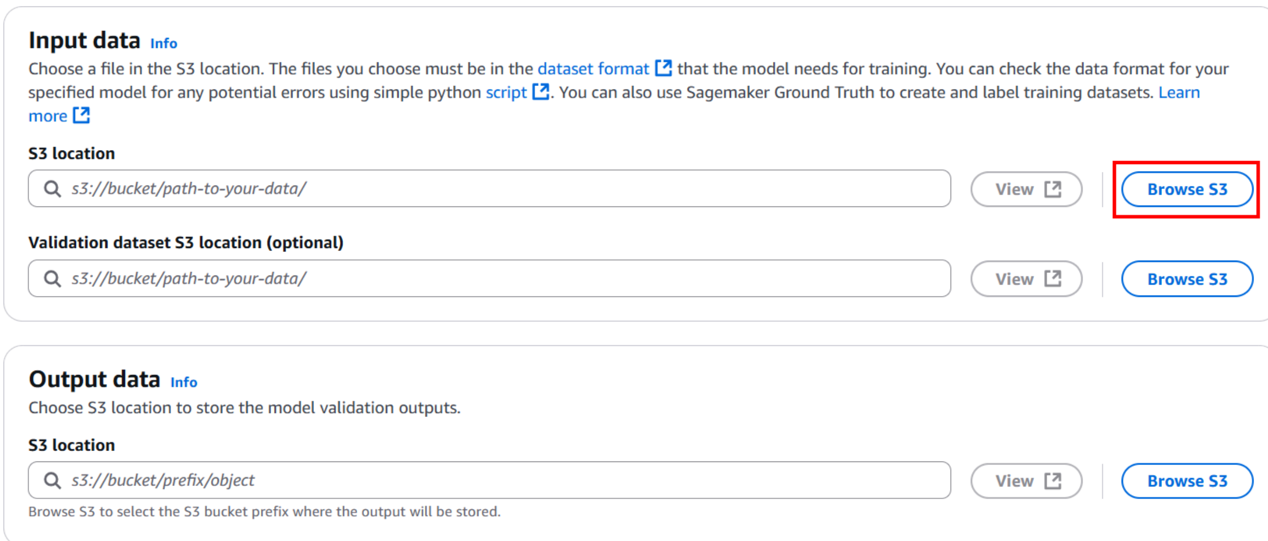
Learning rate warmup steps

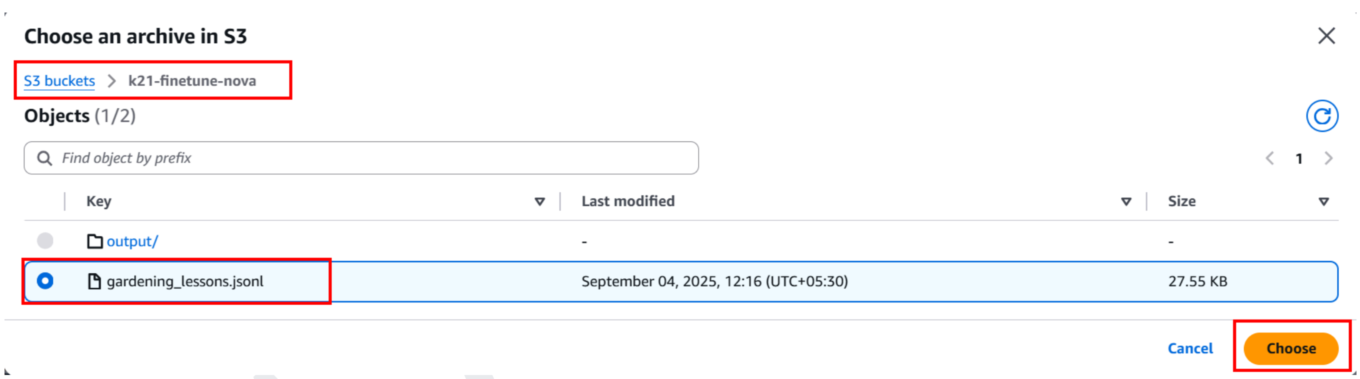
Number of iterations over which learning rate is gradually increased to the initial rate specified.

10

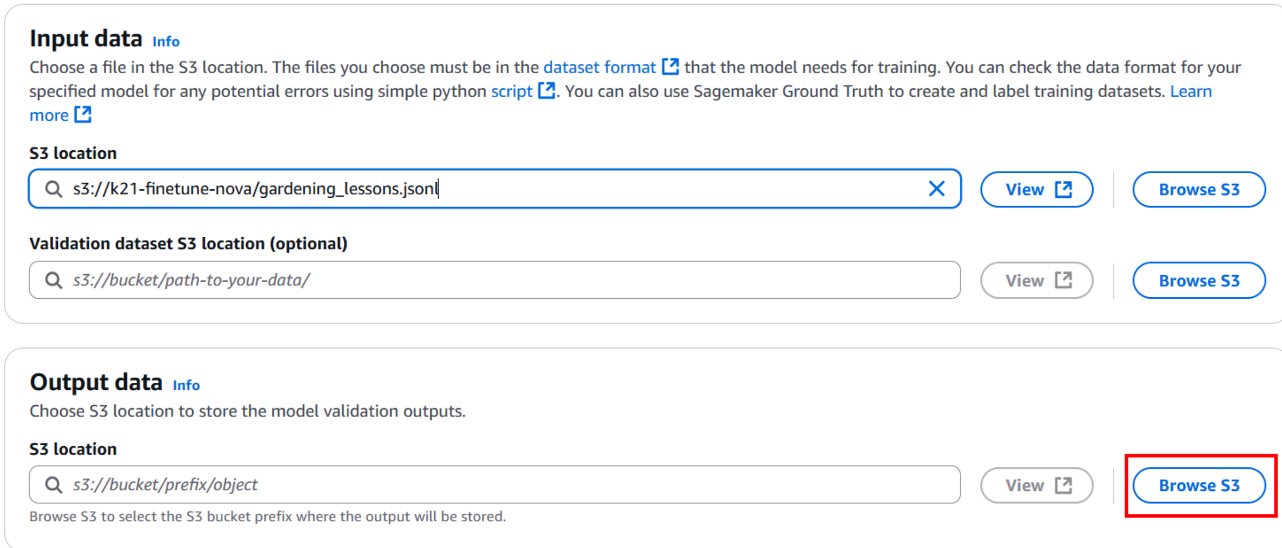
Enter an integer between 0 and 20.

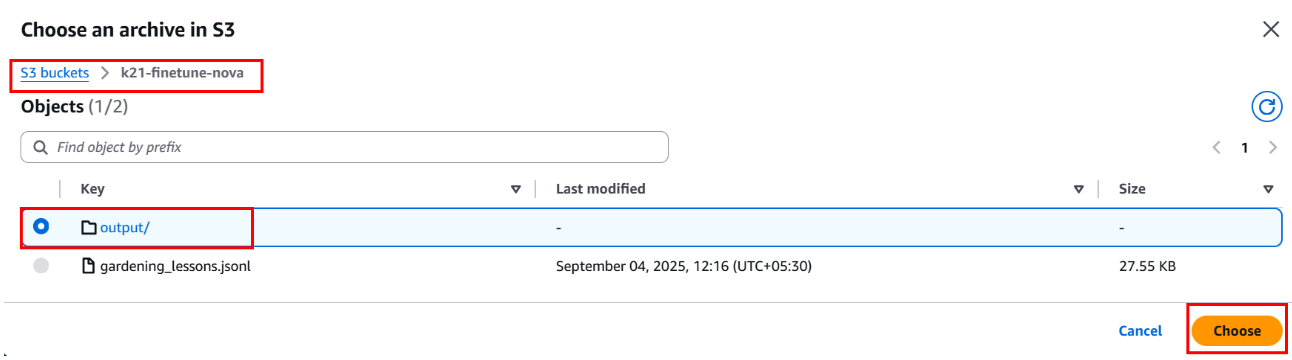
8. In the Input data section, select the S3 bucket we created in the previous step and select the location of the training dataset file (gardening lessons) and the validation dataset file.



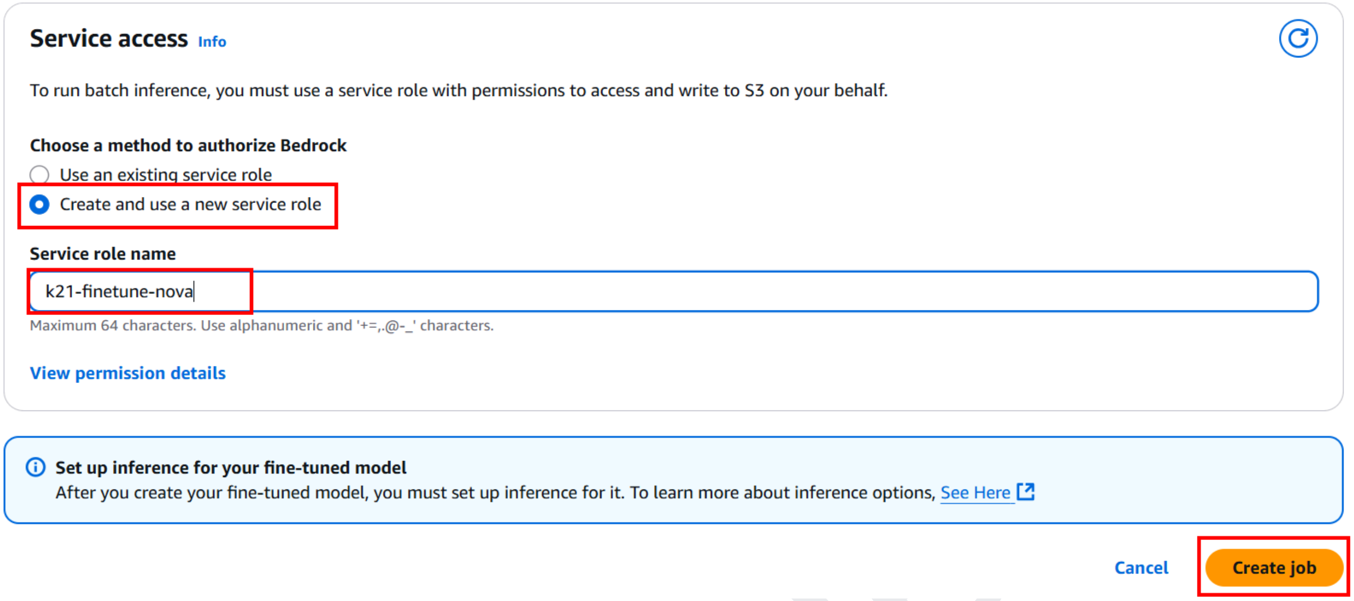


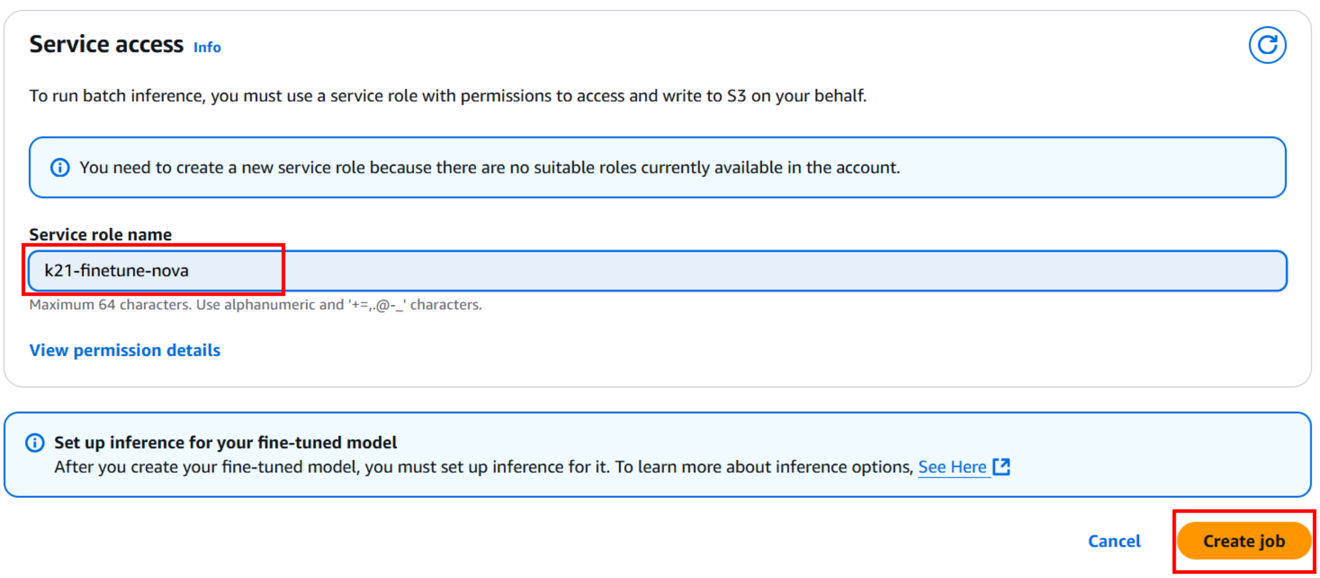
9. In the Output data section, enter the Amazon S3 location where Amazon Bedrock should save the output of the job. Amazon Bedrock stores the training loss metrics and validation loss metrics for each epoch in separate files in the location that you specify.



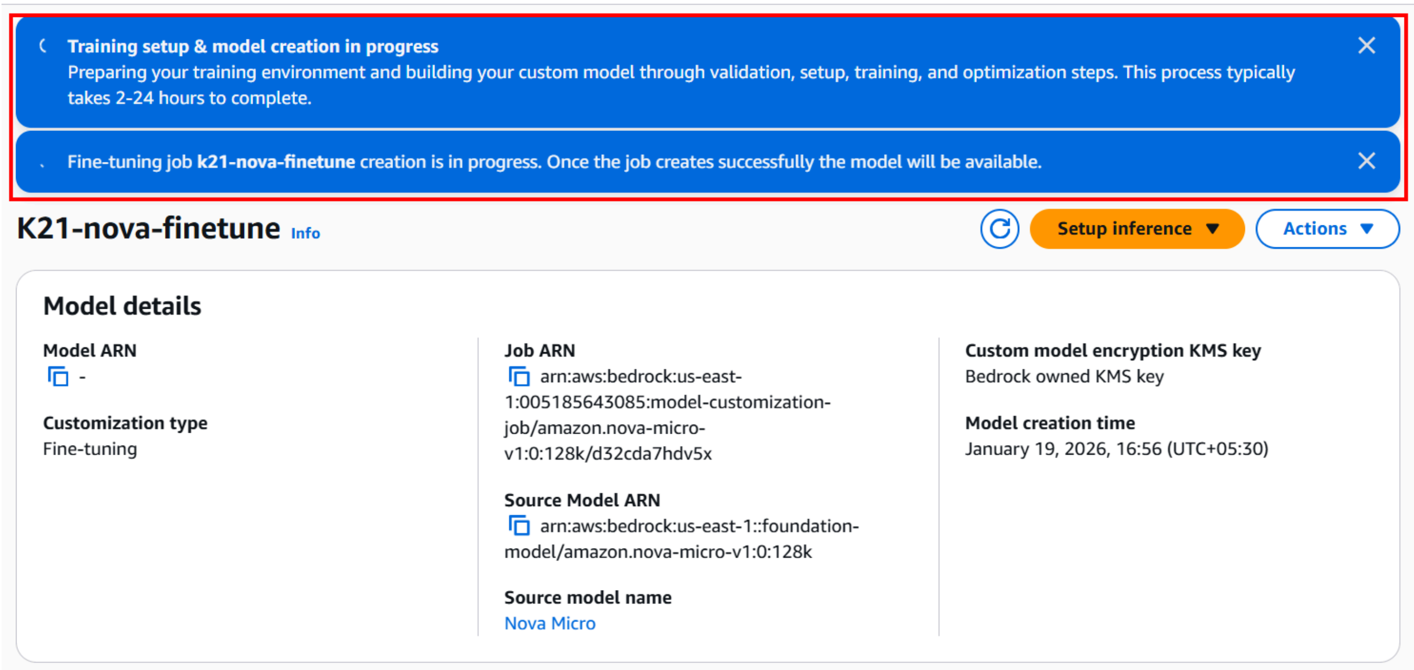


10. In the Service access section, create and use a new service role – Enter a name for the service role. Now choose Create job to begin the job.

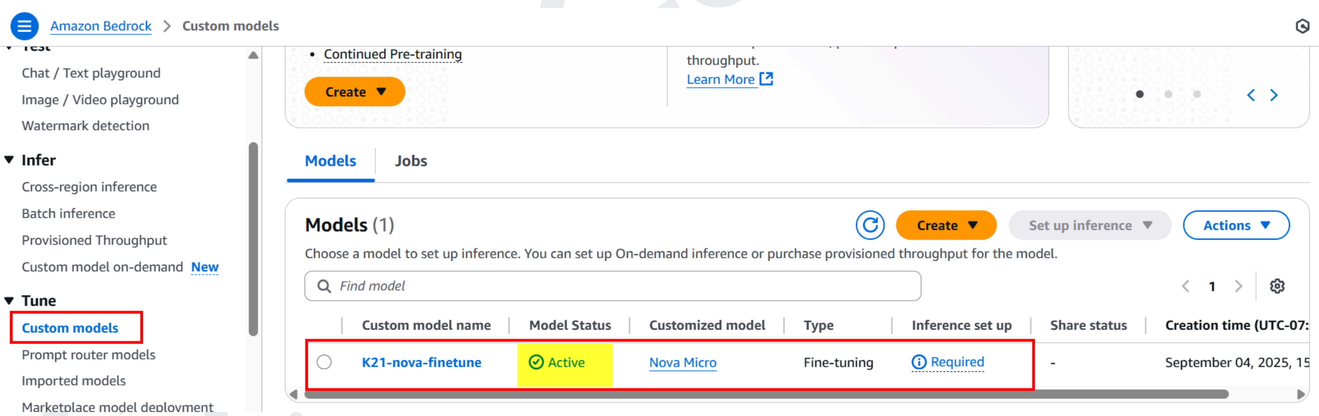

Service access Info
To run batch inference, you must use a service role with permissions to access and write to S3 on your behalf.
Choose a method to authorize Bedrock
Use an existing service role
Create and use a new service role
Service role name
k21-finetune-nova
Maximum 64 characters. Use alphanumeric and '+=,.@-_' characters.
View permission details
Set up inference for your fine-tuned model
After you create your fine-tuned model, you must set up inference for it. To learn more about inference options, See Here
Cancel
Create job


Service access Info
To run batch inference, you must use a service role with permissions to access and write to S3 on your behalf.
You need to create a new service role because there are no suitable roles currently available in the account.
Service role name
k21-finetune-nova
Maximum 64 characters. Use alphanumeric and '+=,.@-_' characters.
View permission details
Set up inference for your fine-tuned model
After you create your fine-tuned model, you must set up inference for it. To learn more about inference options, See Here
Cancel
Create job

11. Now our job is in the Progress state, it will take a few minutes to hours to Active status.



12. It will take from a few hours to 2 days.



Note: For us, it took 4 hours to be in Active Status.

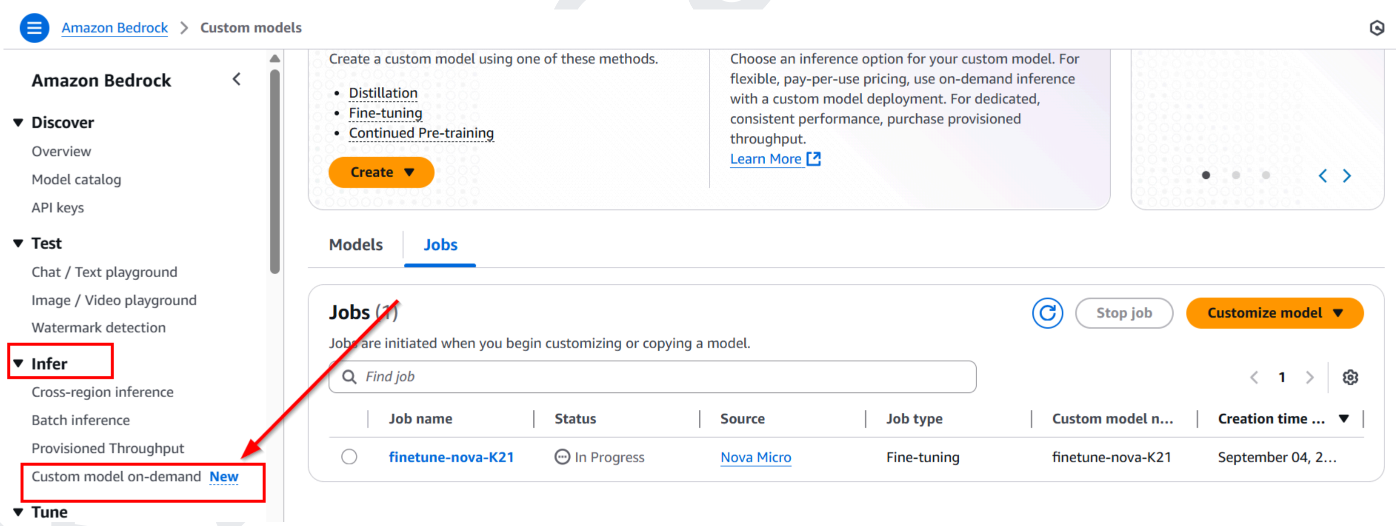
8. Create a Custom Model on Demand with Inference

This step involves **creating a custom AI model** by fine-tuning a pre-trained **foundation model** (FM) using **Amazon Bedrock** and performing **inference**. Inference refers to the process by which the model makes predictions or generates outputs based on new data or queries.

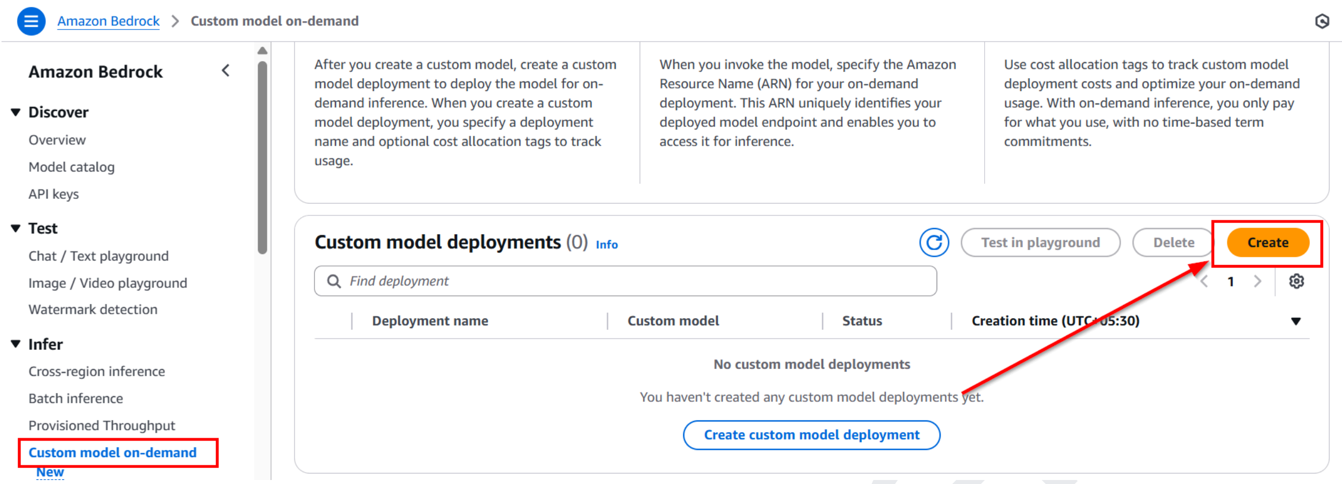
We need this step to adapt a pre-trained model to our **specific use case** (e.g., gardening, customer service). Fine-tuning with your own data ensures that the model performs accurately and is tailored to your unique needs.

Inference allows you to test and use the model in real-world scenarios, making it practical for actual tasks like answering queries, generating content, or automating workflows. This step is essential for creating **relevant, high-performance models** that can deliver meaningful results.

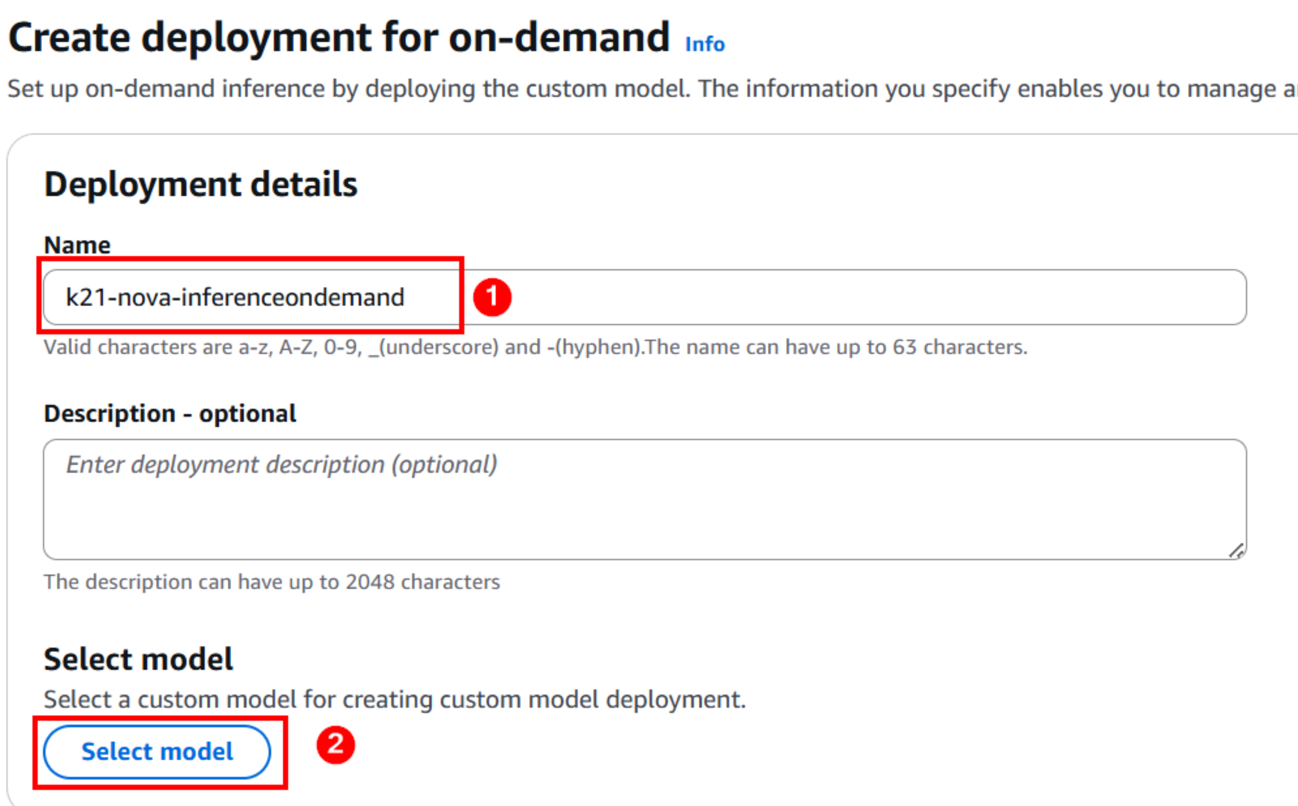
1. Now in the same Bedrock Dashboard, scroll down and under the infer select the custom model on demand.



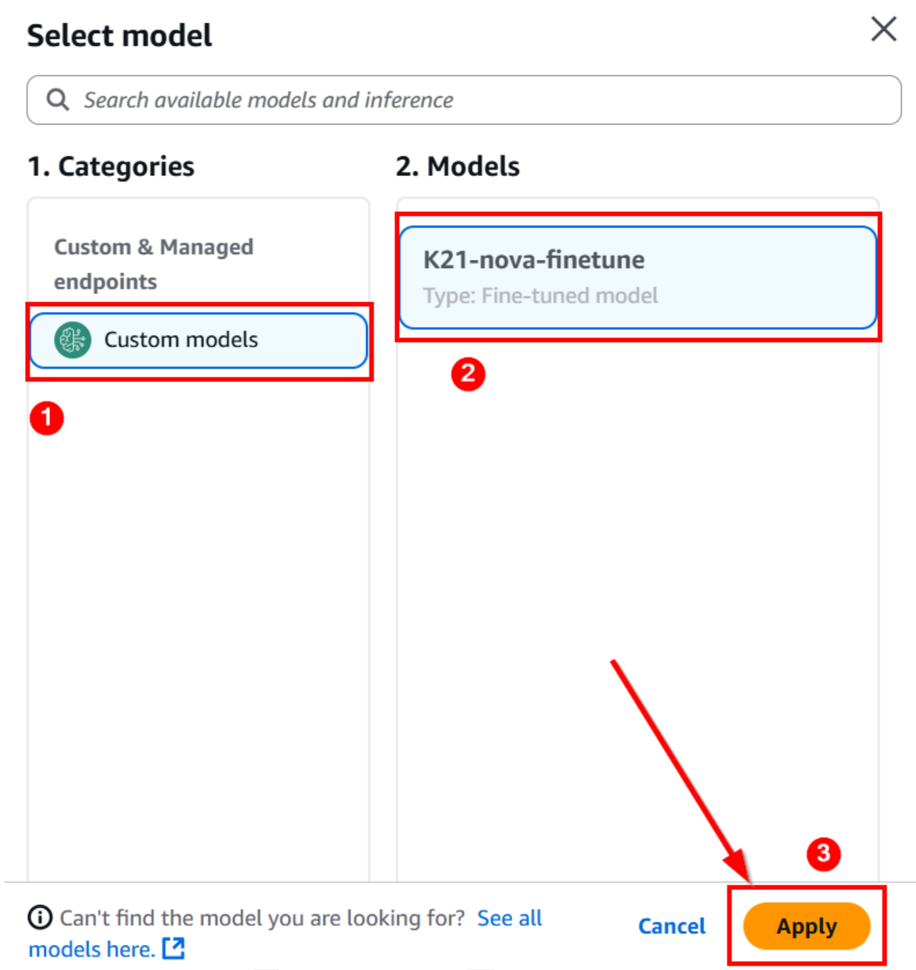
2. Now we will be creating a custom model for inference.



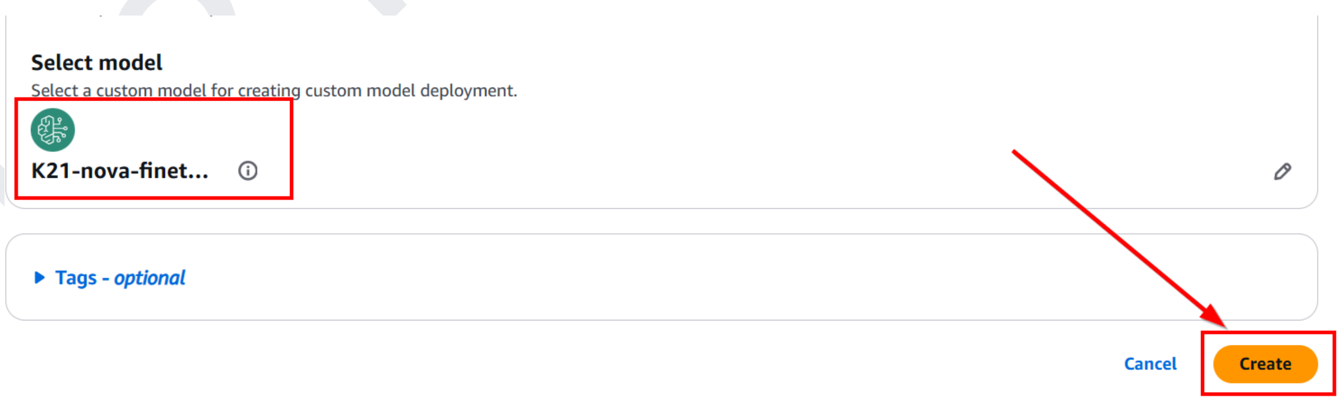
3. Provide the name of your deployment, "**K21-nova-inferenceondemand**," and click on the select model.



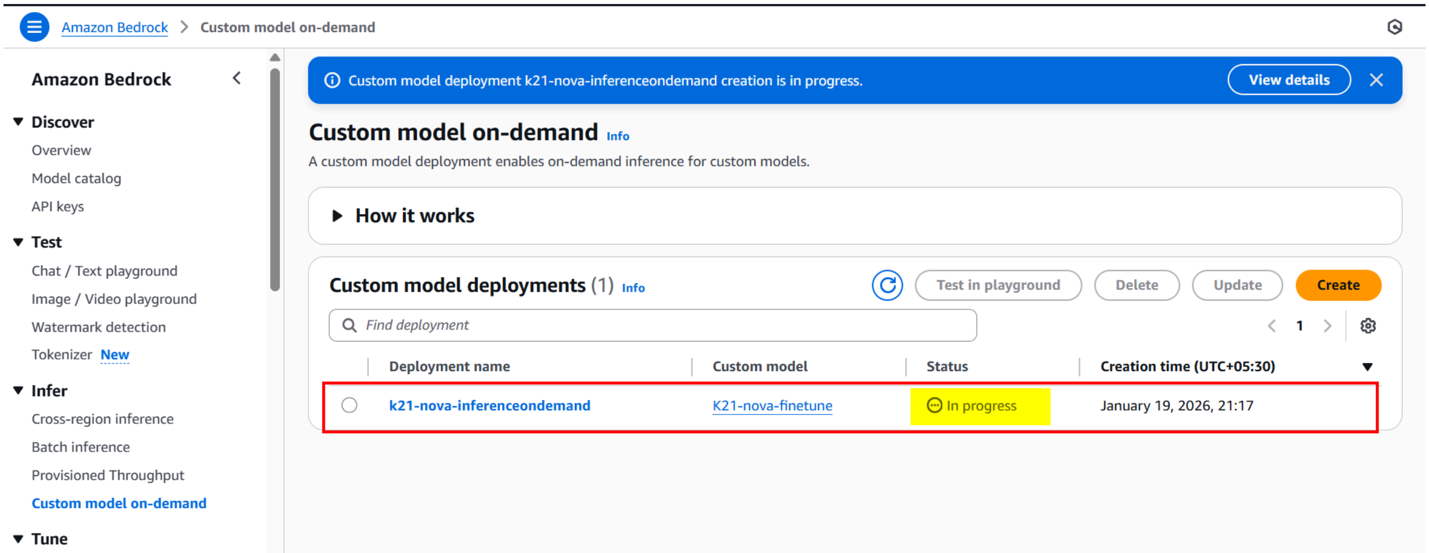
4. In this step, we will be selecting the custom model that we created in the last step and clicking on Apply.



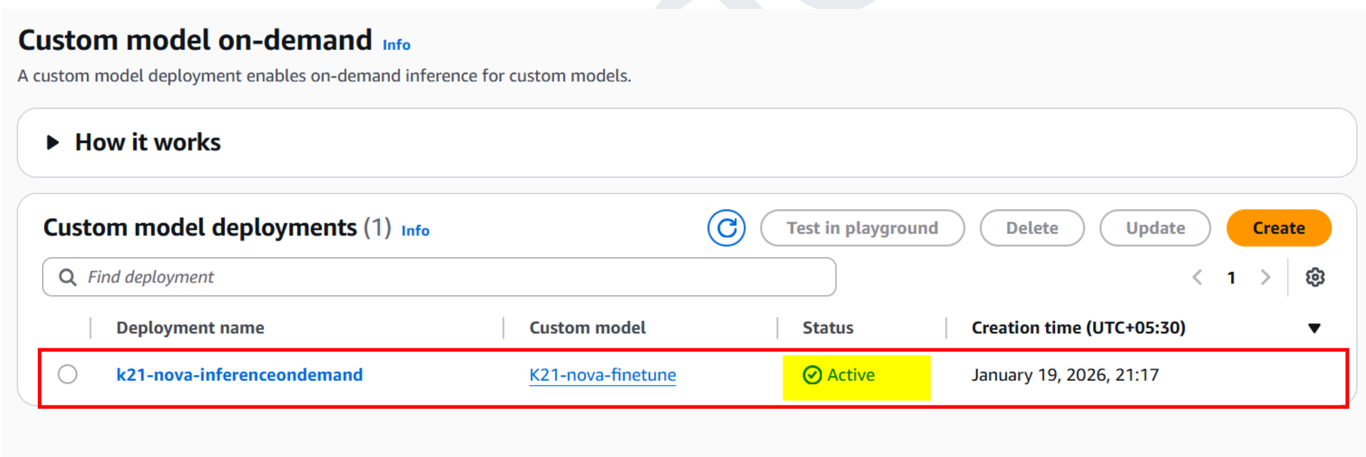
5. Now our model is selected, click on Create.



6. Our custom model is in the Progress state. It will take some time to be active.



7. Now our Custom Model is Active; let's move on to the next Step.



Note: For us, it took 5-10 minutes to be in Active Status.

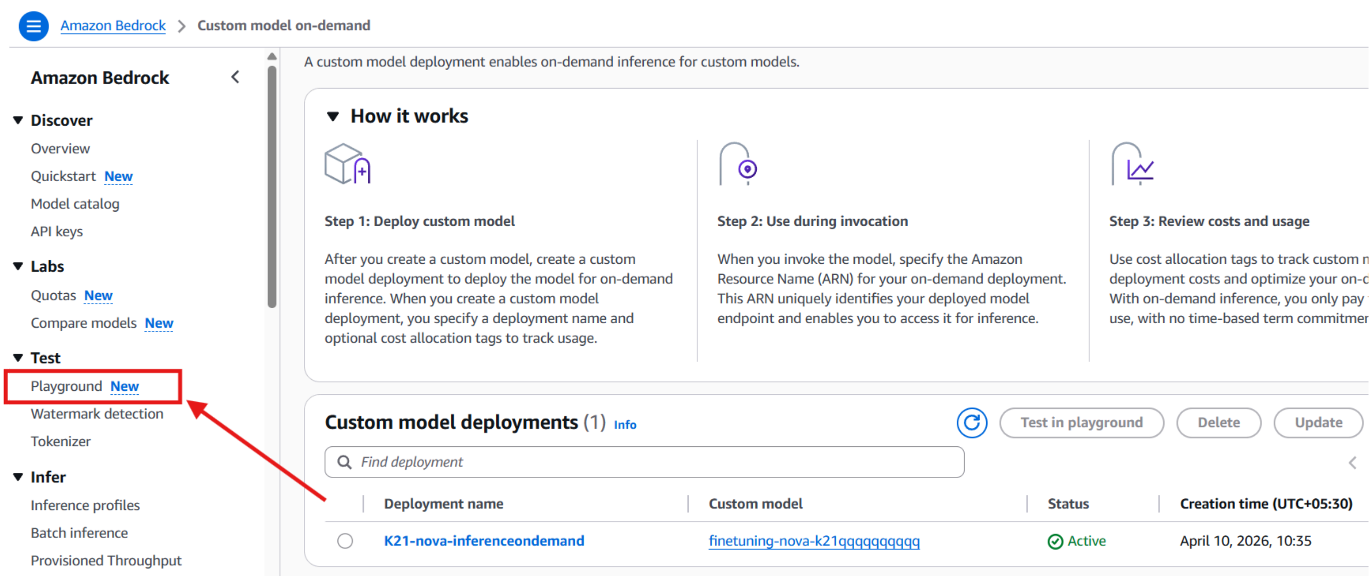
9. Testing Your Custom Model in Amazon Bedrock

After fine-tuning your custom model in Amazon Bedrock, testing it is a crucial step to ensure it performs as expected.

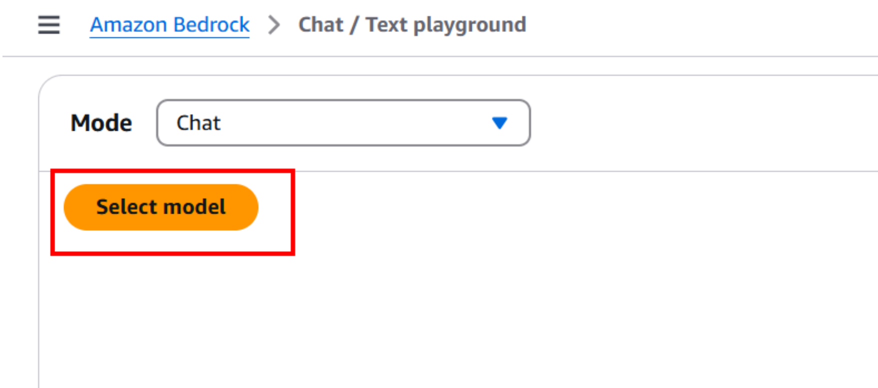
Testing involves providing sample input data and evaluating the model's output for accuracy, relevance, and efficiency. This helps verify that the model can handle real-world data and generate correct results.

During testing, you may need to run inference tasks through Amazon Bedrock's APIs and review the results, making adjustments or retraining the model if necessary to improve its performance before deployment.

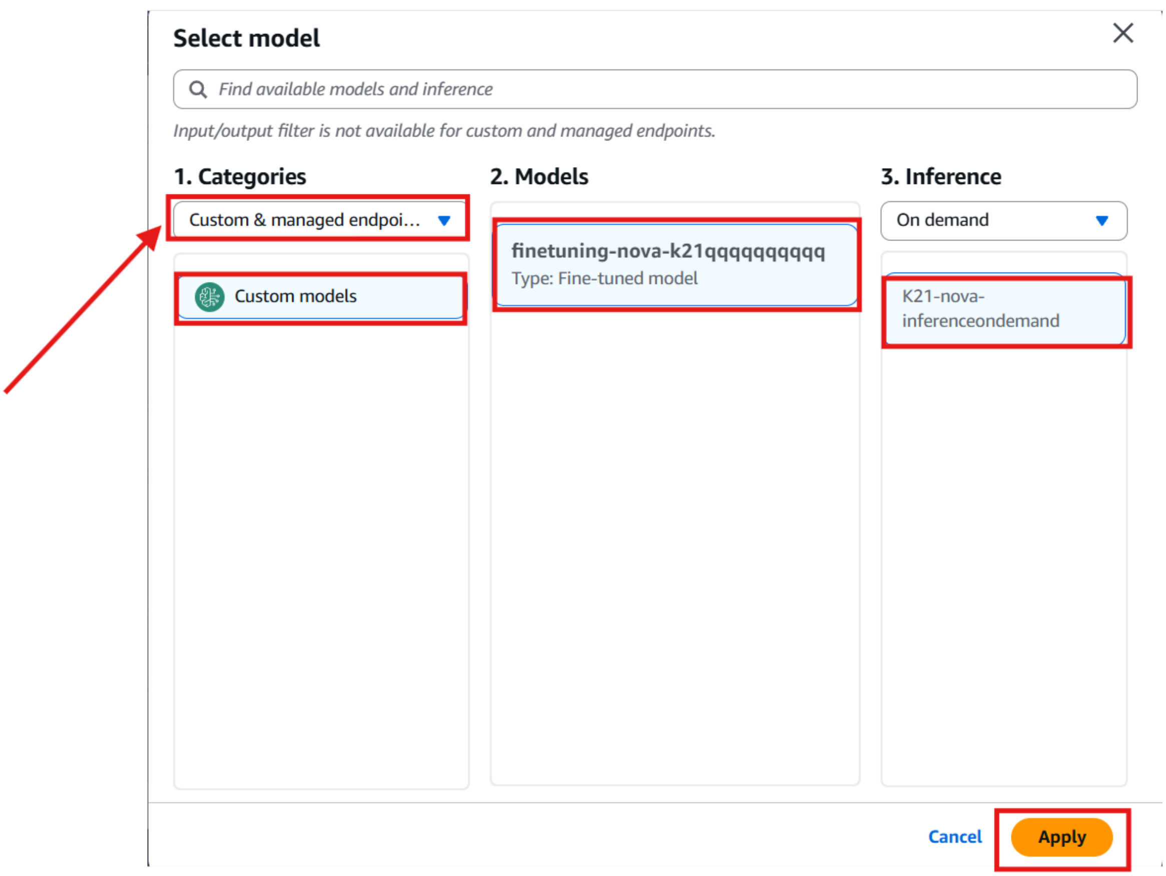
1. In the Same Bedrock Dashboard, go to test and click on Playground.



2. Select Model

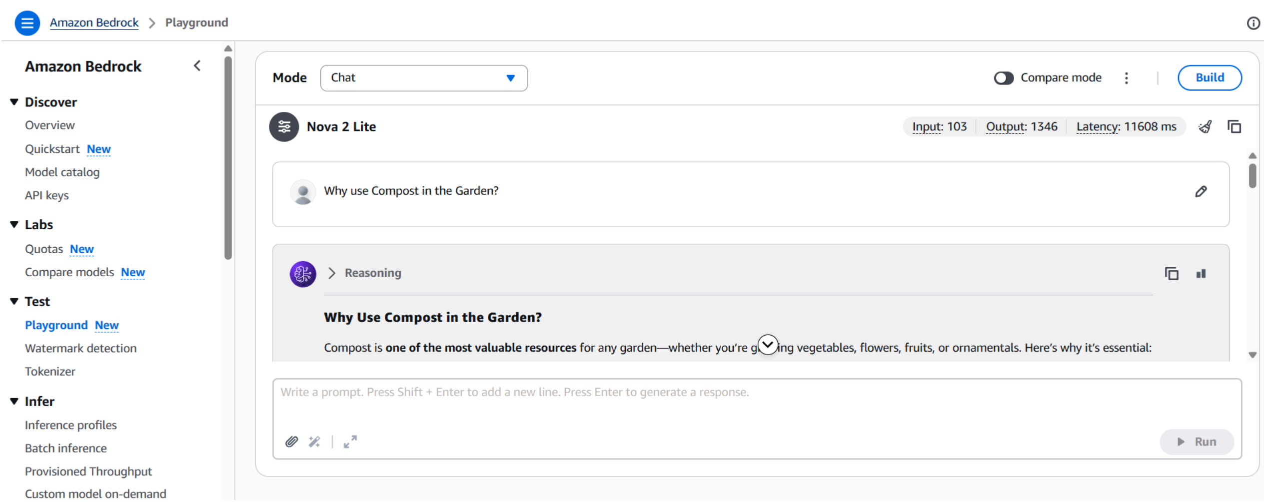


3. We need to choose from the custom model, the model we created earlier, and then inference.

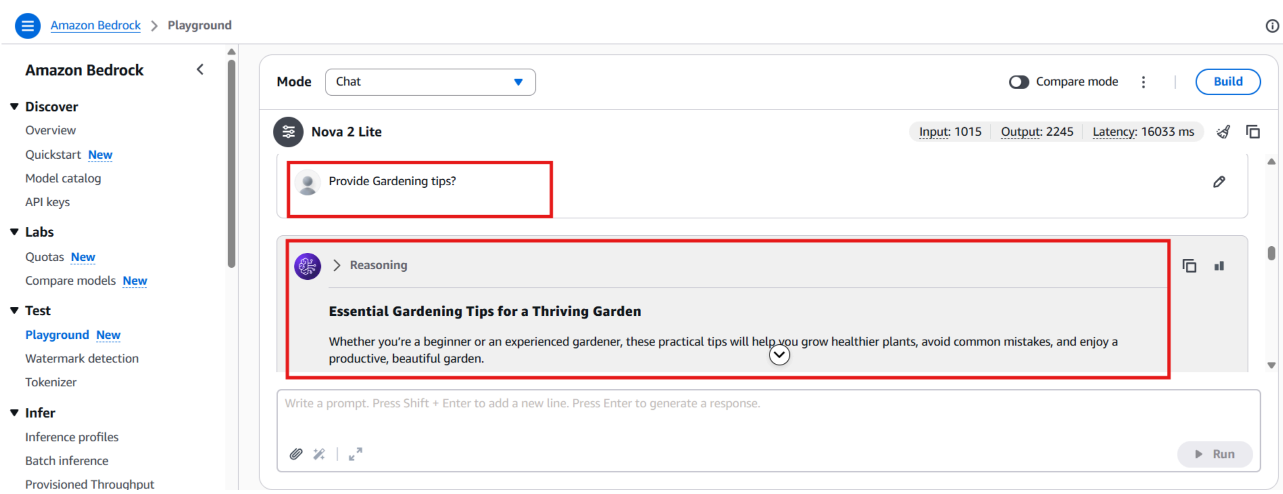


4. Now you can ask any questions about Gardening in this model.

   Q: Why use Compost in the Garden?



5. Now, let's ask one more question: Provide Gardening tips?



Our custom model is answering questions as expected. You can ask more questions about it as well.

In this lab, we worked with a use case where the model was required to respond to gardening-related queries. Initially, the foundation model did not have the necessary knowledge. To address this, we fine-tuned the model using our custom sample data. After fine-tuning, we deployed the model and successfully obtained the expected outputs for gardening queries. This process demonstrated how to enhance a foundation model's ability to handle domain-specific tasks.

10. Optional (Additional Scenarios for Further Exploration)

Fine-tuning is strongest when you're baking in behavior, format, tone, or classification — not facts (facts are a RAG job). Each dataset below teaches a repeatable enterprise behavior, so learners get a clear before/after: the base model rambles, the fine-tuned model returns the exact structured output. AWS's own flagship example is intent classification (Nova Micro 41% → 97% accuracy).

This follows the same flow as the original lab (S3 → Fine-tune job → Custom model on-demand → Playground test → Clean up), repeated once per scenario. Steps 1–2 are done once; repeat Steps 3–6 for each dataset you want to fine-tune.

Note: The following steps are optional and are not mandatory to complete this lab. These additional scenarios are provided for learners who would like to explore more custom model use cases.

As we already created an S3 bucket in the previous steps, you can reuse the same bucket for these scenarios. The detailed steps for creating and configuring a custom model have already been covered earlier, so in this section, we will only provide the high-level steps.

You can follow the same process used in the previous custom model creation exercise and apply it to different scenarios based on your requirements. Each custom model may take approximately 2 hours to 24 hours to create, after which you can test and validate the model.

The purpose of the previous steps was to show how to create a custom model for one sample scenario. You are not limited to that scenario; you can use the same approach to work with other scenarios as well.

📝 **Note on Additional Scenarios**

The six scenarios below have been added to this lab based on direct requests from many of you in our community. We heard that you wanted to see how the same fine-tuning workflow applies beyond a single use case — and across the industries you actually work in or are preparing for. Each scenario follows the exact same process covered in the core lab, applied to a different domain and dataset, so the steps will feel familiar as you work through them.

We hope these additional scenarios give you a broader view of what is possible with Amazon Bedrock fine-tuning and help you connect this skill to your own professional context. We will continue updating this lab as more use cases are requested - so keep the feedback coming in the community.

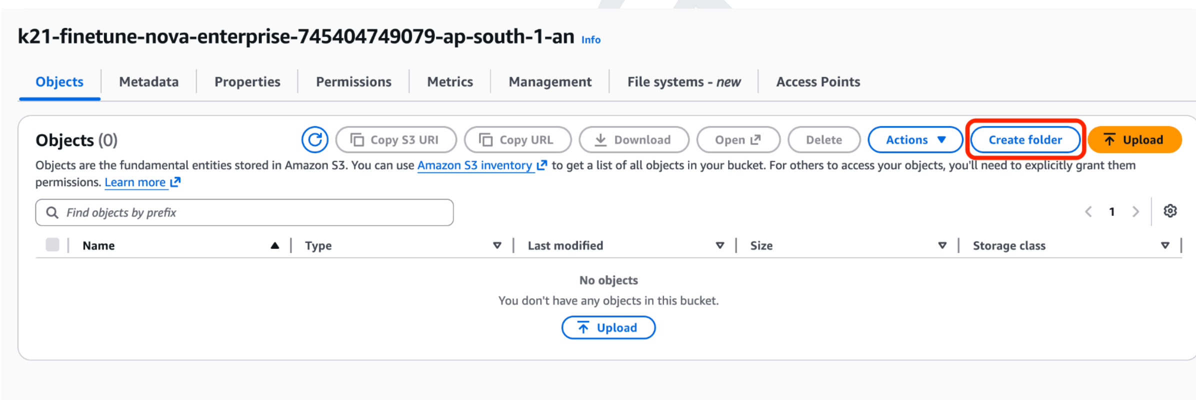
11. Create the S3 bucket (one time)

Note: Since we already created the S3 bucket in the previous step, you can reuse it for this exercise. Create separate folders within the existing bucket to organize the input data and output results required for the custom model workflow, or create a new bucket as well.

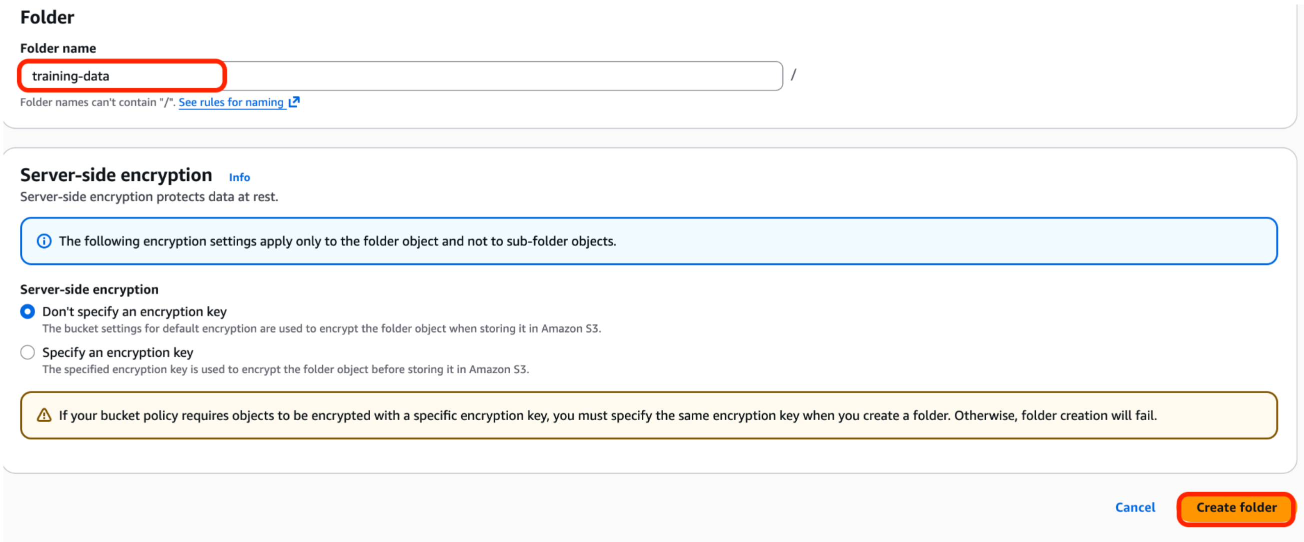
Download all the `.jsonl` files from the GitHub folder:

https://github.com/k21academyuk/AWS-GenAI_AIML/tree/Custom-Model-jsonl-files

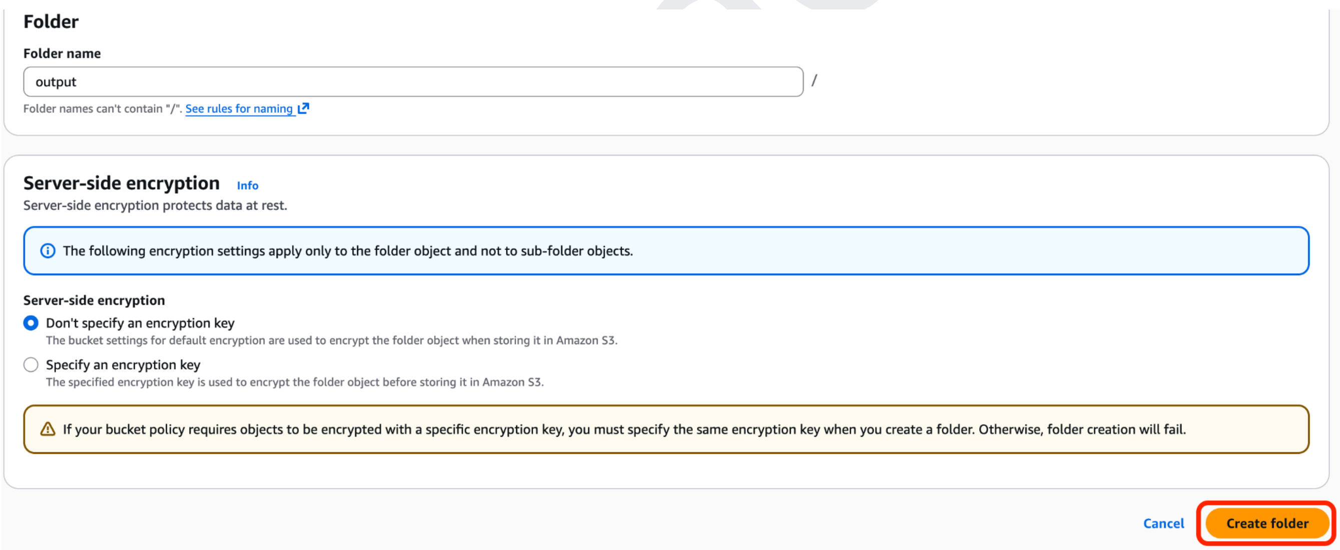
1. Inside your new bucket, create these folders (bucket → **Create folder**, type the name, **Create folder**):
    - training-data/
    - output/



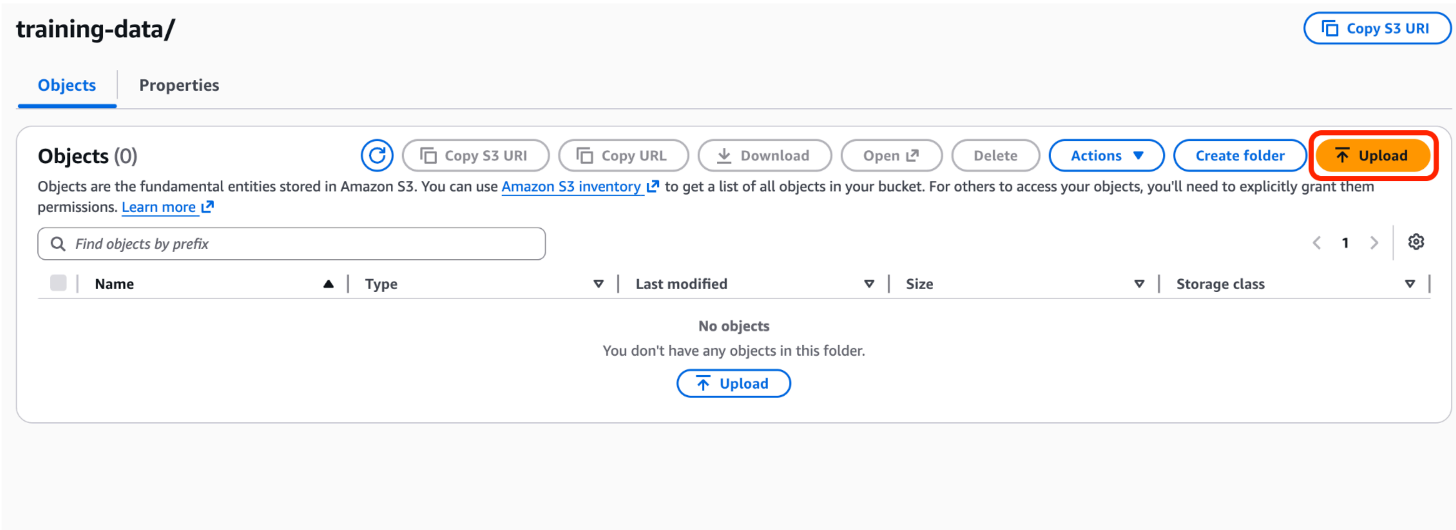
2. Enter `training-data` as the folder name, keep the default encryption settings, and click **Create folder**.



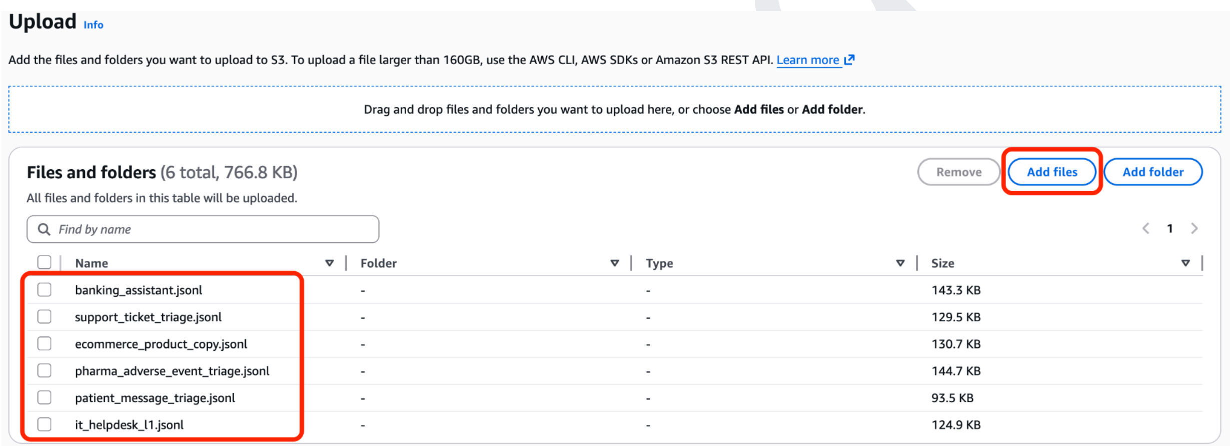
3. Create a second folder named Output.



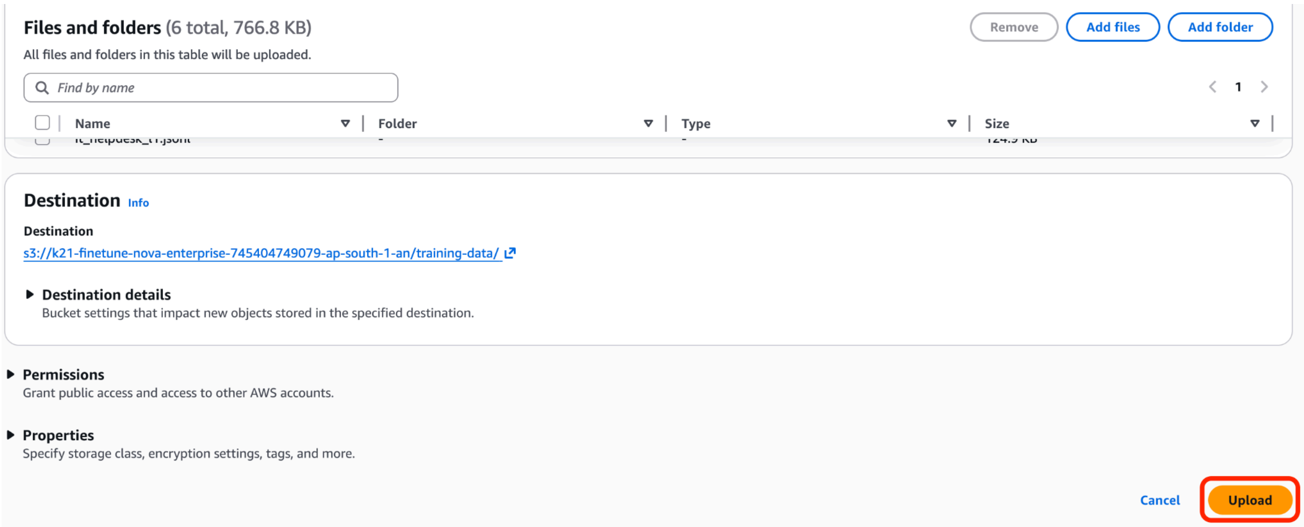
4. Open the **training-data** folder, click **Upload**, and upload your training dataset files.



5. Click **Add files**, then select all the `.jsonl` files and upload them here, one file at a time.



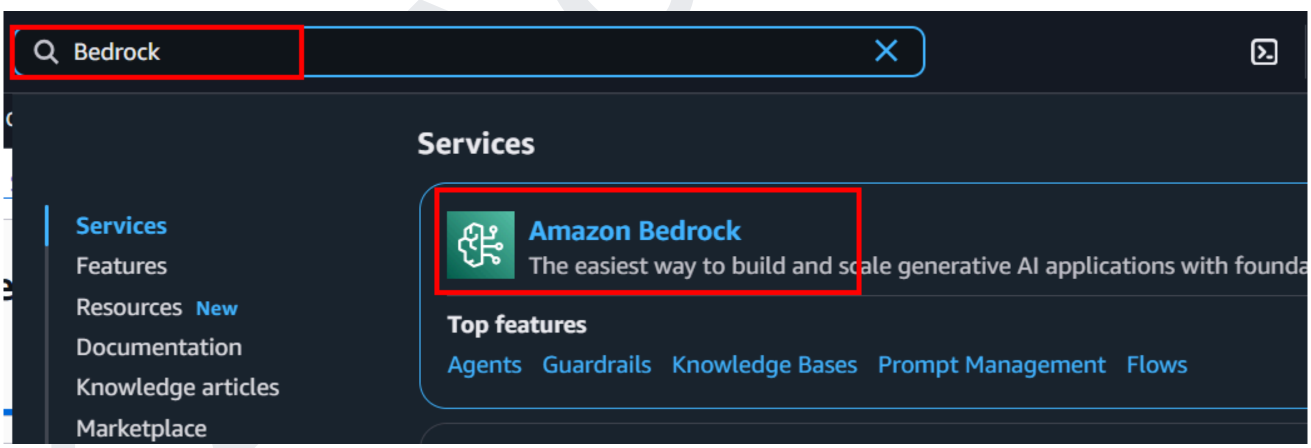
6. Then click Upload.



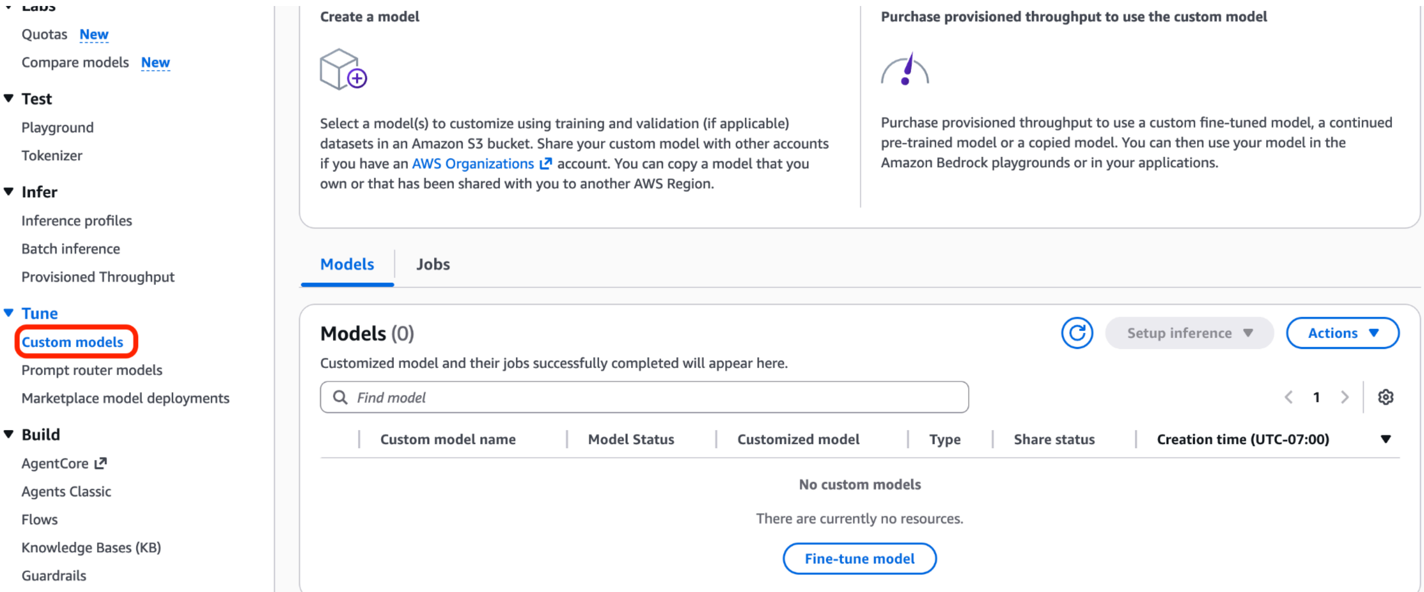
12. Scenario 1: Support Ticket Triage *(recommended starting scenario)*

This scenario trains the model to automatically classify and route incoming customer support tickets based on their content and urgency. In real-world support operations, teams receive hundreds of tickets daily covering billing issues, technical outages, account problems, and general inquiries. Without automation, agents spend significant time reading and manually sorting these tickets before any resolution work begins. By fine-tuning the model on labeled support ticket examples, it learns to instantly identify the ticket category, assign a priority level, and recommend the right team — cutting response time and reducing agent workload. This is one of the most common and high-impact enterprise use cases for fine-tuning, making it the recommended starting point.

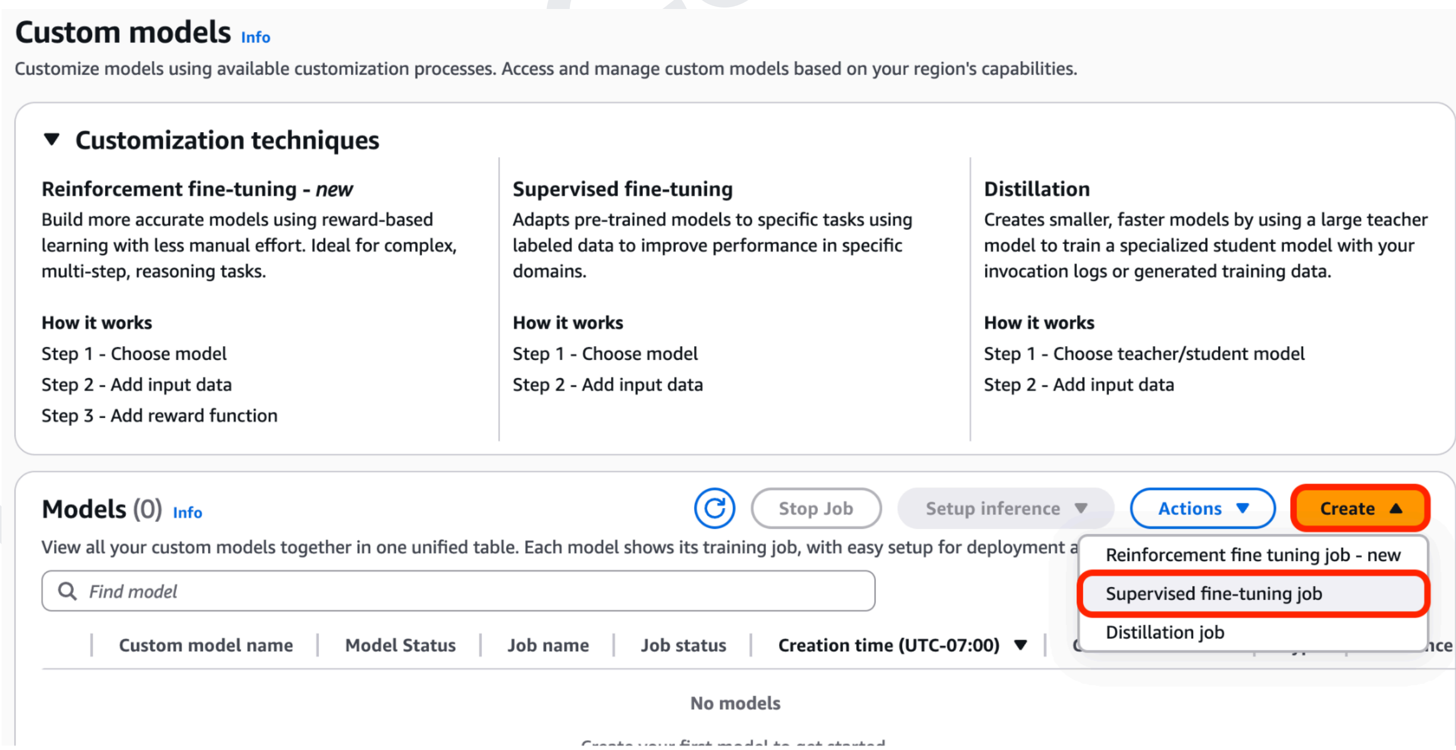
1. In the AWS Management Console, search for **Amazon Bedrock** using the search bar and open the **Amazon Bedrock** service. From here, you can access and manage the AI resources created during the lab, including knowledge bases, guardrails, agents, and other Bedrock components.



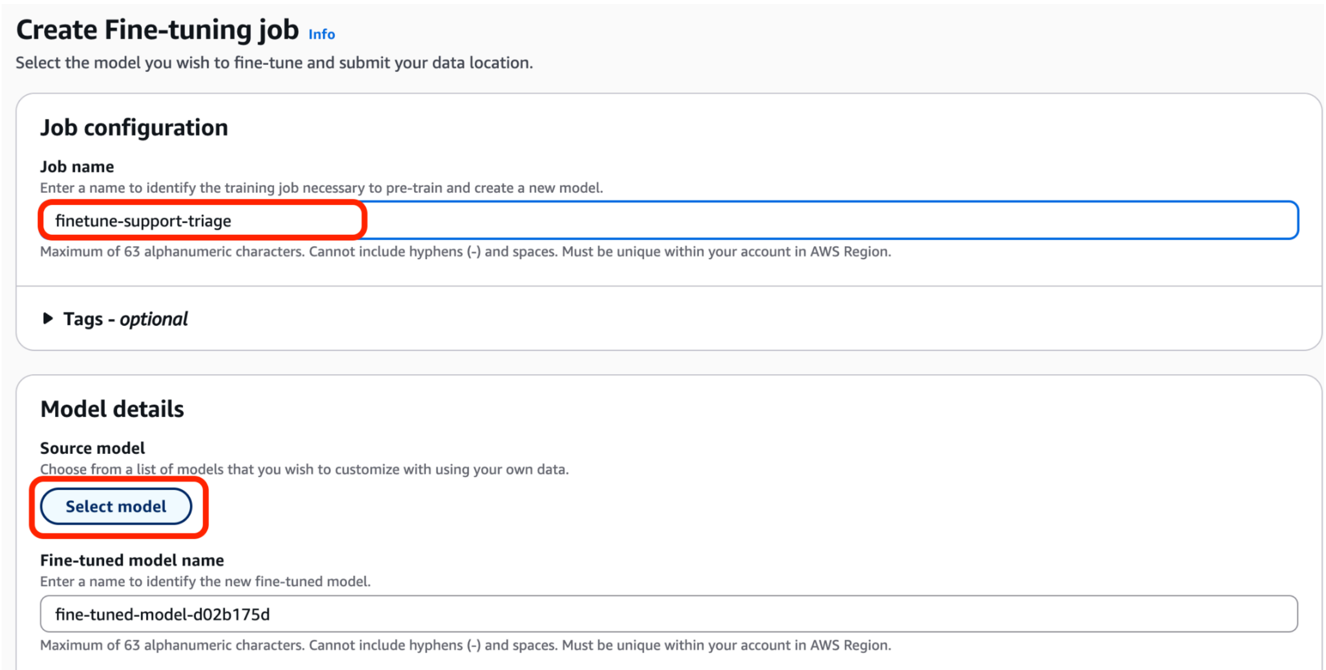
2. Go to **Custom models** under the **Tune** section and verify that no custom models remain. Delete any listed models if necessary.



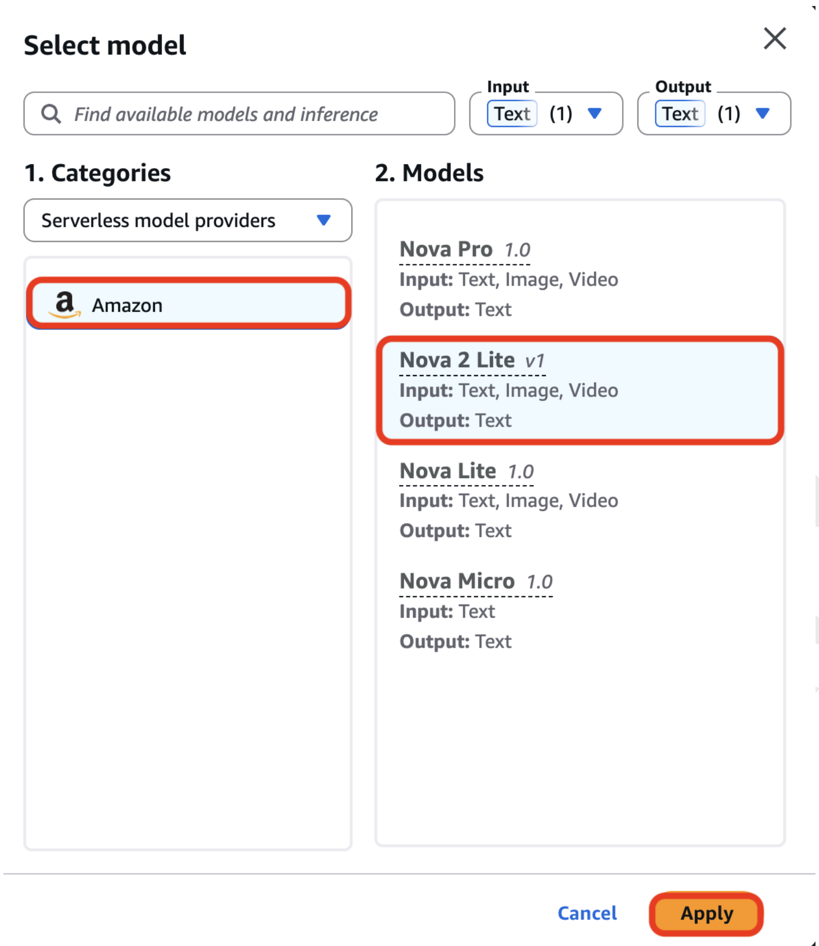
3. Click **Create**, then select **Supervised fine-tuning job** to create a new custom model training job.



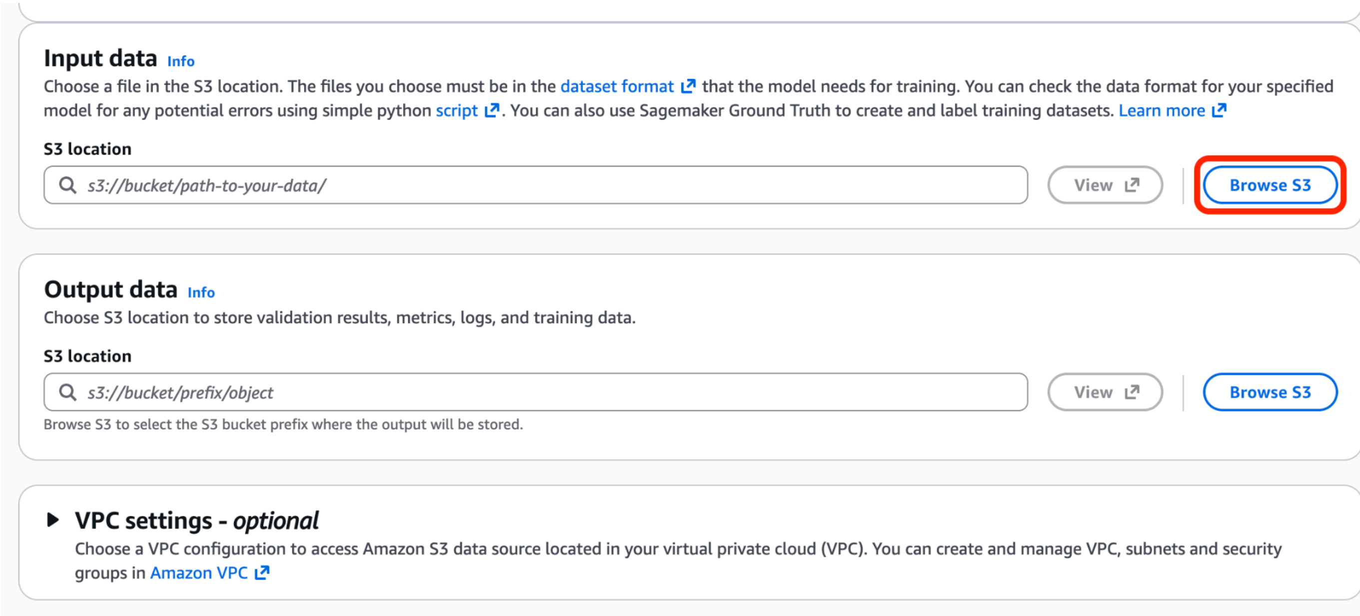
4. Enter a unique **Job name**, then click **Select model** to choose the foundation model you want to fine-tune.



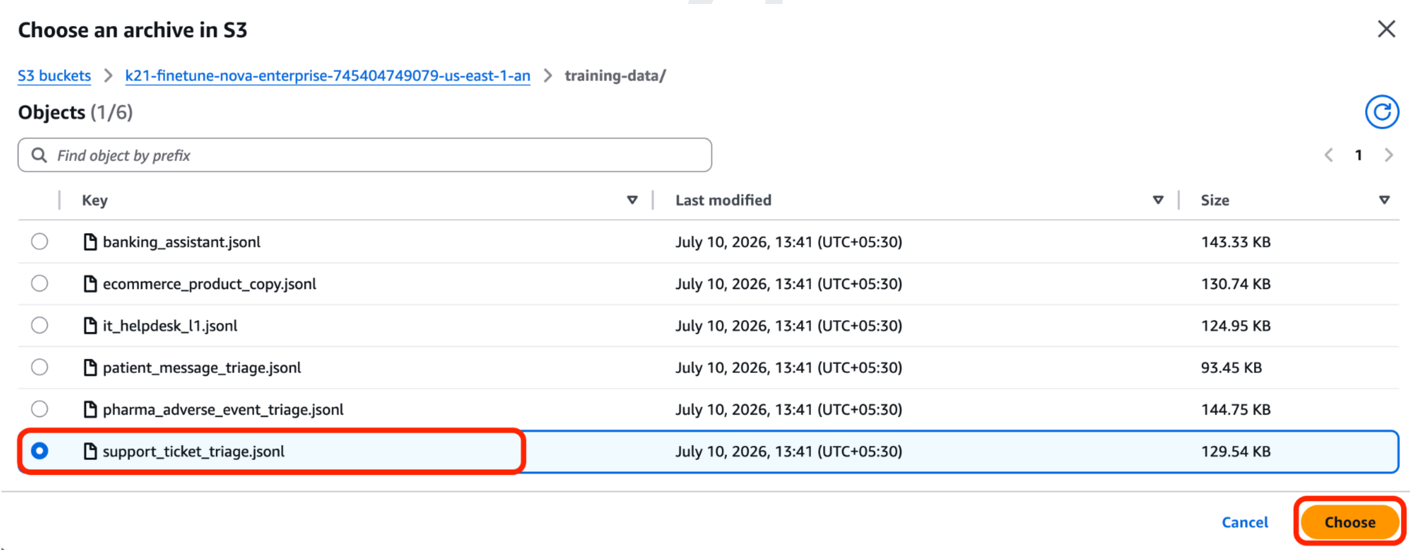
5. Select **Amazon** as the model provider, choose **Nova 2 Lite v1** as the source model, and click **Apply**.



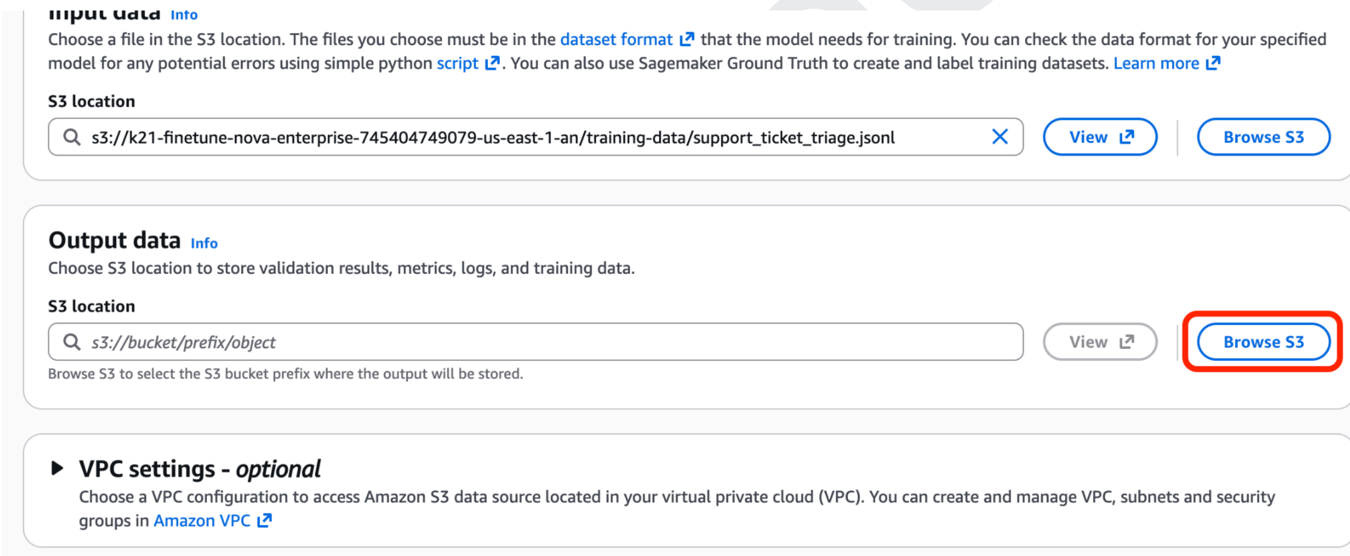
6. Hyperparameters → leave defaults.
7. Click **Browse S3** under **Input data** to select the training dataset from your Amazon S3 bucket. Then, choose an S3 location under **Output data** where the fine-tuning job results and model artifacts will be stored.



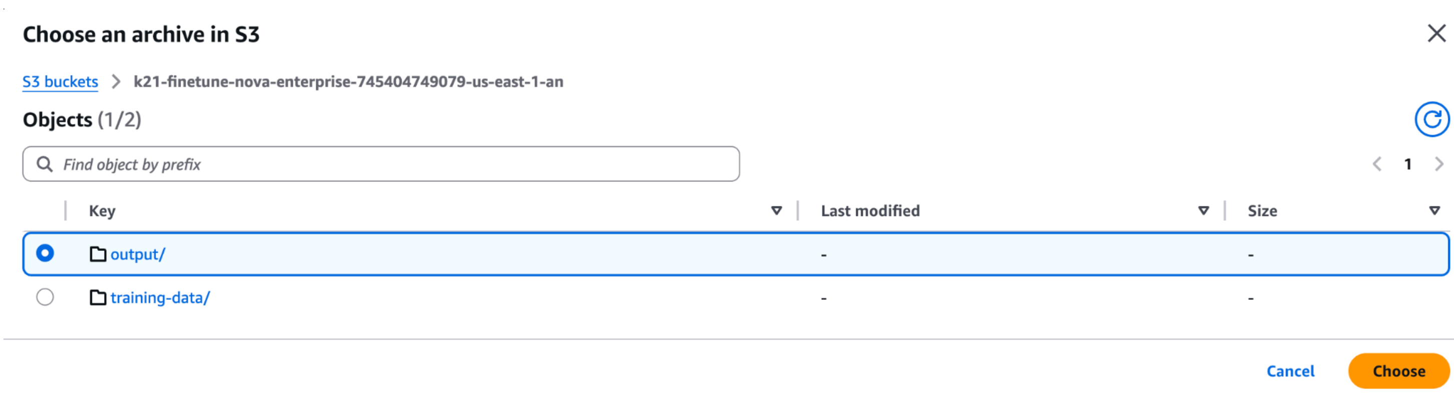
8. Select the **support_ticket_triage.jsonl** training dataset from the S3 bucket, then click **Choose** to use it as the input data for the fine-tuning job.



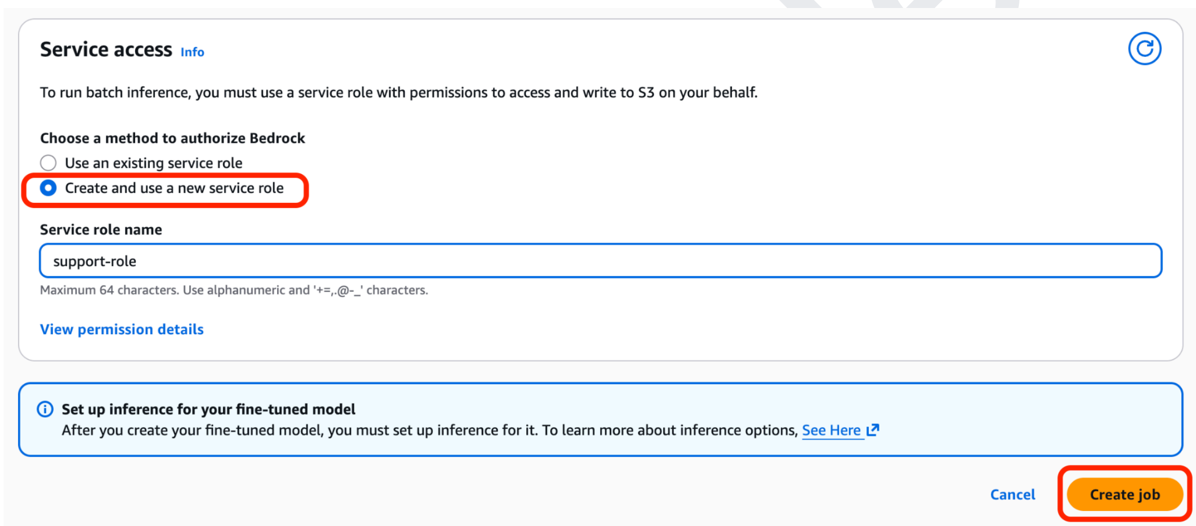
9. For the **Output data** location, click **Browse S3** and select the S3 folder where the fine-tuning outputs, logs, and model artifacts will be stored (for example, `s3://k21-finetune-nova-enterprise/output/support-triage/`).



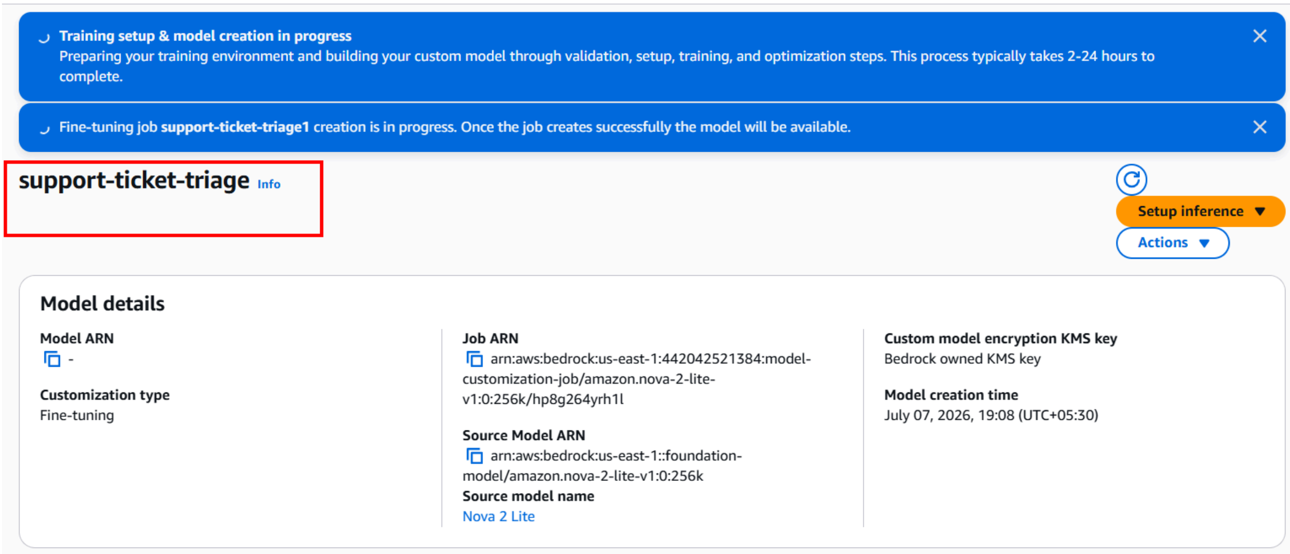
10. Select the **output/** folder from the S3 bucket, then click **Choose** to use it as the destination for storing the fine-tuning job outputs.



11. Under **Service access**, choose **Create and use a new service role**, enter **support-role** as the service role name, and then click **Create job** to start the fine-tuning job.



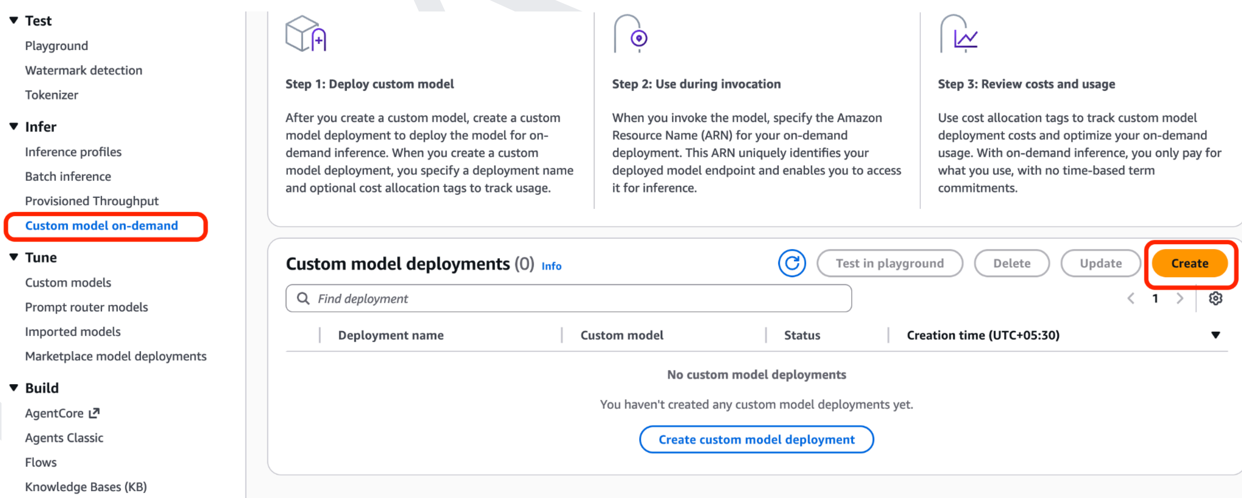
12. Job status starts at **In progress**. Wait until it shows **Active** (a few hours typically).



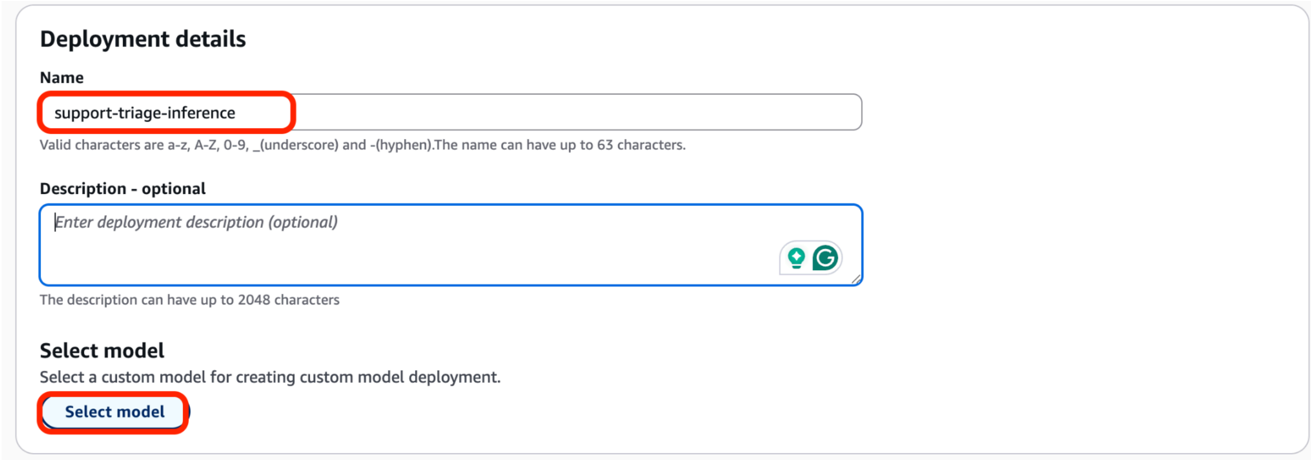
Note: For us, it took 4 hours to be in Active Status.

13. Deploy for on-demand inference

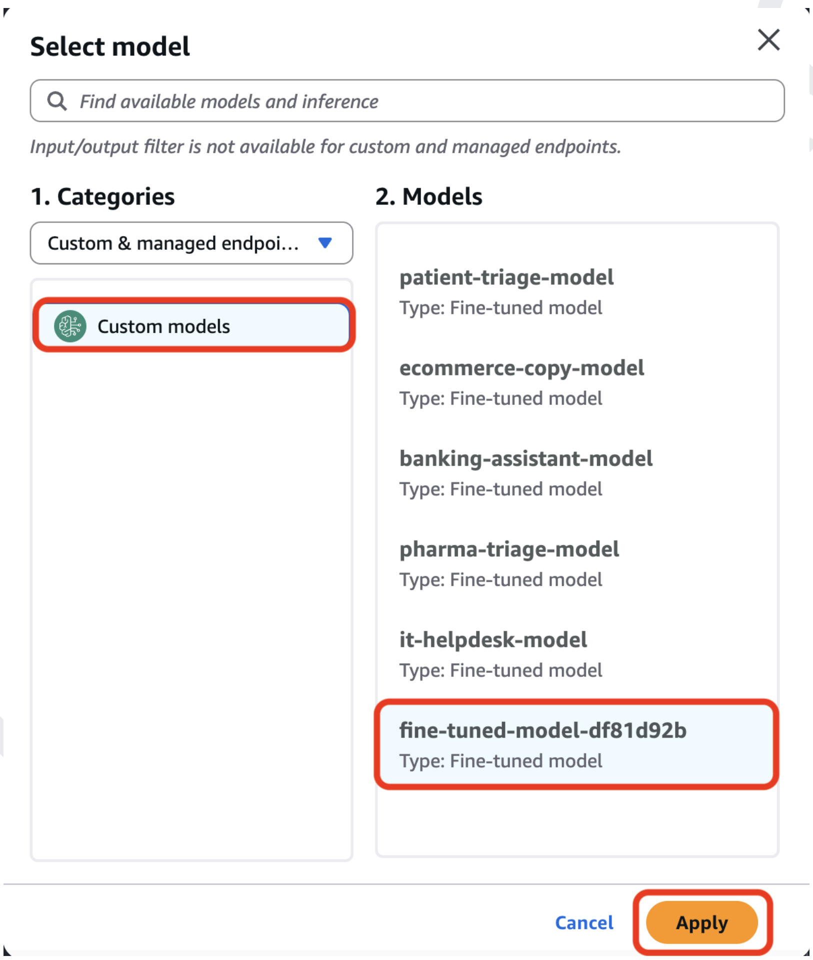
14. In the Bedrock dashboard, under **Infer**, choose **Custom model on demand**.



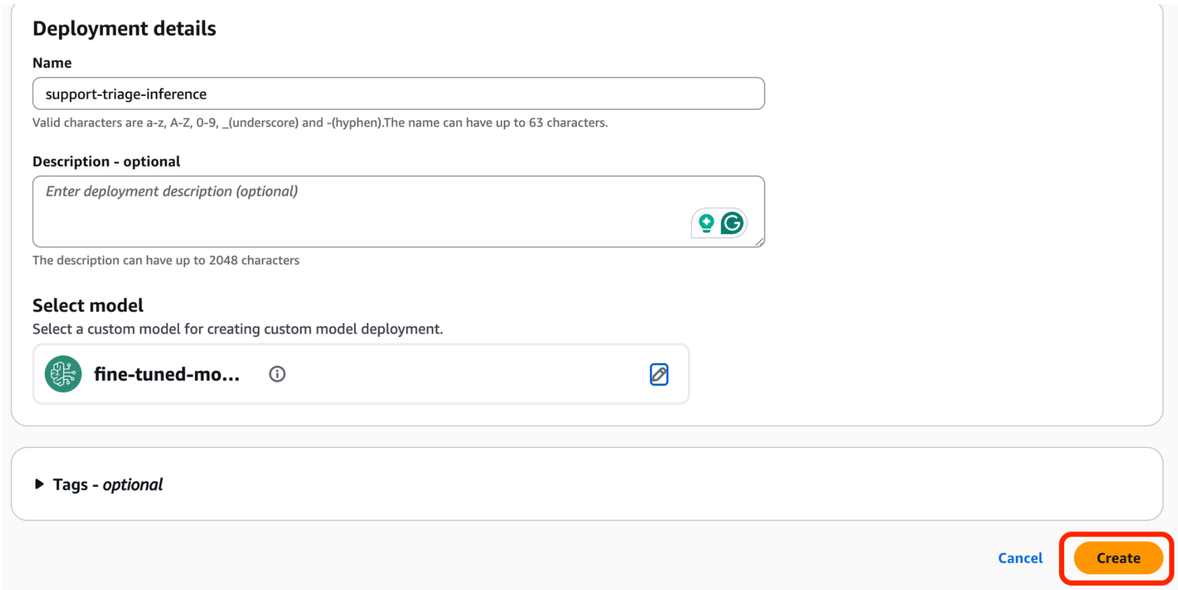
15. In the **Deployment details** section, enter **support-triage-inference** as the deployment name, then click **Select model** to choose the fine-tuned model for deployment.



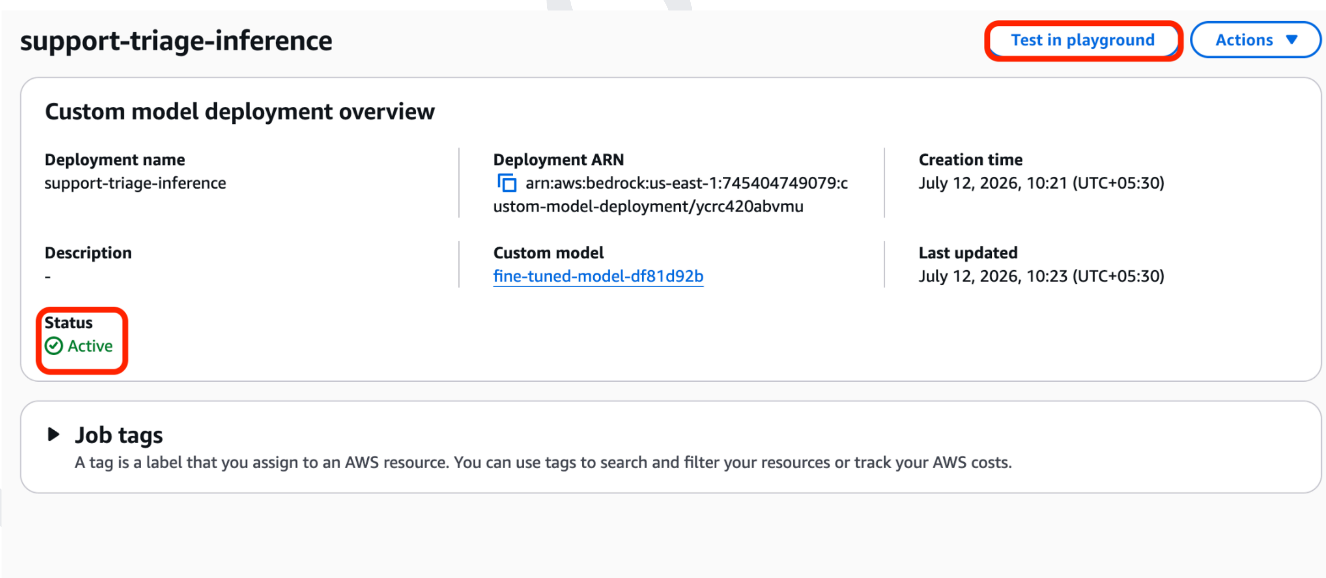
16. In the **Select model** dialog, choose **Custom models**, select your newly fine-tuned model, and click **Apply** to use it for the deployment.



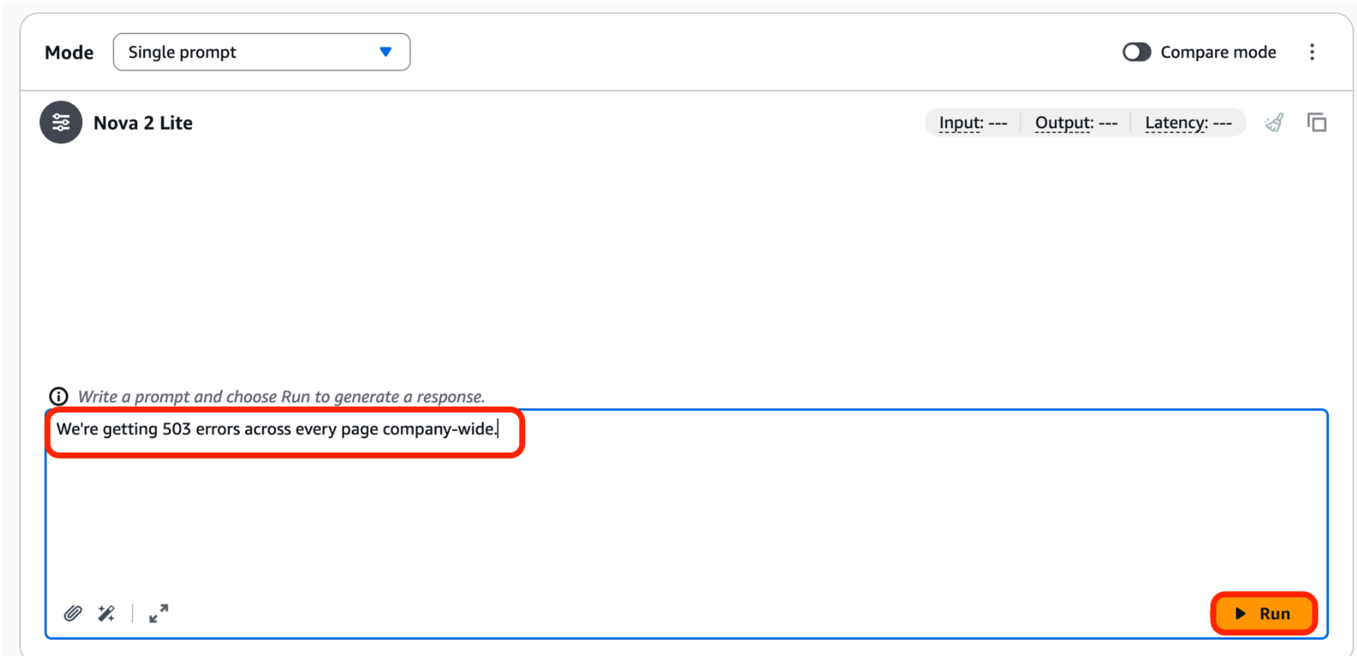
17. Verify the deployment name and selected fine-tuned model, then click **Create** to deploy the custom model for inference.



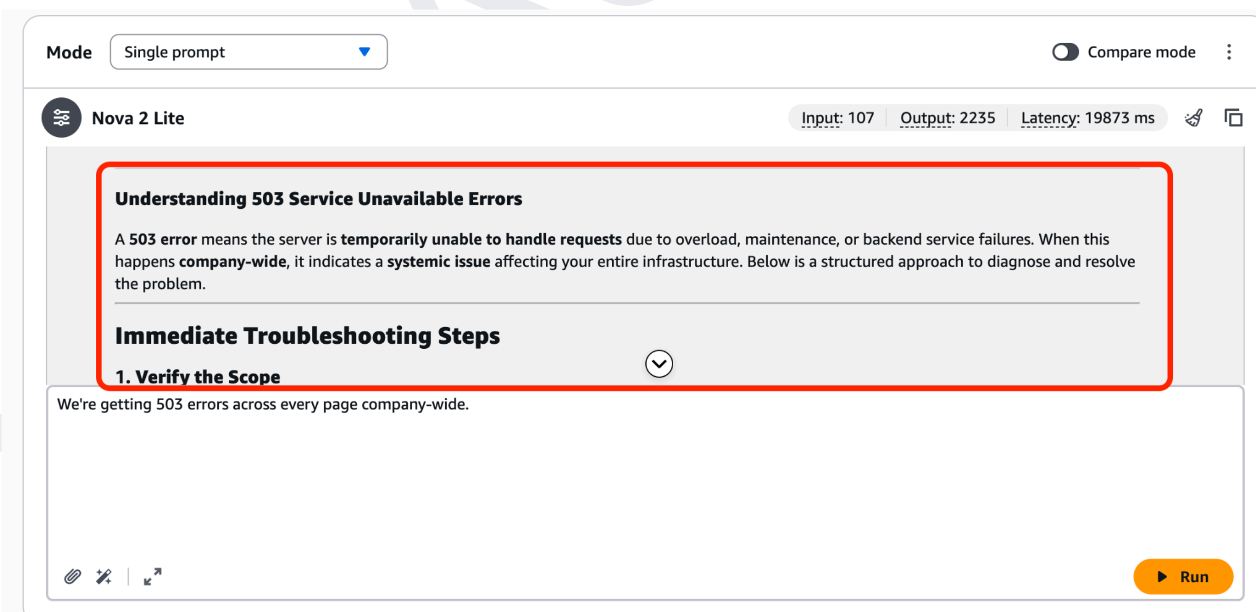
18. Wait for the status to change from **In progress** to **Active** (usually 5–10 min).
19. Wait until the deployment status changes to **Active**, then click **Test in playground** to verify the fine-tuned model by sending sample prompts.



20. Enter a sample prompt, such as **"We're getting 503 errors across every page company-wide."**, and click **Run** to test the deployed fine-tuned model and review its response.

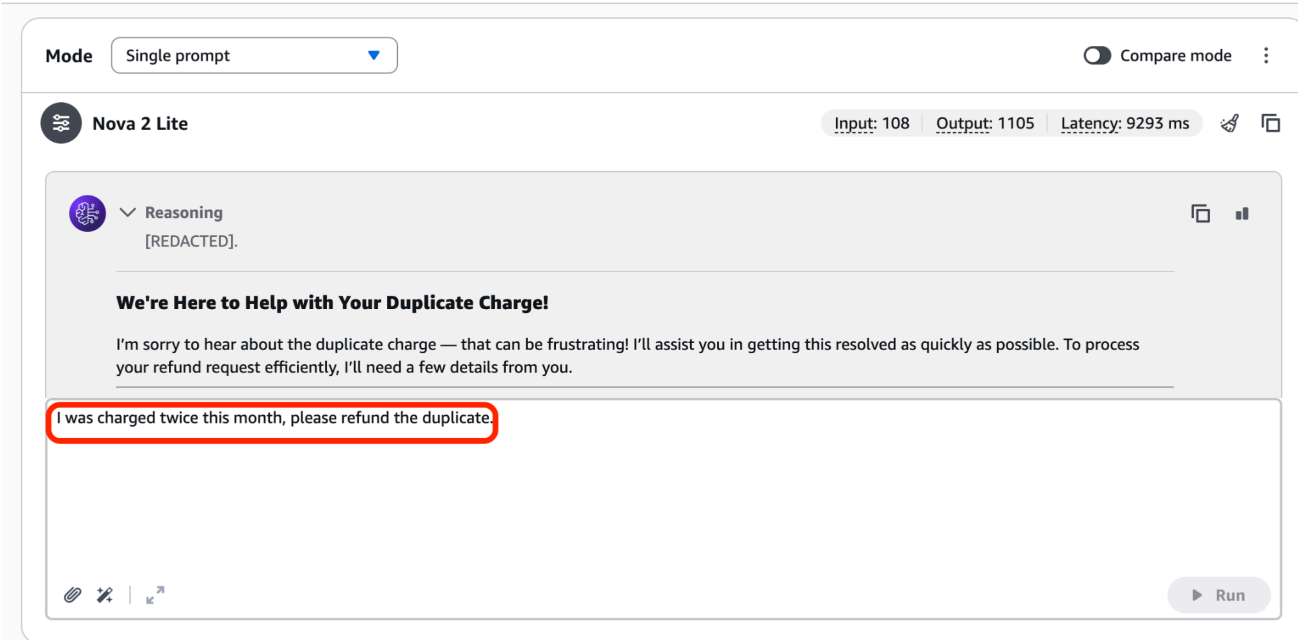


21. Review the model's response in the playground to verify that the fine-tuned model correctly understands the prompt and provides an accurate, domain-specific troubleshooting recommendation.



22. Enter another sample prompt, such as **"I was charged twice this month, please refund the duplicate."**, and click **Run** to verify that the fine-tuned model generates an appropriate,

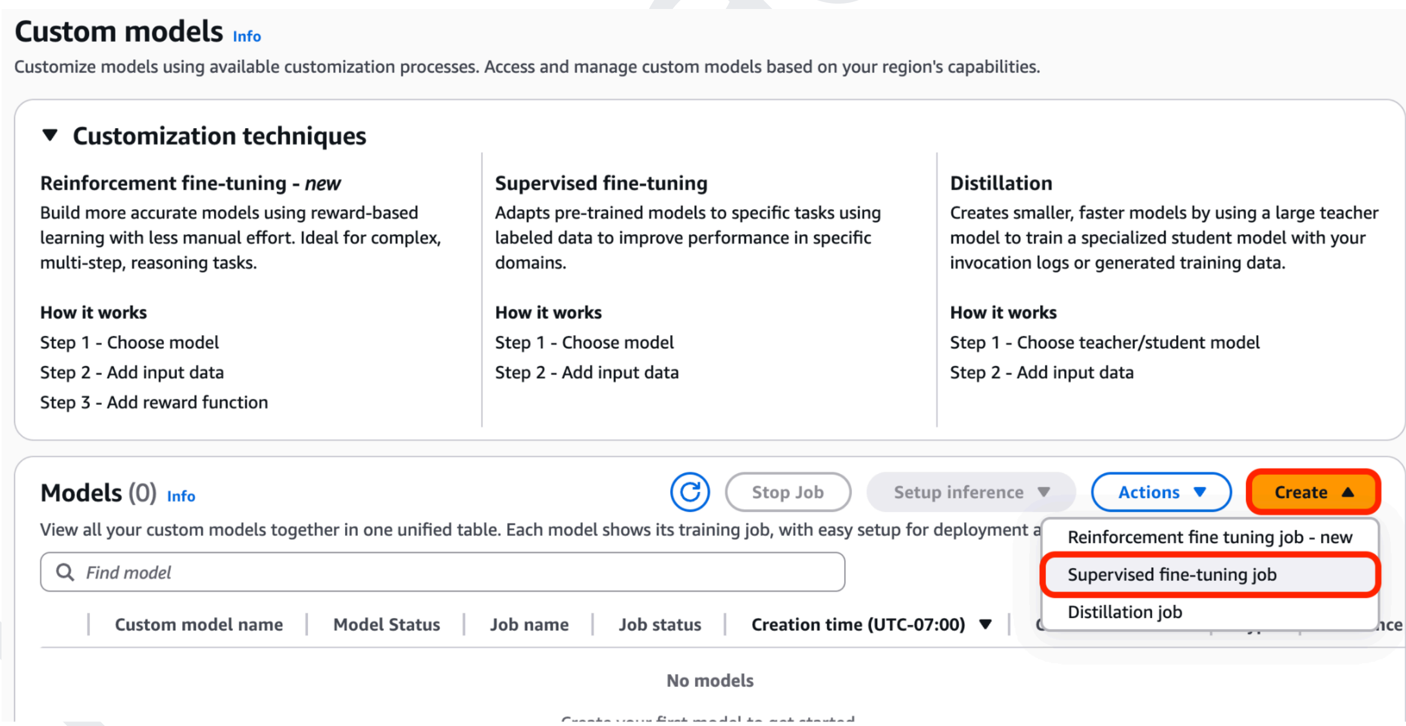
domain-specific customer support response.



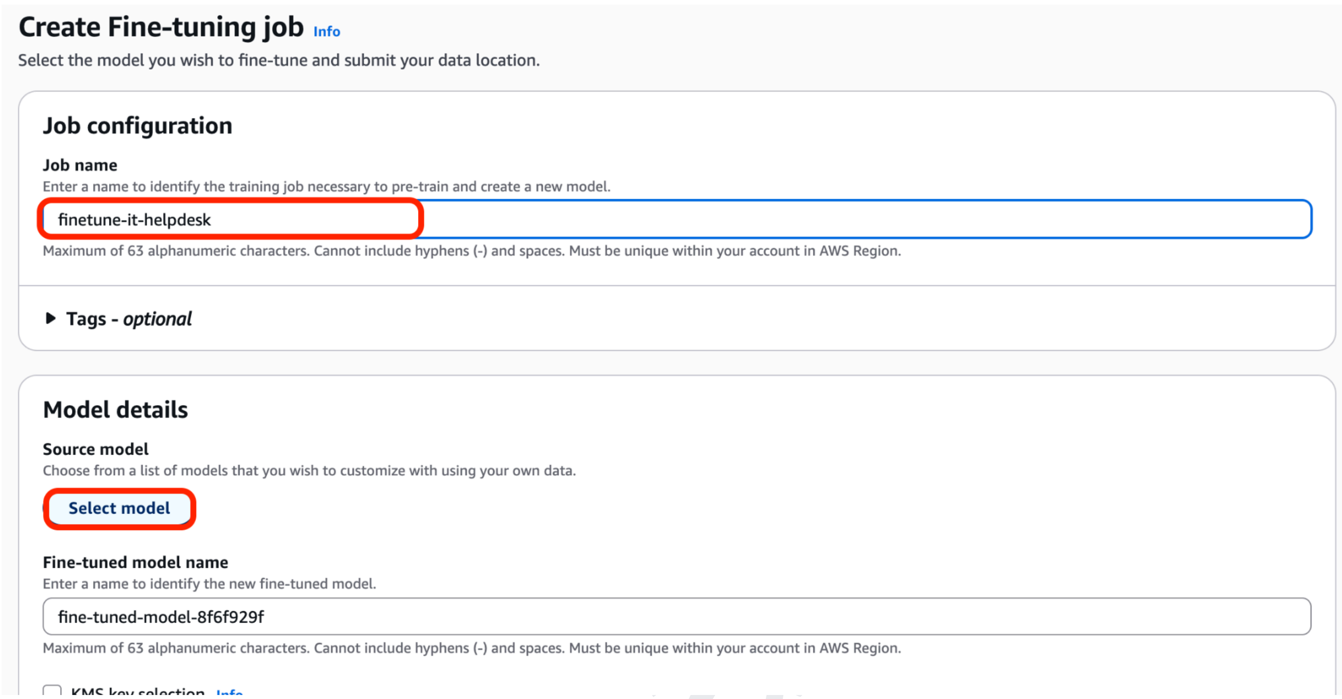
13. Scenario 2: IT / DevOps L1 Helpdesk

This scenario teaches the model to act as a first-line IT support assistant capable of diagnosing and resolving common employee technology issues such as VPN disconnections, slow system performance, application crashes, and Microsoft Teams audio problems. L1 helpdesk teams handle a high volume of repetitive, predictable issues that follow known troubleshooting paths. Fine-tuning the model on real helpdesk examples means it can respond with accurate, step-by-step guidance without human intervention, allowing IT staff to focus on more complex escalations. For organizations with large workforces, this directly reduces ticket backlogs and improves employee productivity and satisfaction

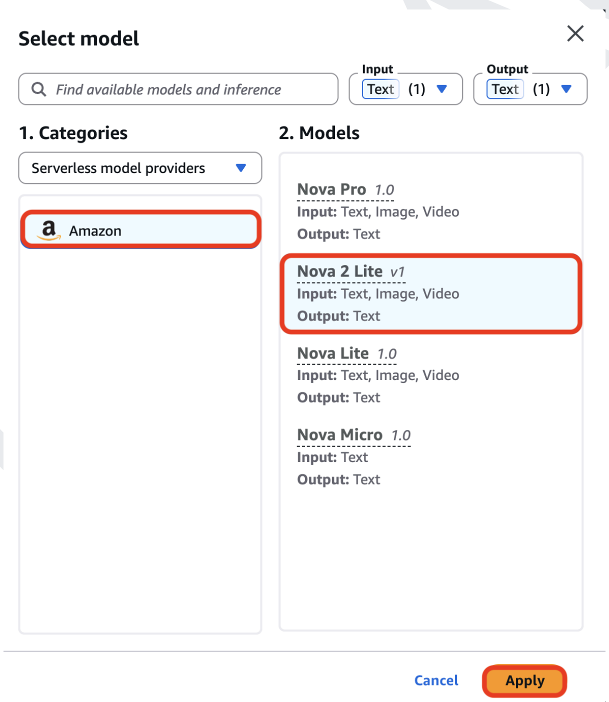
1. From the **Create** menu, select **Supervised fine-tuning job** to start creating a new fine-tuning job using your labeled training dataset.



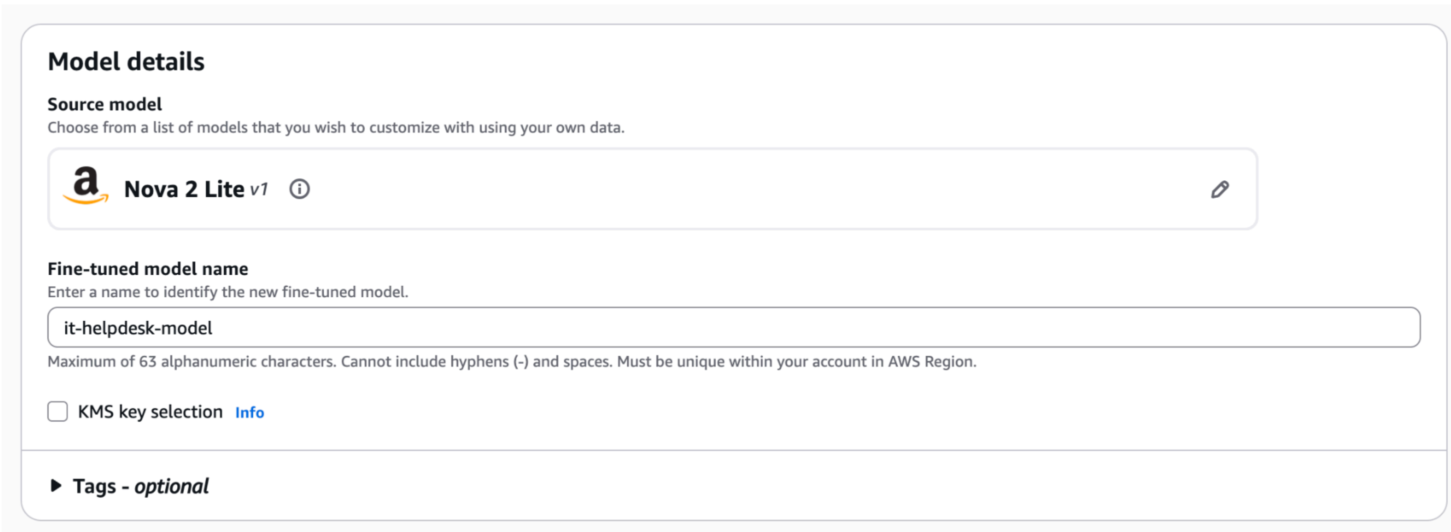
2. Enter **`finetune-it-helpdesk`** as the **Job name**, then click **Select model** to choose the foundation model that you want to fine-tune.



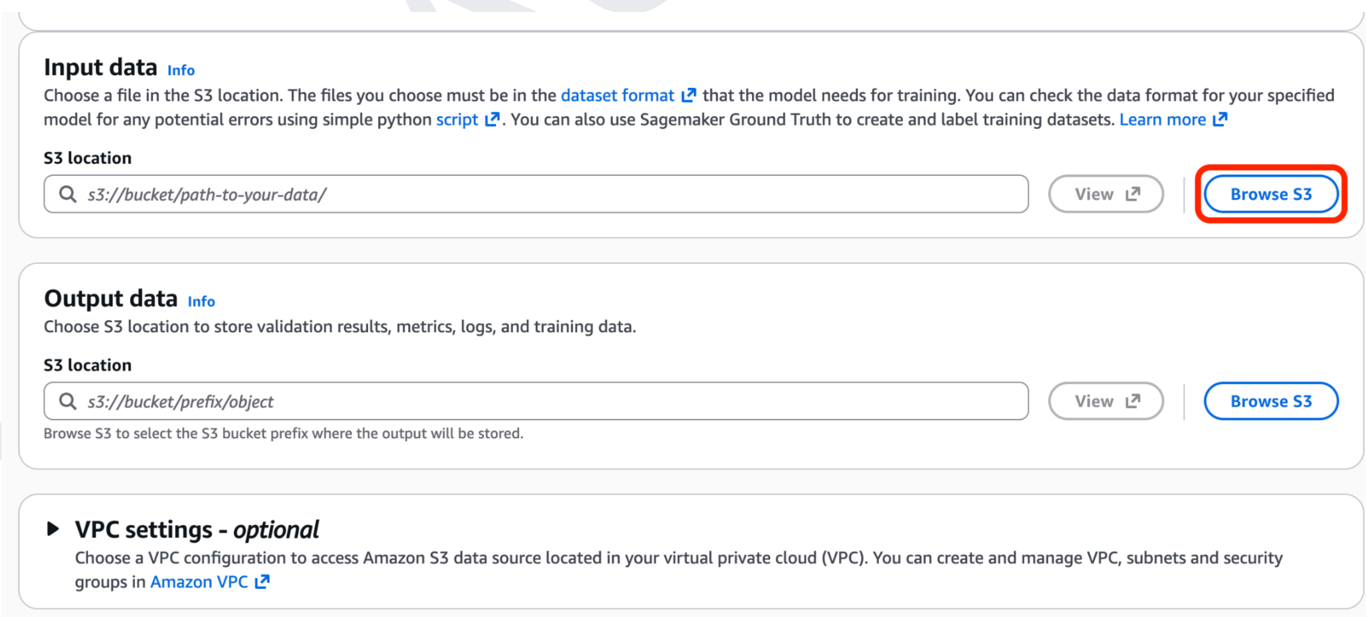
3. Select **Amazon** as the model provider, choose **Nova 2 Lite v1** as the source model, and click **Apply**.



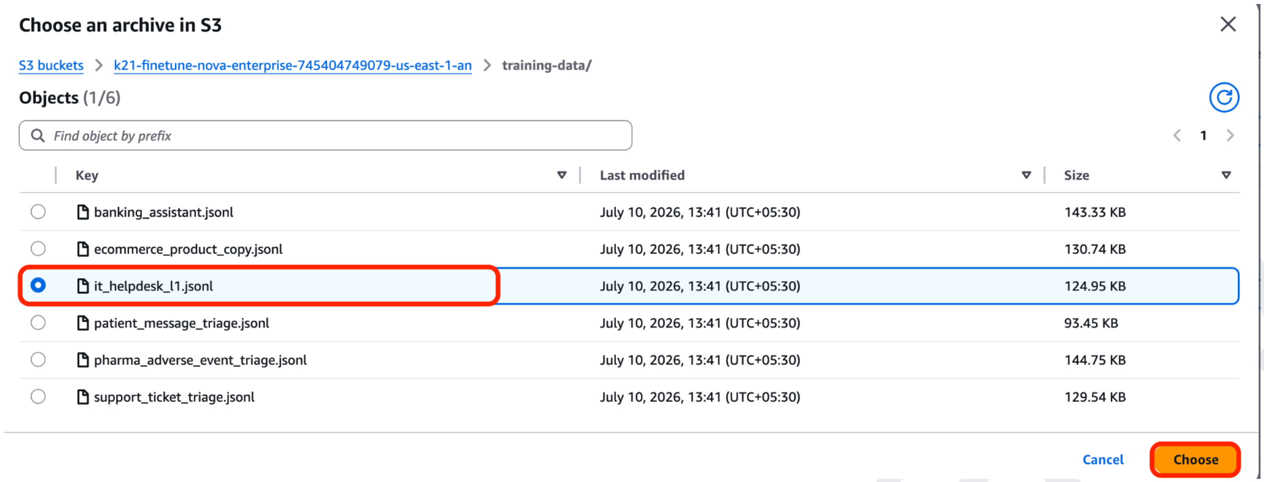
4. Verify that **Nova 2 Lite v1** is selected as the **Source model**, enter `it-helpdesk-model` as the **Fine-tuned model name**, and leave the remaining settings as default.



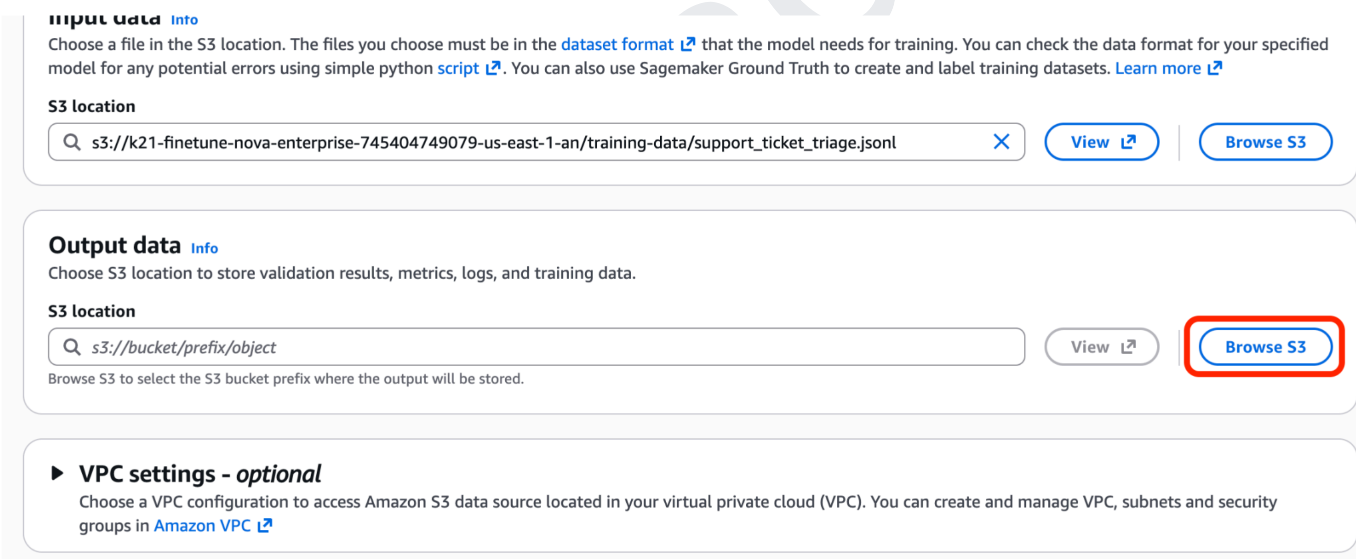
5. Hyperparameters: defaults.
6. Click **Browse S3** under **Input data** to select the training dataset from your Amazon S3 bucket. Then, choose an S3 location under **Output data** where the fine-tuning job results and model artifacts will be stored.



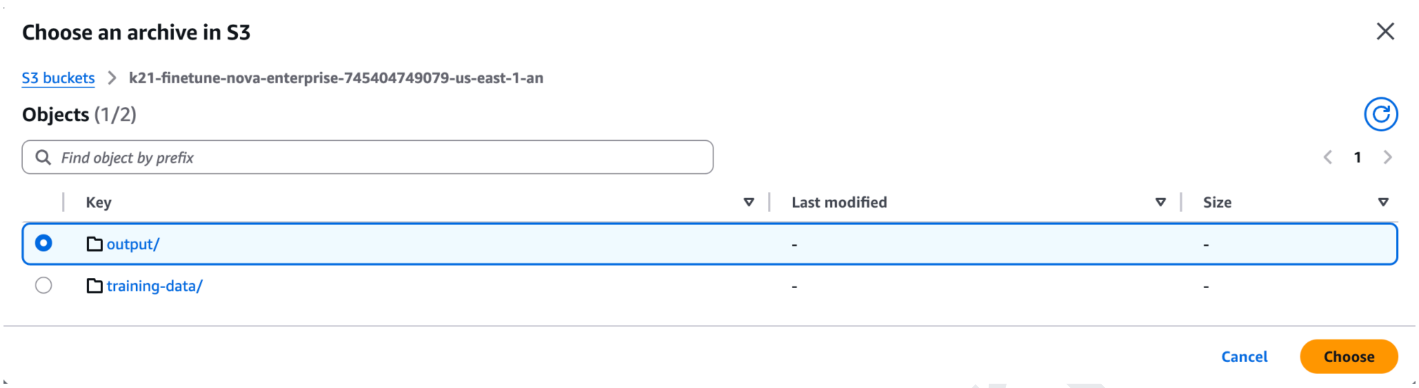
7. Select `it_helpdesk_[1].jsonl`, then click **Choose** to use it as the training dataset for the fine-tuning job.



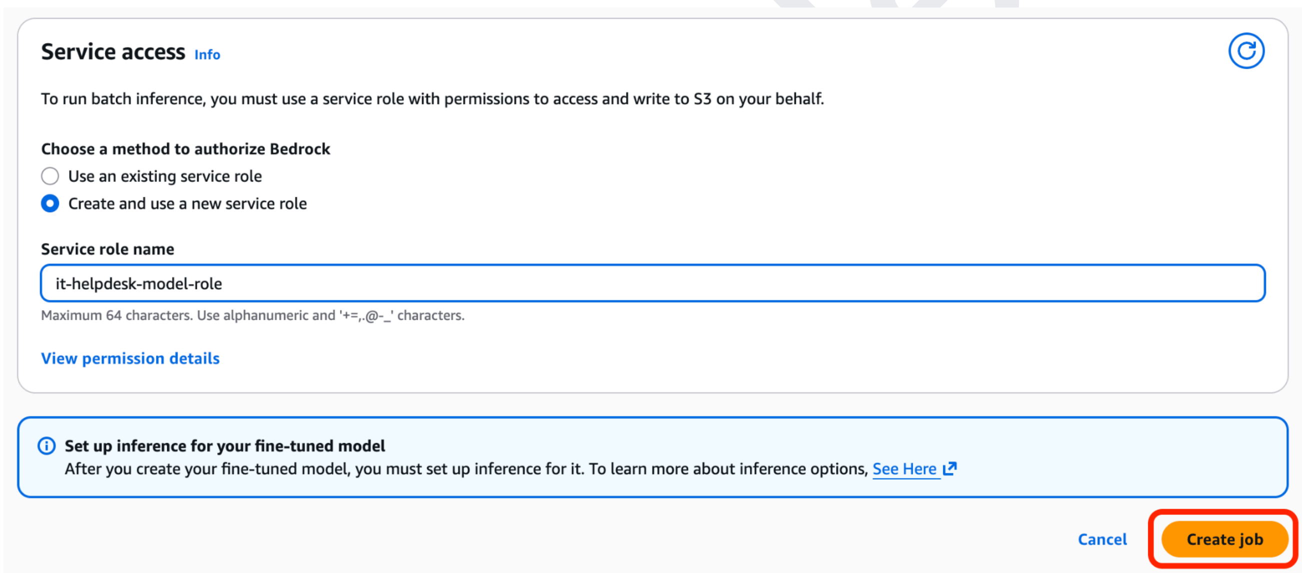
8. For the **Output data** location, click **Browse S3** and select the S3 folder where the fine-tuning outputs, logs, and model artifacts will be stored (for example, s3://k21-finetune-nova-enterprise/output/support-triage/).



9. Select the **output/** folder from the S3 bucket, then click **Choose** to use it as the destination for storing the fine-tuning job outputs.



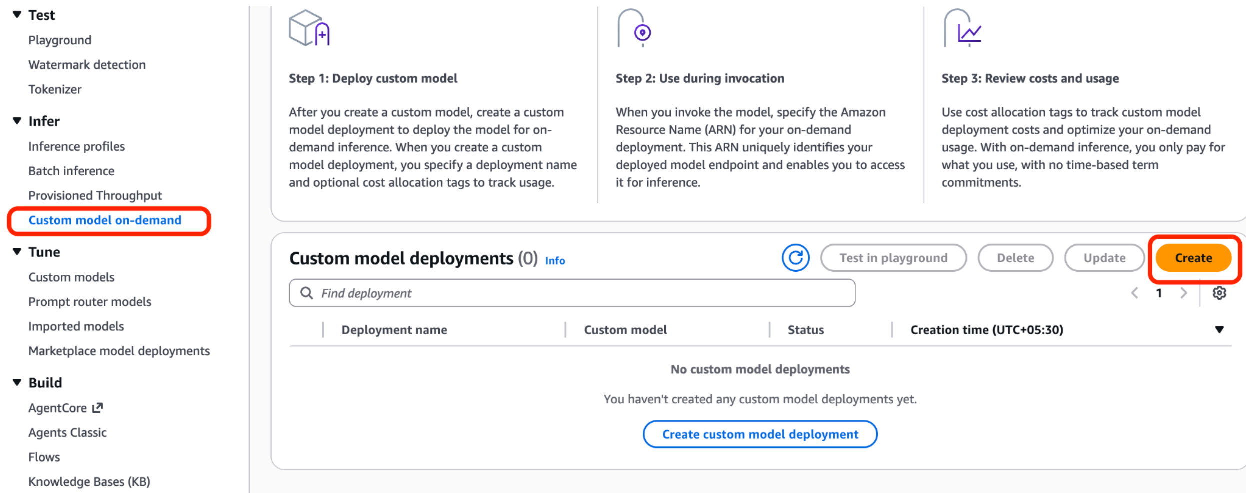
10. Select **Create and use a new service role**, enter `it-helpdesk-model-role` as the **Service role name**, and click **Create job** to start the fine-tuning job.



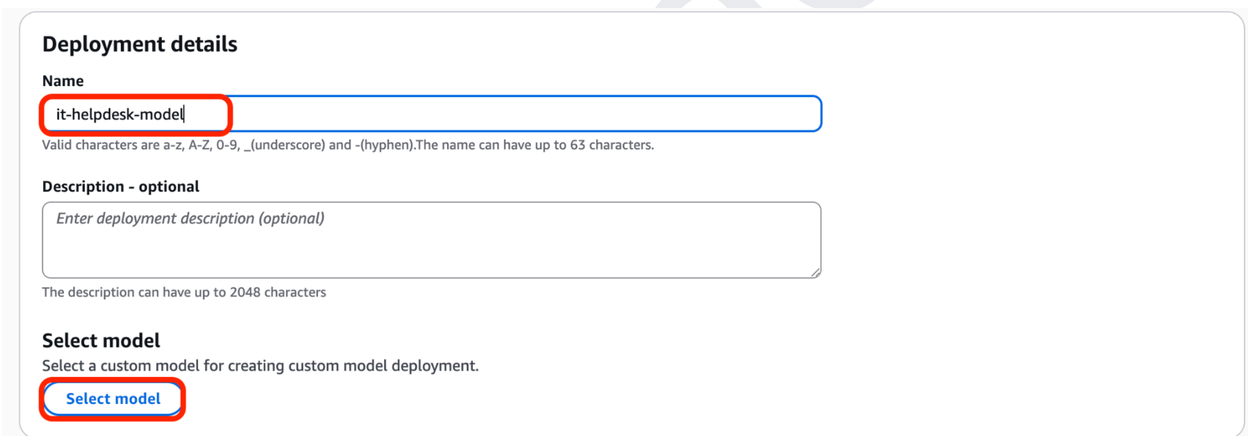
Note: For us, it took 4 hours to be in Active Status.

11. Deploy for on-demand inference

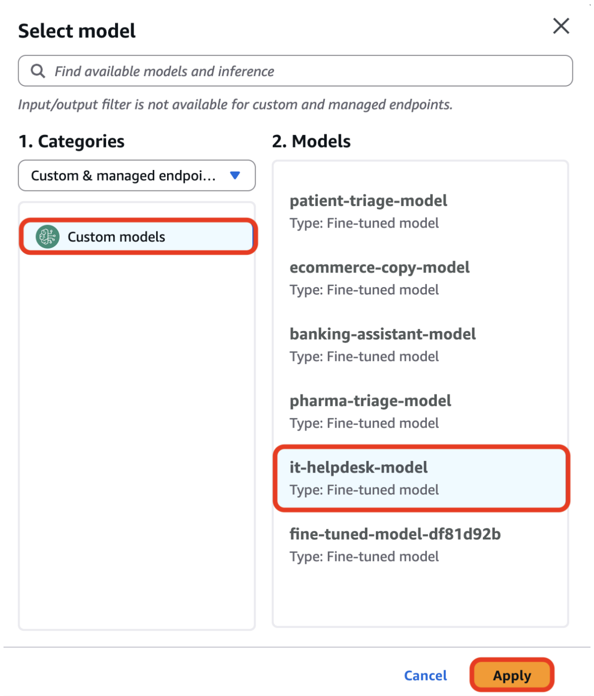
12. From the left navigation pane, select **Custom model on-demand**, then click **Create** to deploy your fine-tuned model for inference.



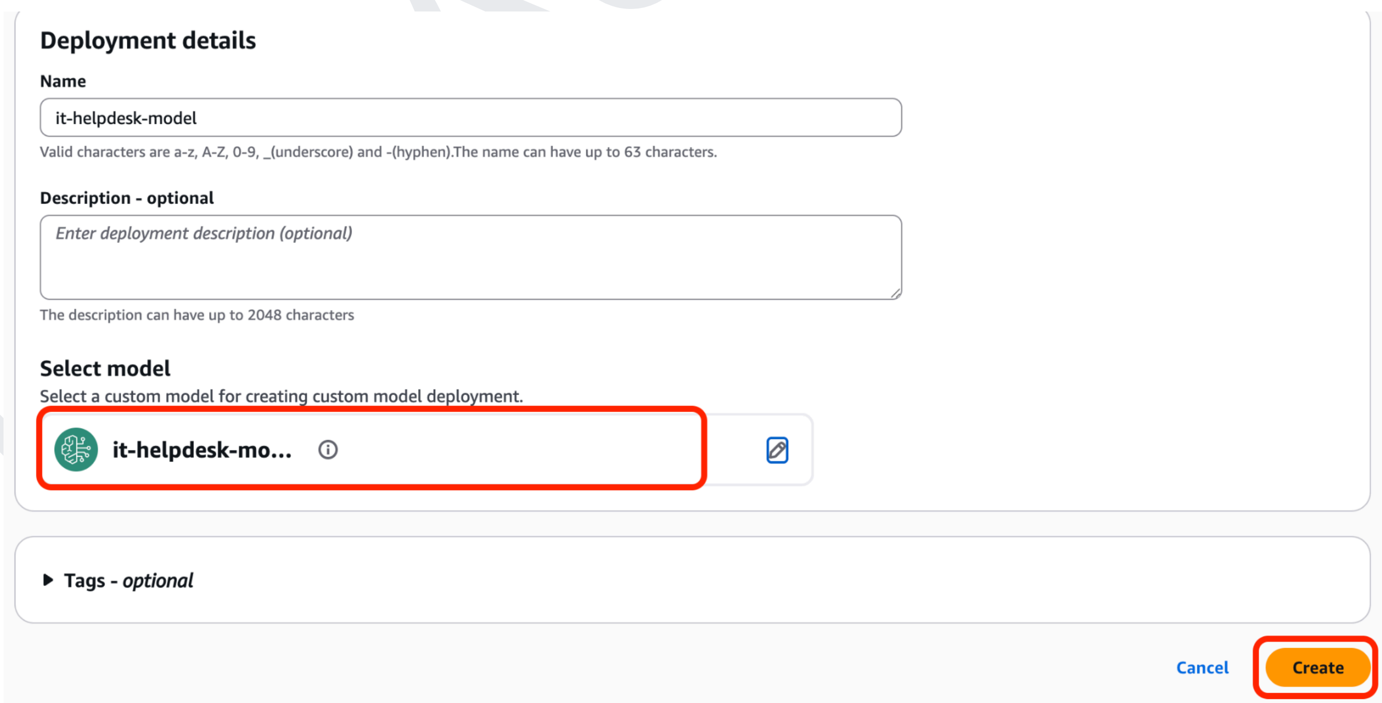
13. Enter **`it-helpdesk-model`** as the **Deployment name**, then click **Select model** to choose the fine-tuned model for deployment.



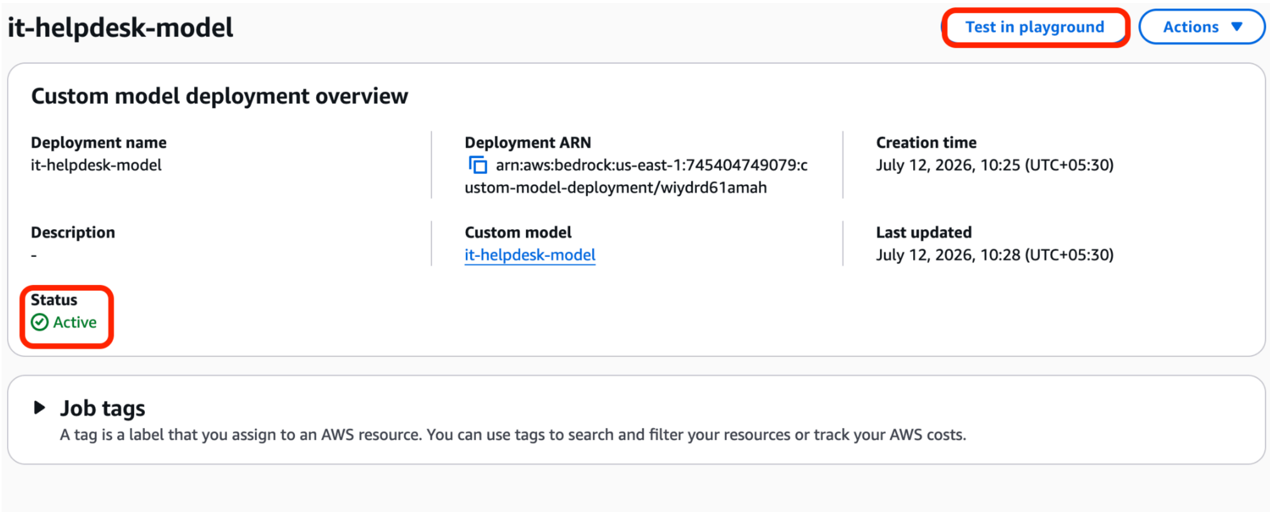
14. Select **Custom models**, choose `it-helpdesk-model`, and then click **Apply** to use it for the deployment.



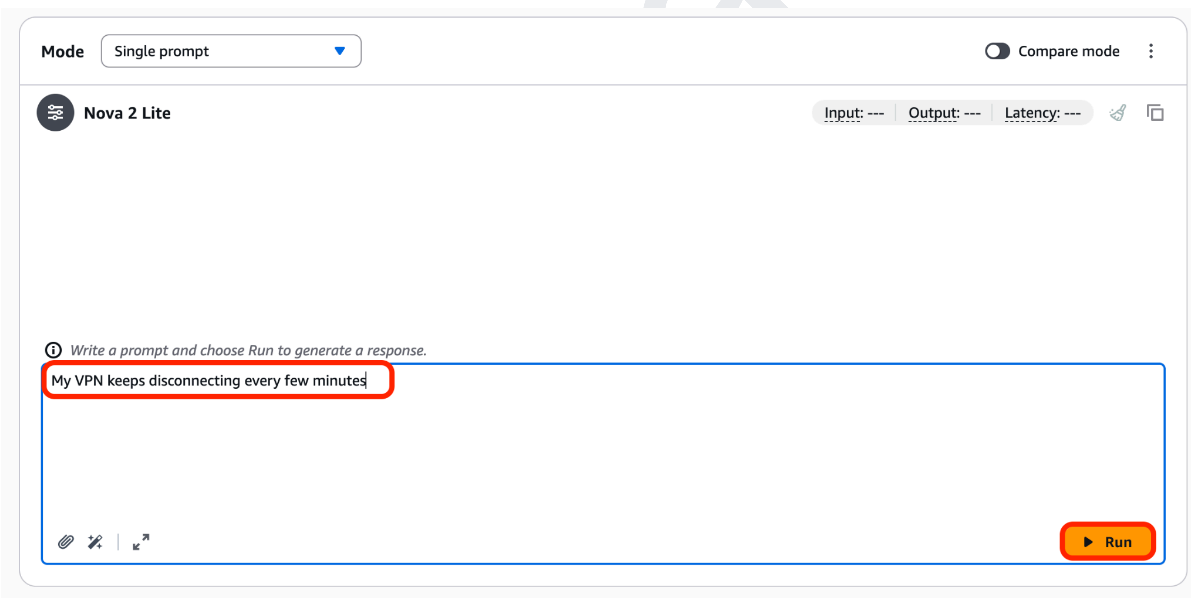
15. Verify that `it-helpdesk-model` is selected as the deployment model, then click **Create** to deploy the fine-tuned model for inference.



16. Wait until the deployment status changes to **Active**, then click **Test in playground** to verify your fine-tuned model with sample prompts.

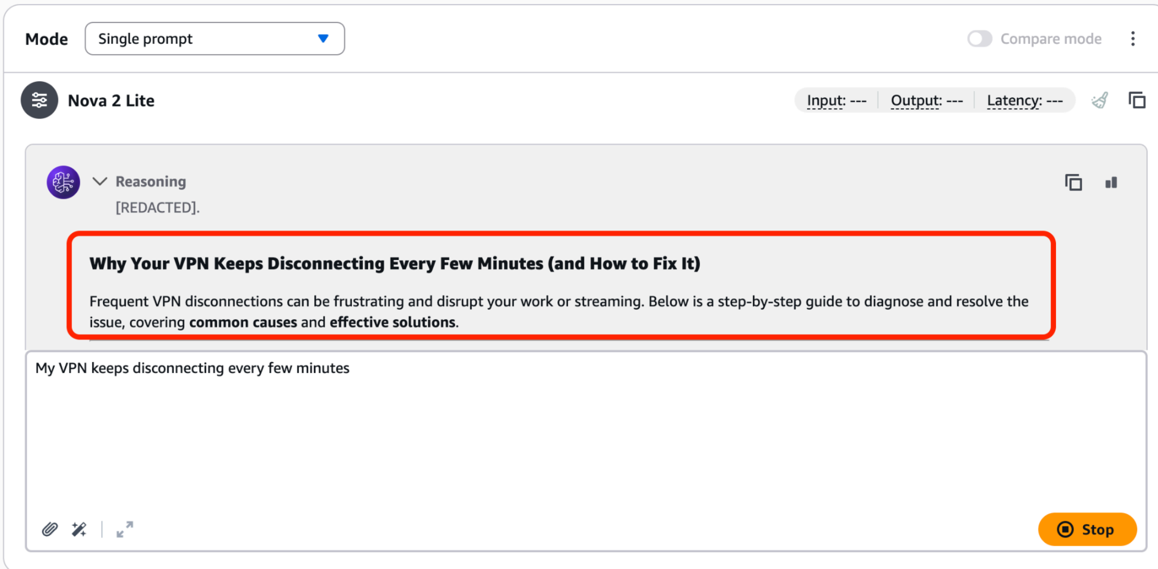


17. Enter the prompt `My VPN keeps disconnecting every few minutes`, then click **Run** to test the fine-tuned model's response.

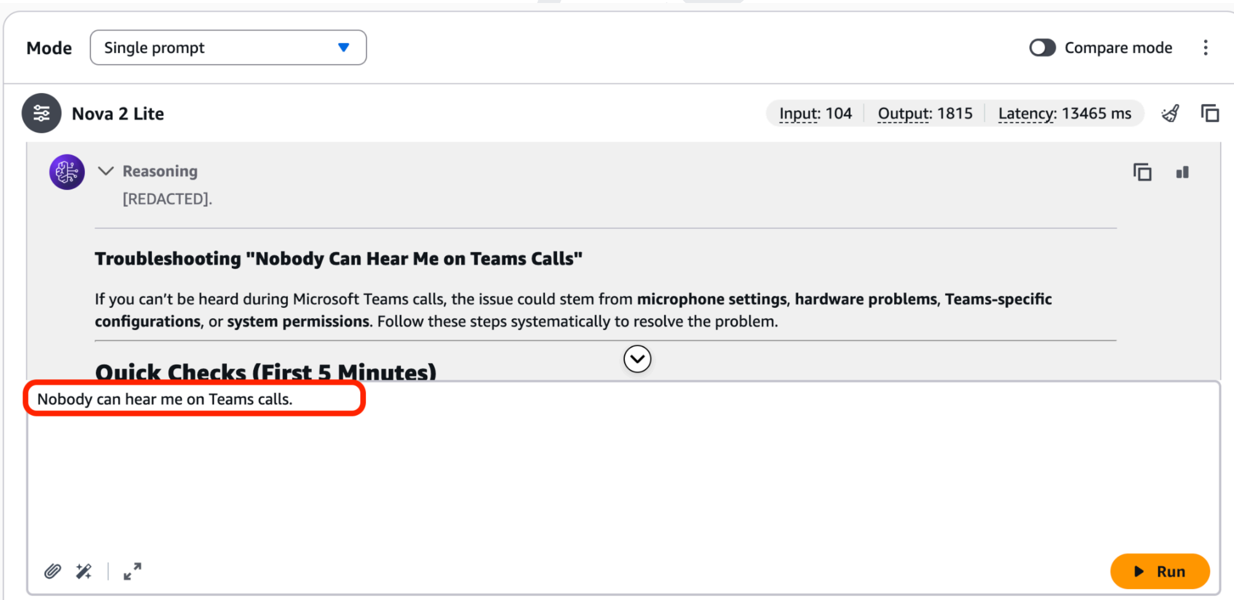


18. Review the generated response to confirm that the fine-tuned model provides relevant troubleshooting guidance for the VPN connectivity issue, indicating that the customization was

successful.



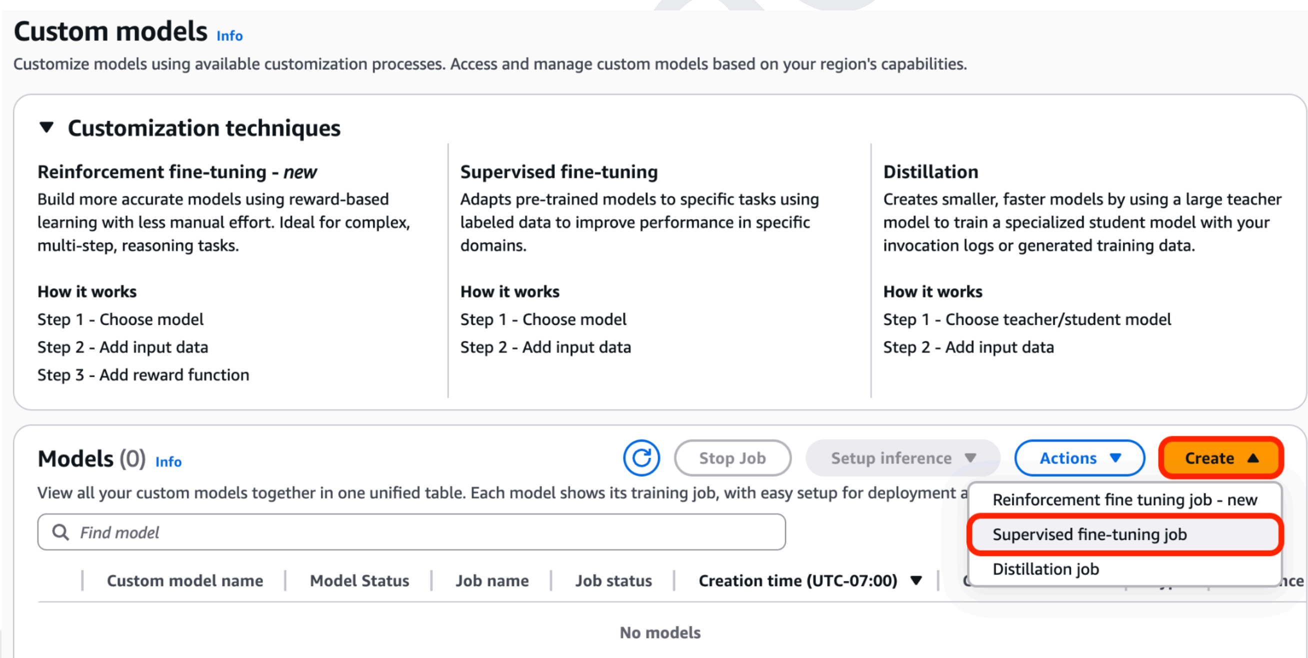
19. Review the generated response to verify that the fine-tuned model provides accurate and relevant troubleshooting steps for the Microsoft Teams audio issue, confirming that the model has been successfully customized.



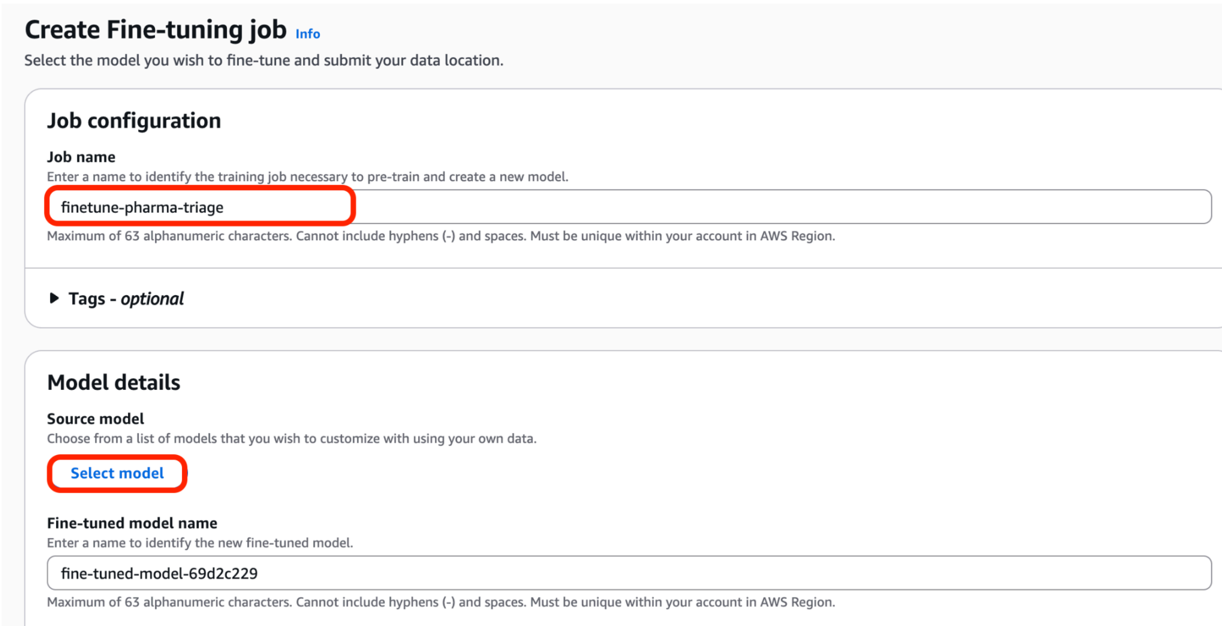
14. Scenario 3: Pharmacovigilance Adverse-Event Triage

This scenario fine-tunes the model to analyze patient safety reports and classify them as either Serious Adverse Events (SAEs) or non-serious events, following standard pharmacovigilance guidelines. In the pharmaceutical and clinical trial industry, every adverse event reported by a patient or healthcare provider must be reviewed, classified, and escalated according to strict regulatory timelines — missing a serious event can have legal and patient safety consequences. Training the model on adverse-event examples enables it to quickly surface critical cases that require immediate reporting, while routing non-serious events through standard documentation workflows. This reduces the manual burden on safety teams and helps ensure regulatory compliance at scale.

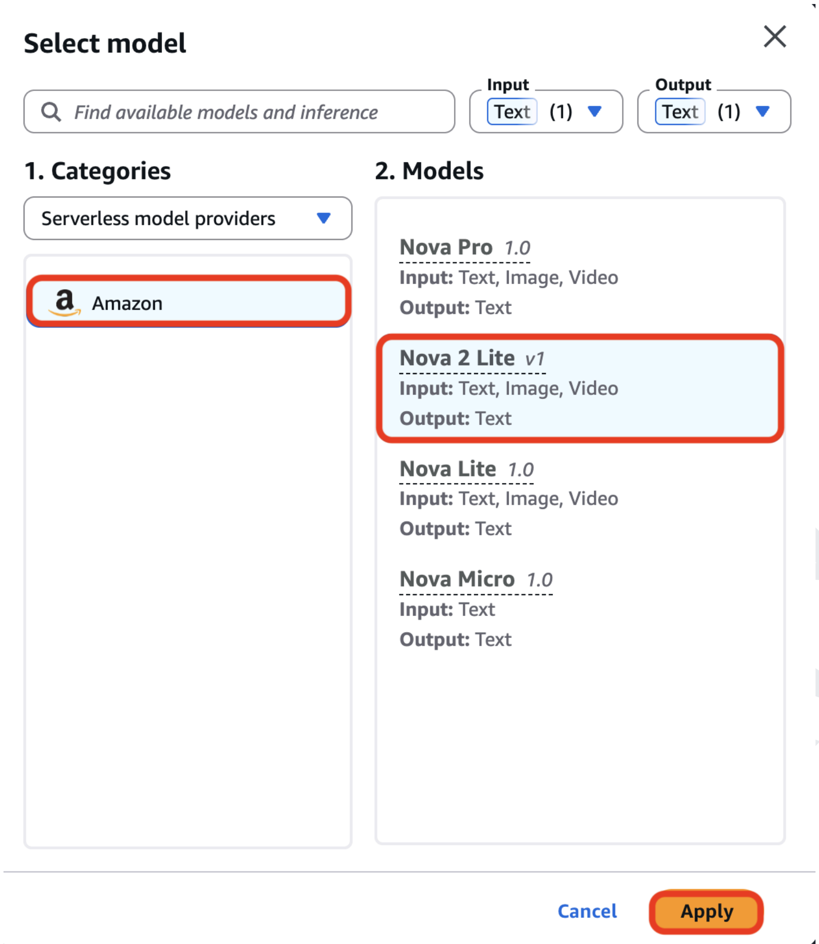
1. Click **Create** and select **Supervised fine-tuning job** to start creating a new fine-tuning job for your custom model.



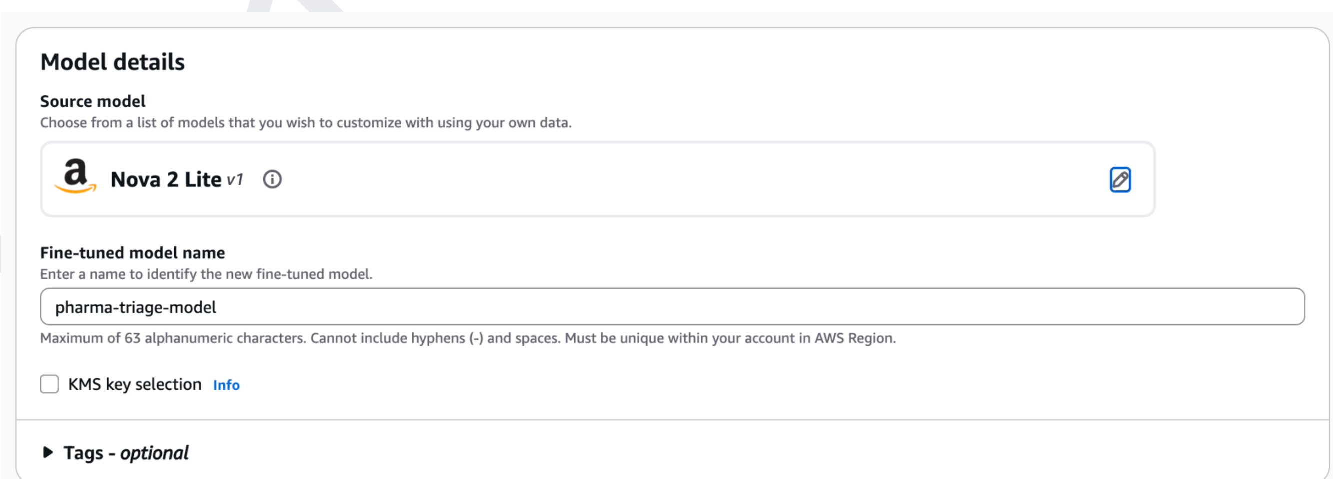
2. In the **Job name** field, enter **`finetune-pharma-triage`**, then click **Select model** under **Source model**.



3. Select **Amazon**, choose **Nova 2 Lite v1**, and click **Apply**.

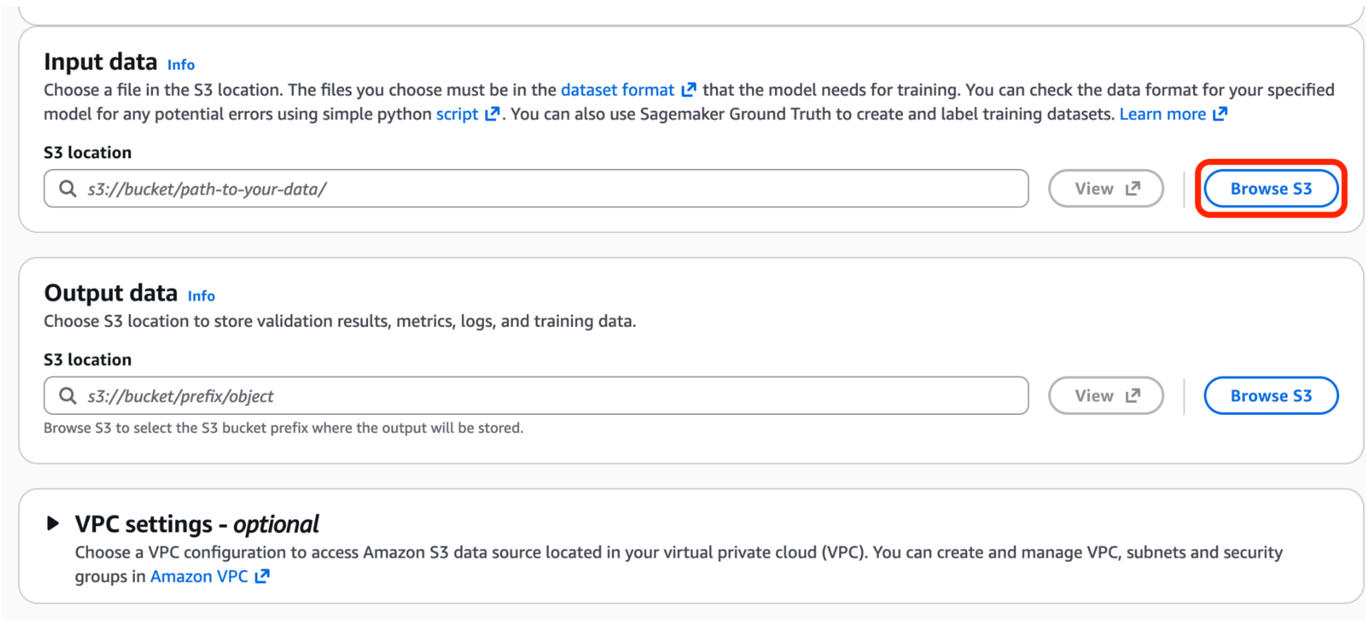


4. Verify that **Nova 2 Lite v1** is selected, enter `pharma-triage-model` as the **Fine-tuned model name**, then scroll down to **Input data**.

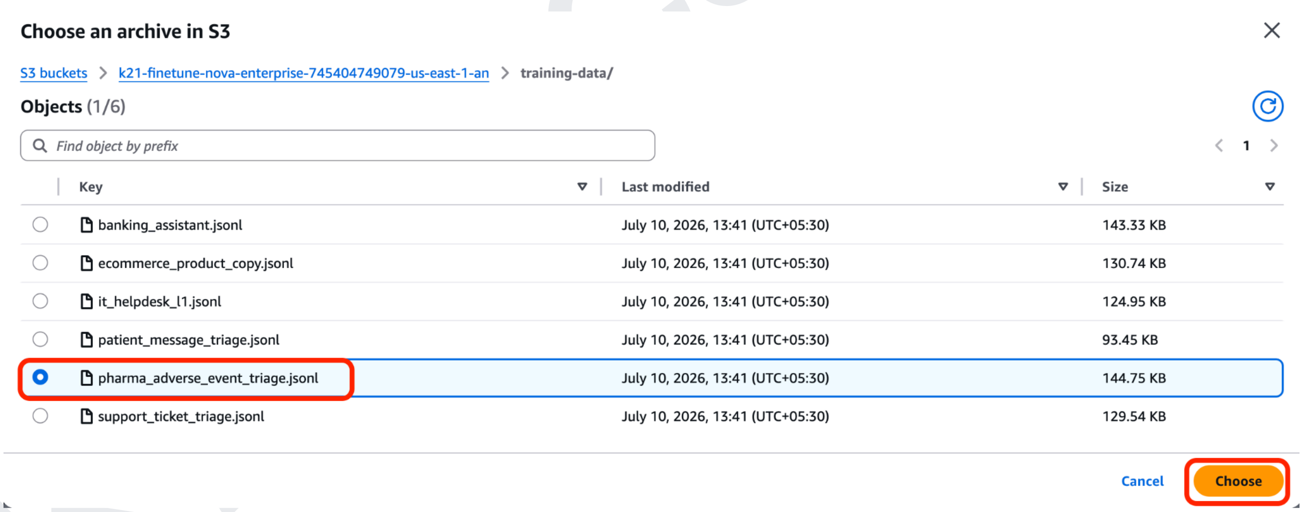


5. Hyperparameters: defaults.

6. Click **Browse S3** under **Input data** to select the training dataset from your Amazon S3 bucket. Then, choose an S3 location under **Output data** where the fine-tuning job results and model artifacts will be stored.

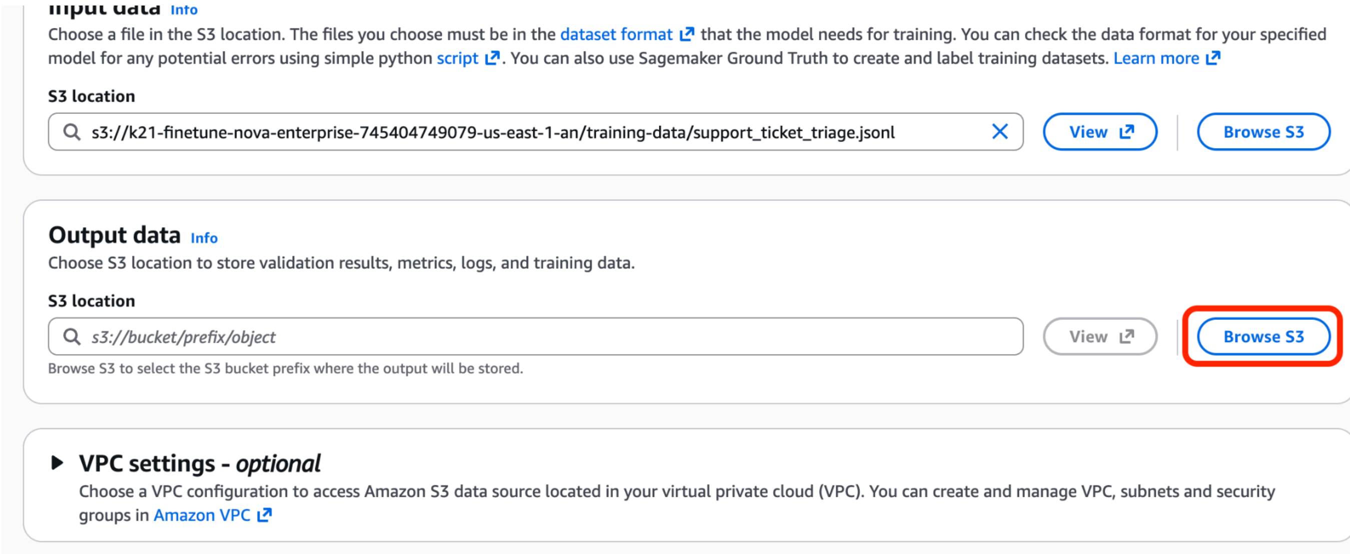


7. Select `pharma_adverse_event_triage.jsonl`, then click **Choose**.

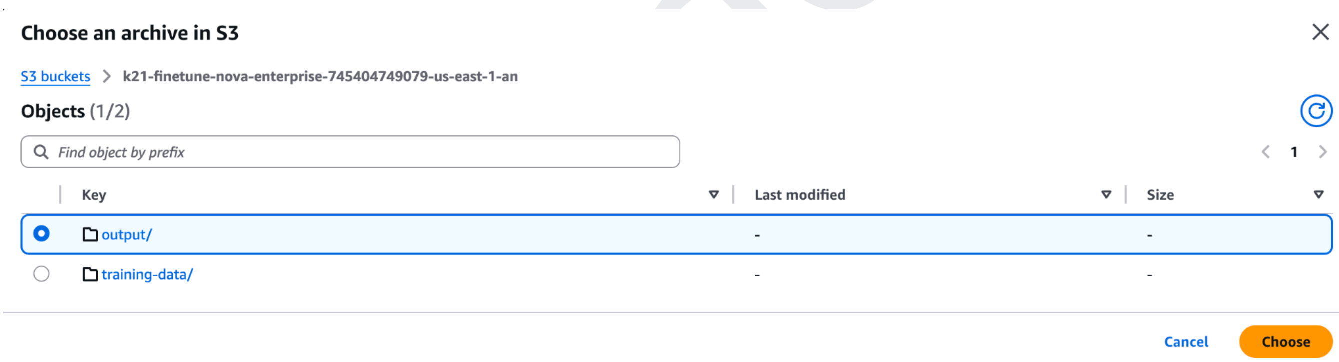


8. For the **Output data** location, click **Browse S3** and select the S3 folder where the fine-tuning outputs, logs, and model artifacts will be stored (for example,

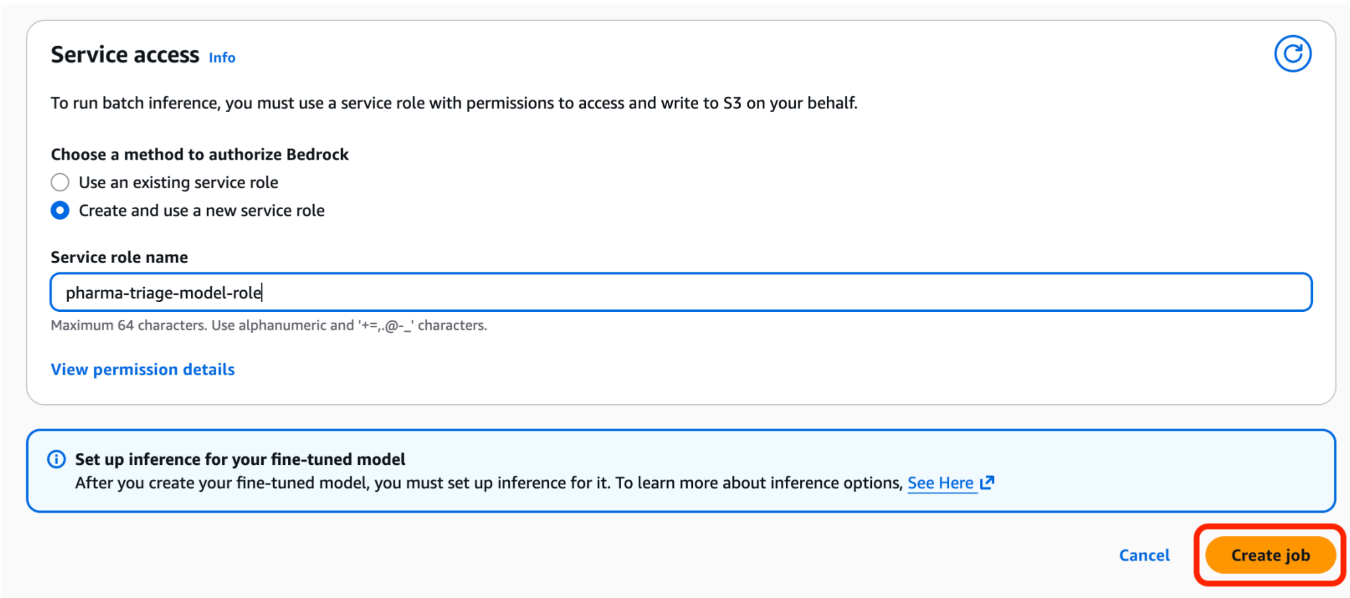
s3://k21-finetune-nova-enterprise/output/support-triage/).



9. Select the **output/** folder from the S3 bucket, then click **Choose** to use it as the destination for storing the fine-tuning job outputs.

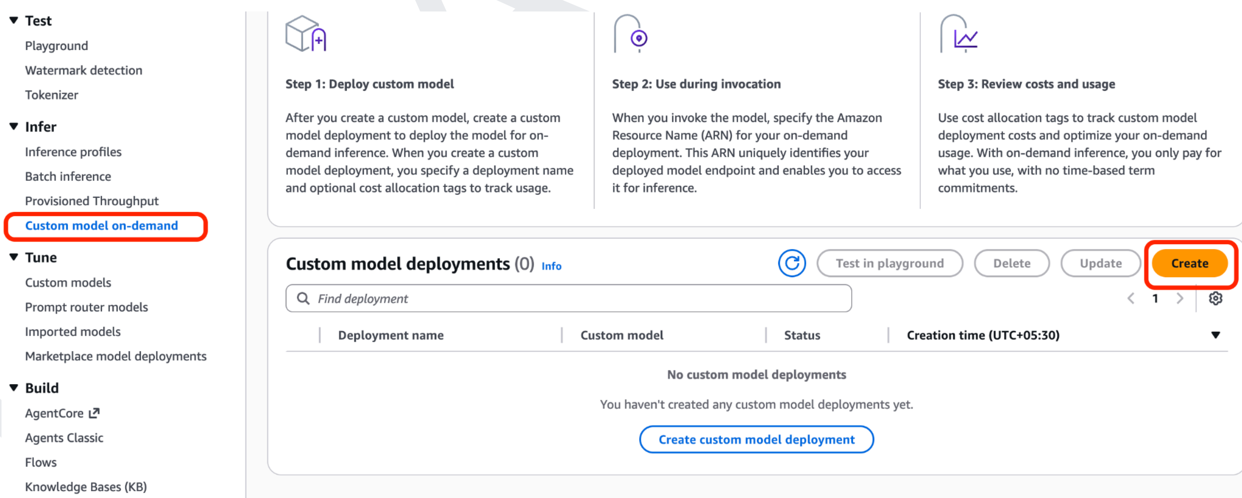


10. Select **Create and use a new service role**, enter `pharma-triage-model-role` as the service role name, and click **Create job**.

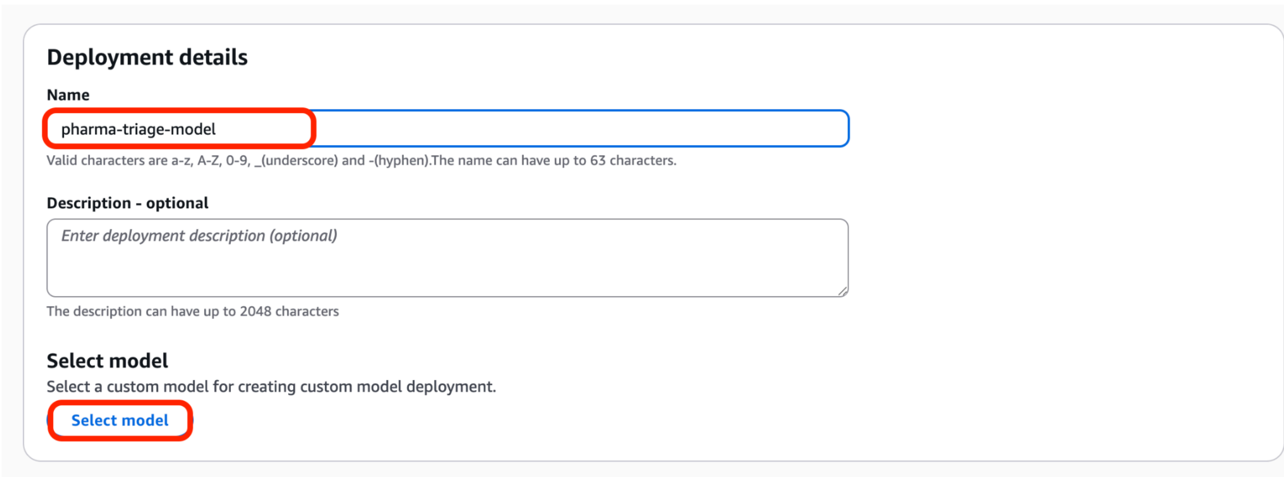


Note: For us, it took 4 hours to be in Active Status.

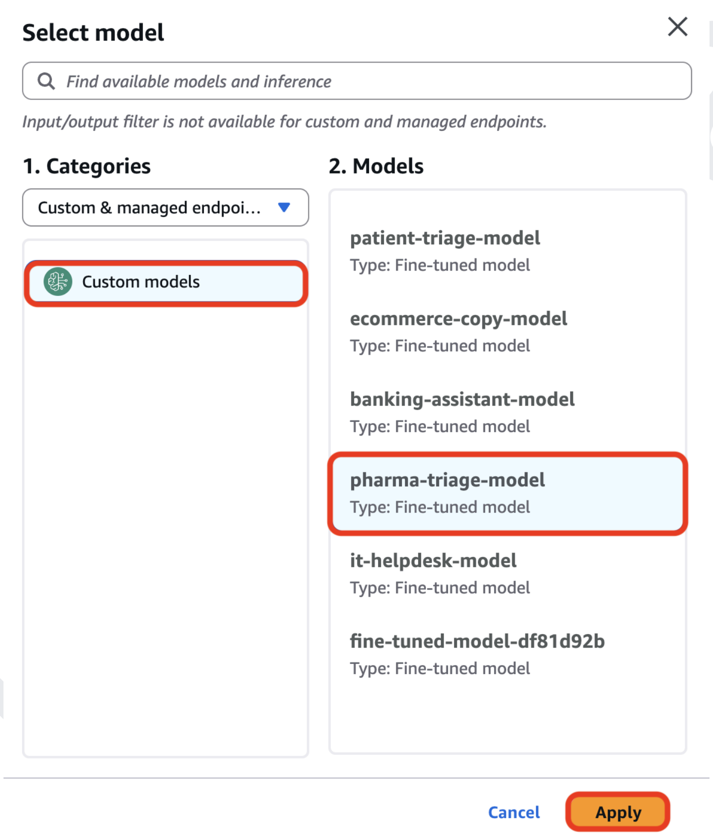
11. Deploy for on-demand inference
12. In the Bedrock dashboard, under **Infer**, choose **Custom model on demand**.



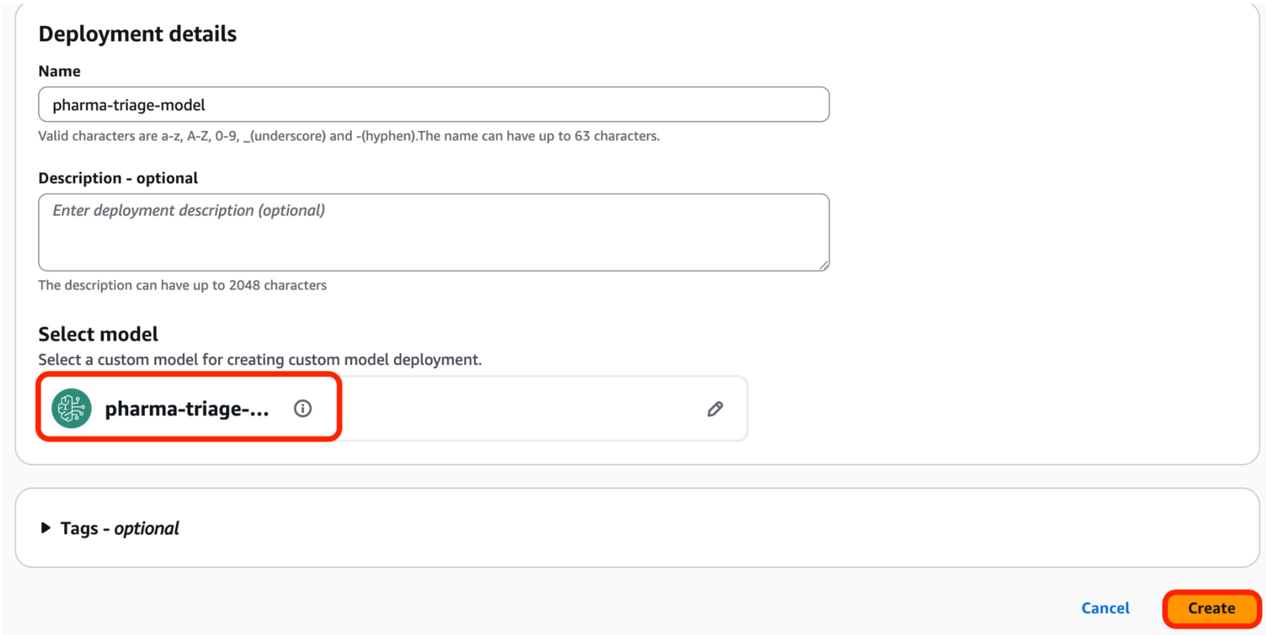
13. In **Deployment details**, enter `pharma-triage-model` as the deployment name, then click **Select model**.



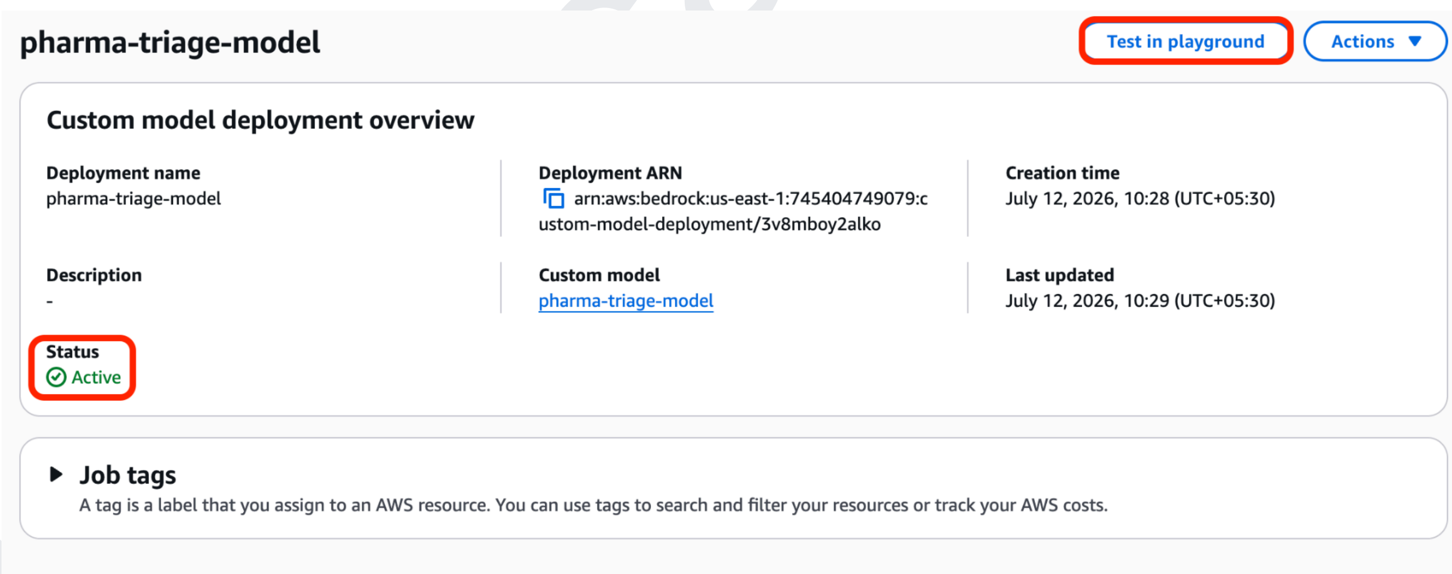
14. Select **Custom models**, choose `pharma-triage-model`, and click **Apply**.



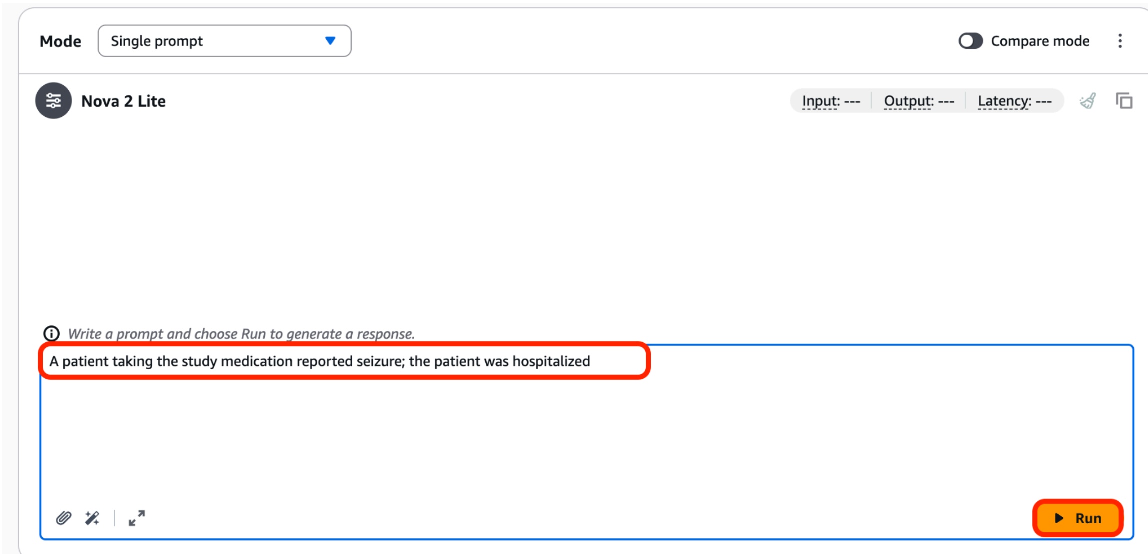
15. Verify that **pharma-triage-model** is selected, then click **Create** to deploy the custom model.



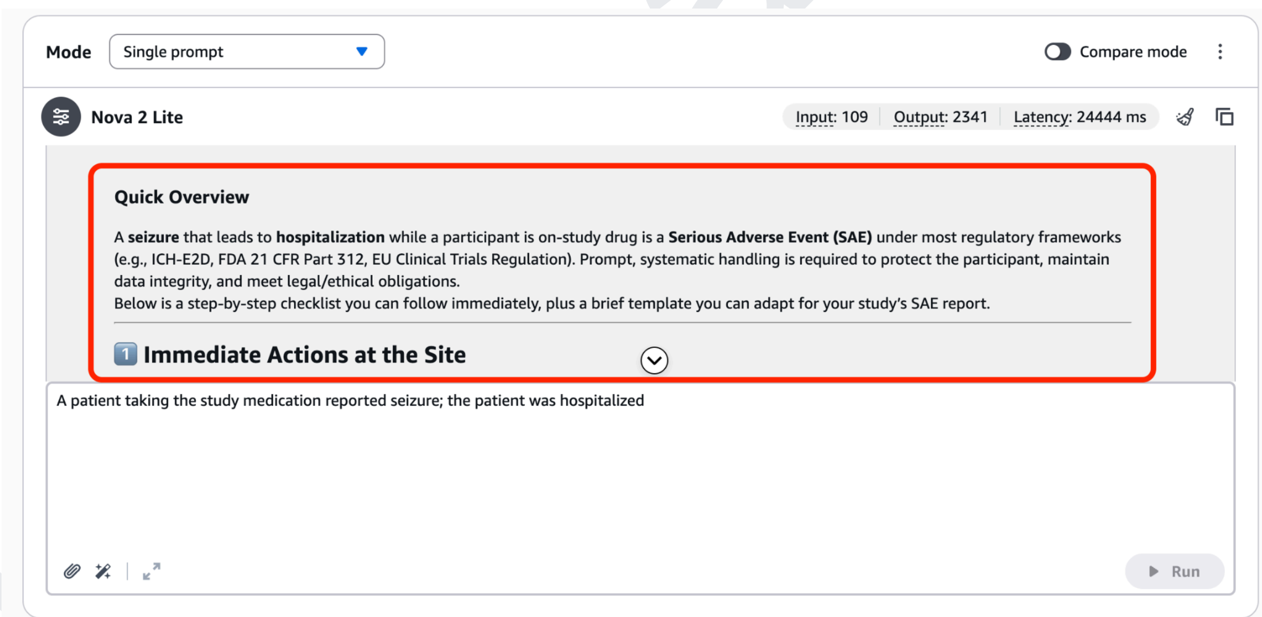
16. Once the deployment status changes to **Active**, click **Test in playground** to test your fine-tuned model.



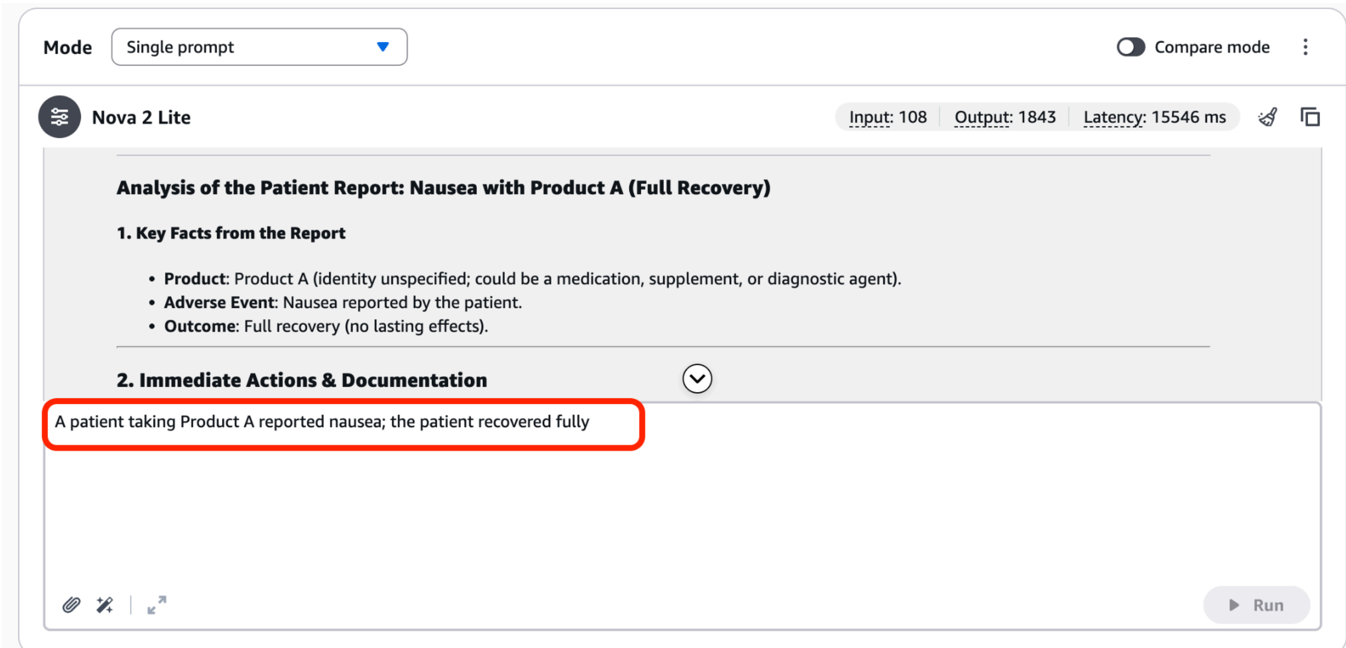
17. In the playground, enter the prompt **"A patient taking the study medication reported seizure; the patient was hospitalized"**, then click **Run**.



18. Verify that the model correctly identifies the event as a **Serious Adverse Event (SAE)** and provides appropriate guidance for handling and reporting it.



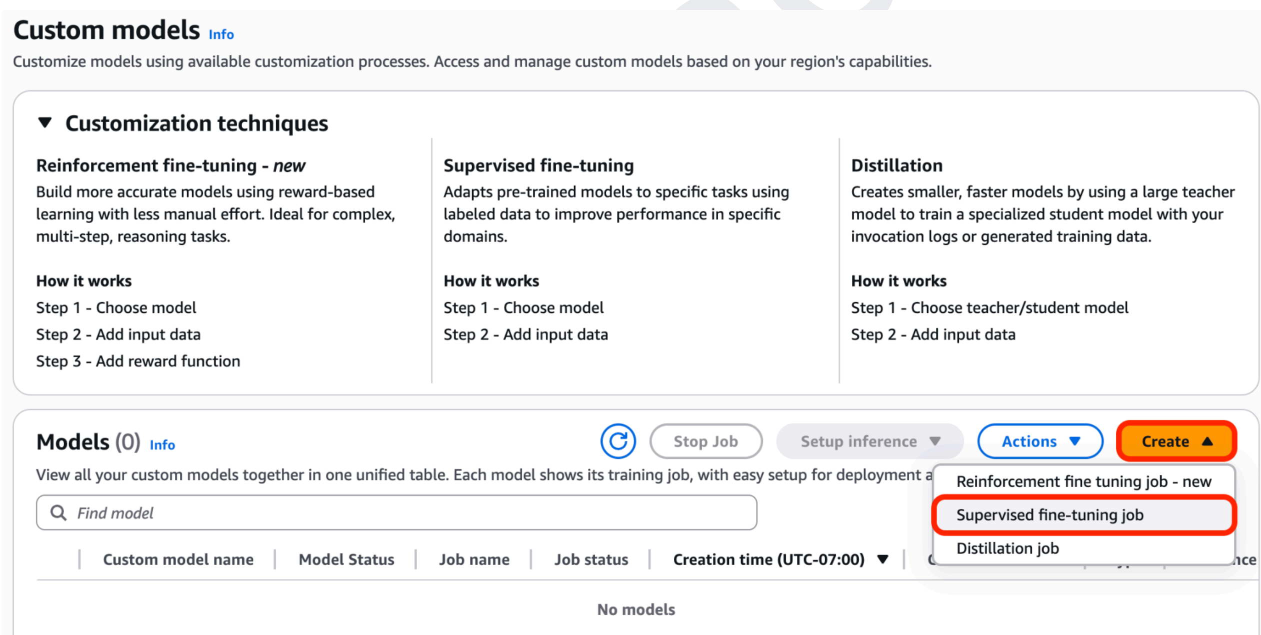
19. Verify that the model correctly identifies the adverse event as **non-serious**, summarizes the key details, and recommends appropriate documentation and follow-up actions.



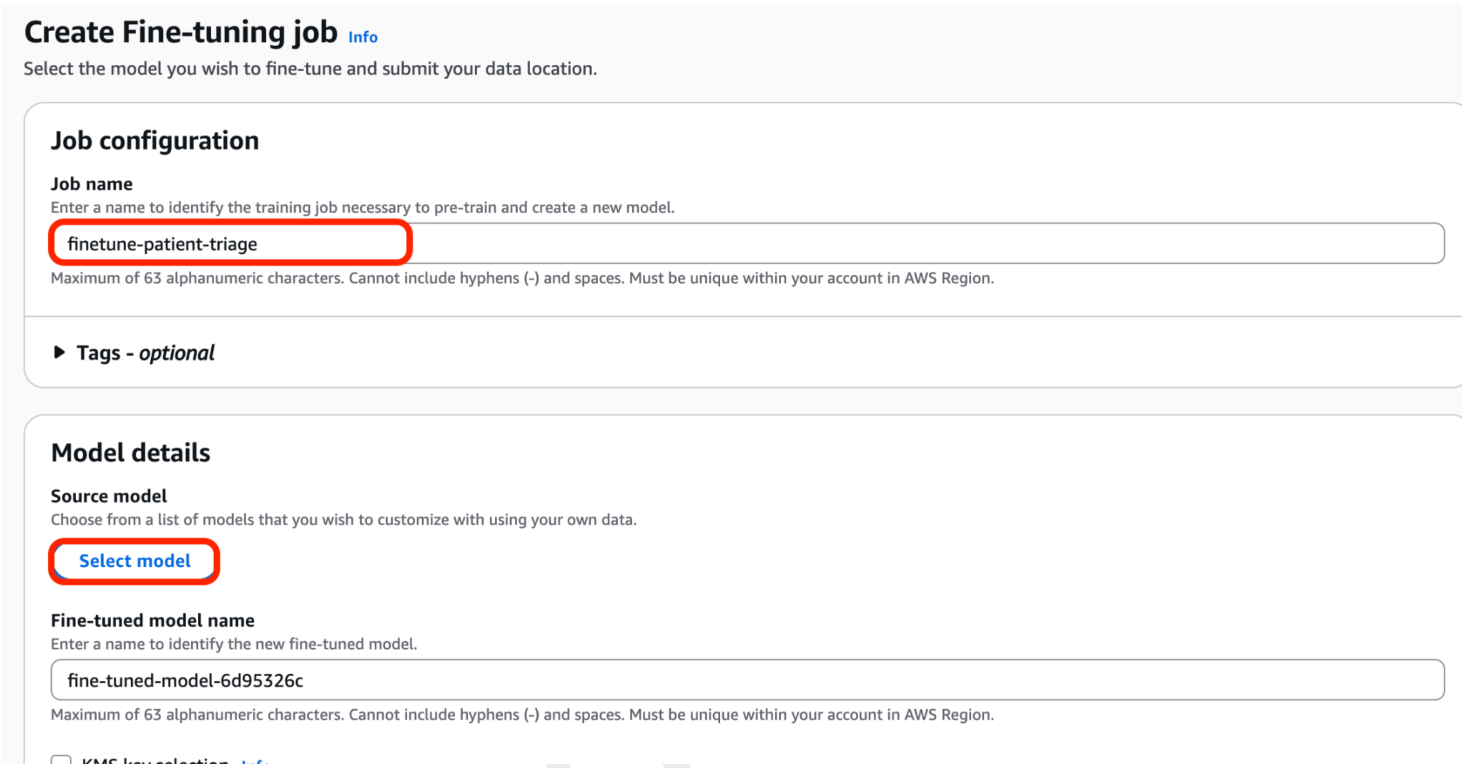
15. Scenario 4: Patient Message Triage (Clinic Routing)

This scenario trains the model to read incoming patient messages and route them to the correct care pathway based on symptom urgency - distinguishing between medical emergencies, urgent care needs, routine appointments, and general administrative inquiries. In busy clinics and telehealth platforms, care coordinators are often overwhelmed by the volume of patient messages and the need to correctly prioritize responses. A fine-tuned triage model can instantly flag life-threatening symptoms like chest pain for emergency escalation, while directing routine appointment requests to scheduling staff. This improves patient safety, reduces triage errors, and helps clinical teams allocate their attention where it matters most.

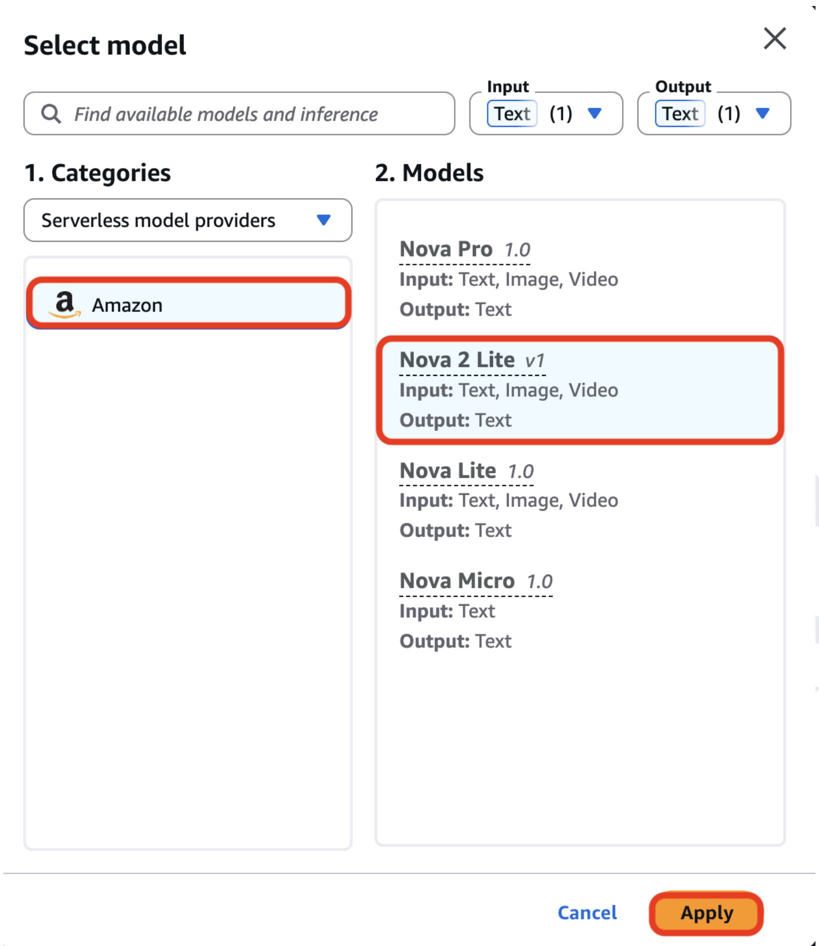
1. Click **Create**, then select **Supervised fine-tuning job** to create a new custom model training job.



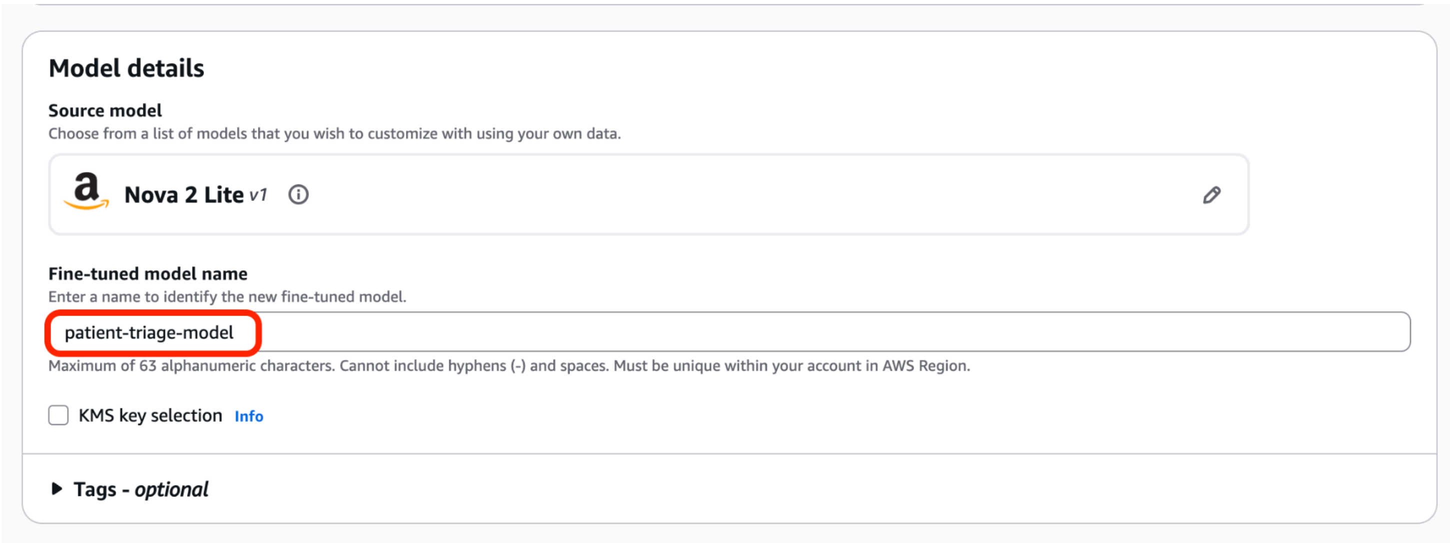
2. In the **Job name** field, enter **`finetune-patient-triage`**, then click **Select model** to choose the foundation model for fine-tuning.



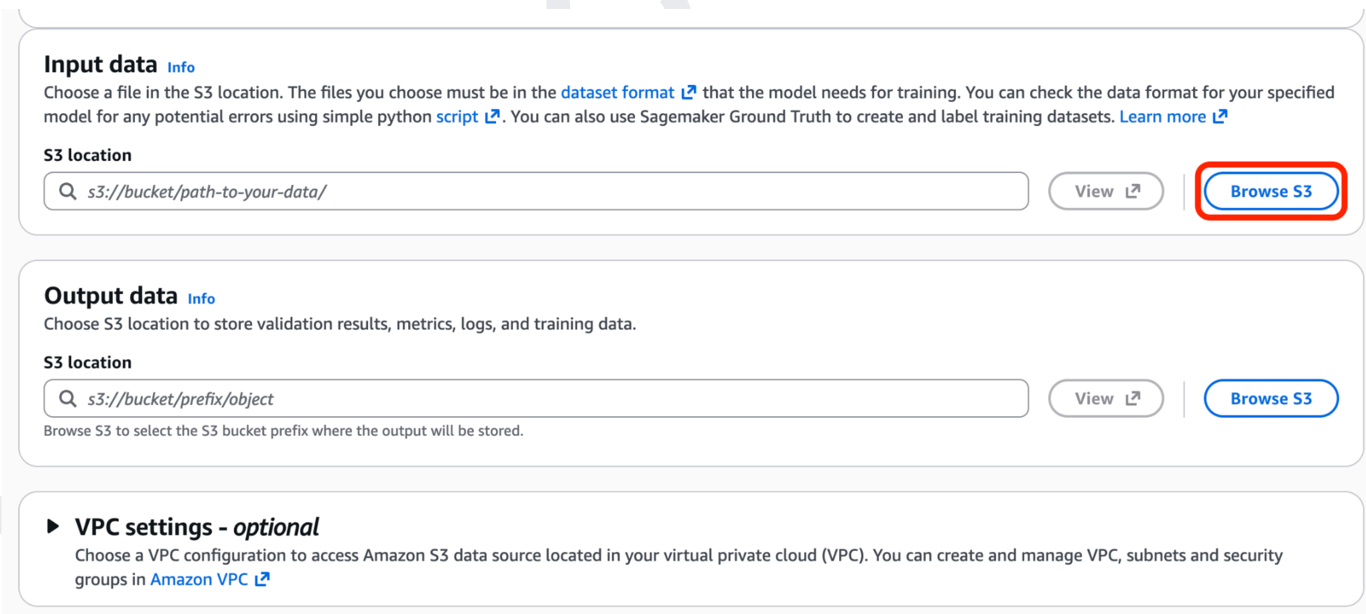
3. Select **Amazon** as the model provider, choose **Nova 2 Lite v1** as the source model, and click **Apply**.



4. Confirm that **Nova 2 Lite v1** is selected as the source model. In the **Fine-tuned model name** field, enter `patient-triage-model`, then continue to the training data configuration.

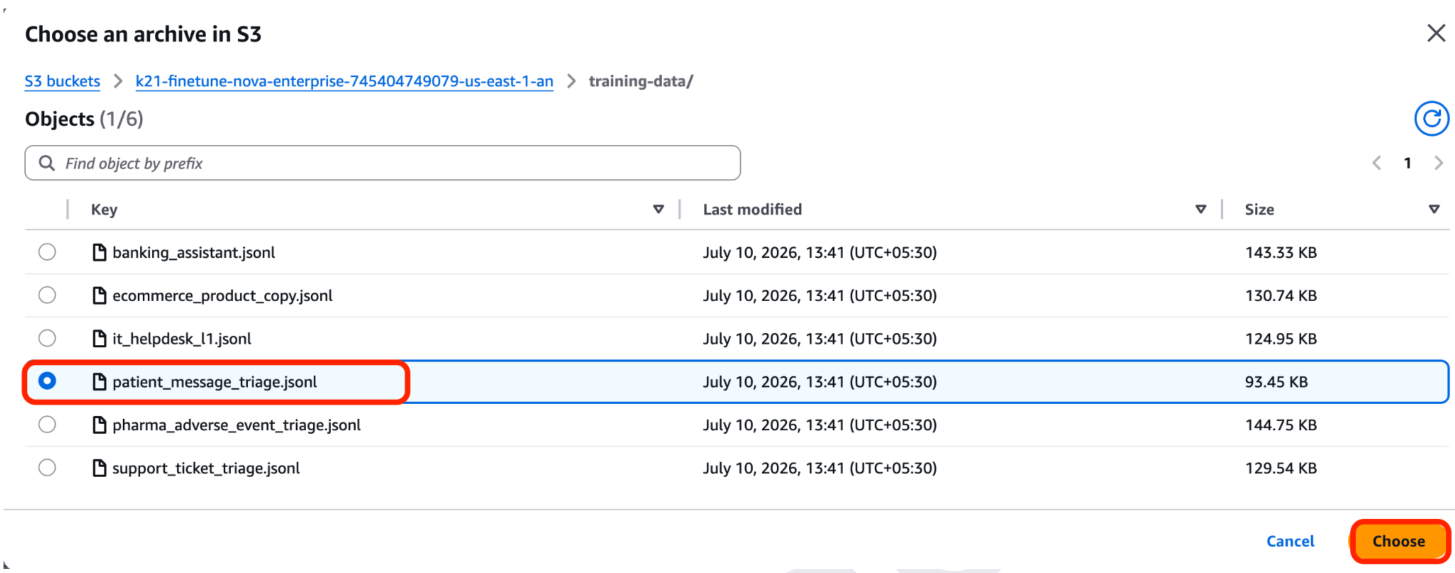


5. Hyperparameters: defaults.
6. Click **Browse S3** under **Input data** to select the training dataset from your Amazon S3 bucket. Then, choose an S3 location under **Output data** where the fine-tuning job results and model artifacts will be stored.

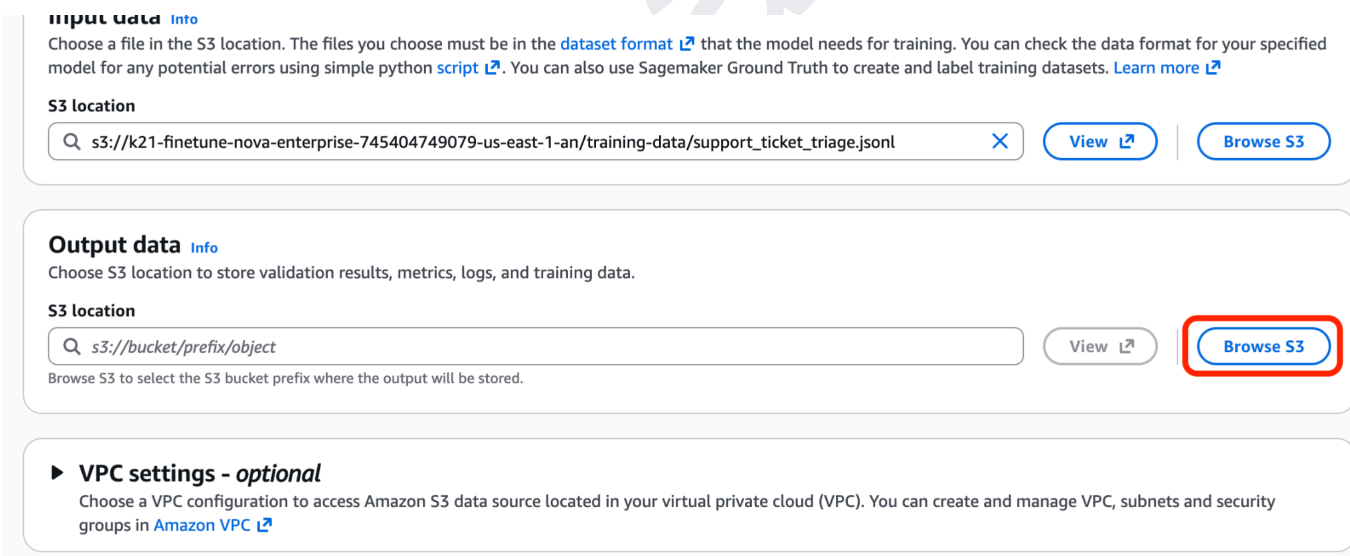


7. Under **Training data**, select **Choose** and browse to your S3 bucket. Select the `patient_message_triage.jsonl` training dataset, then click **Choose** to use it for

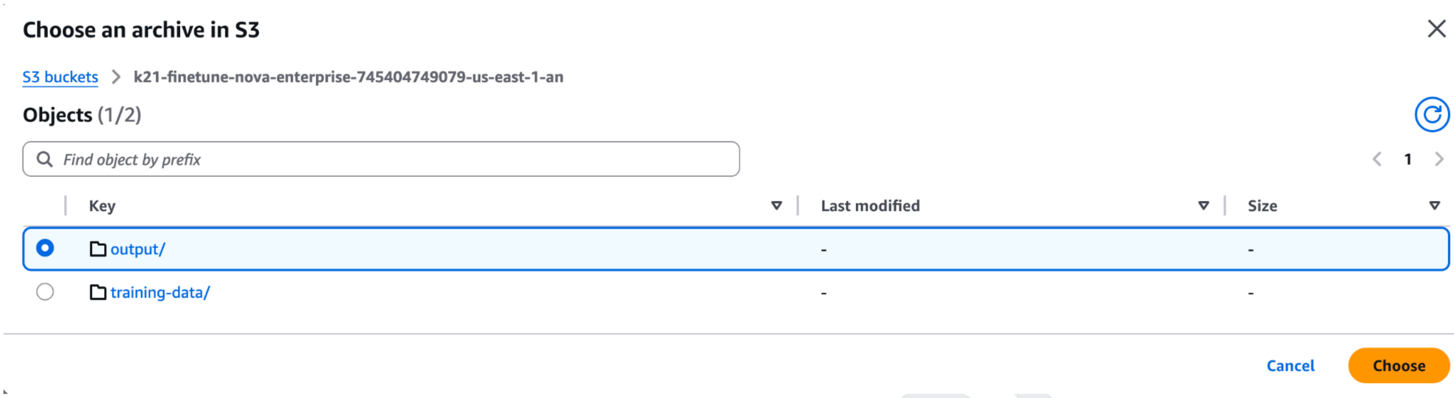
fine-tuning.



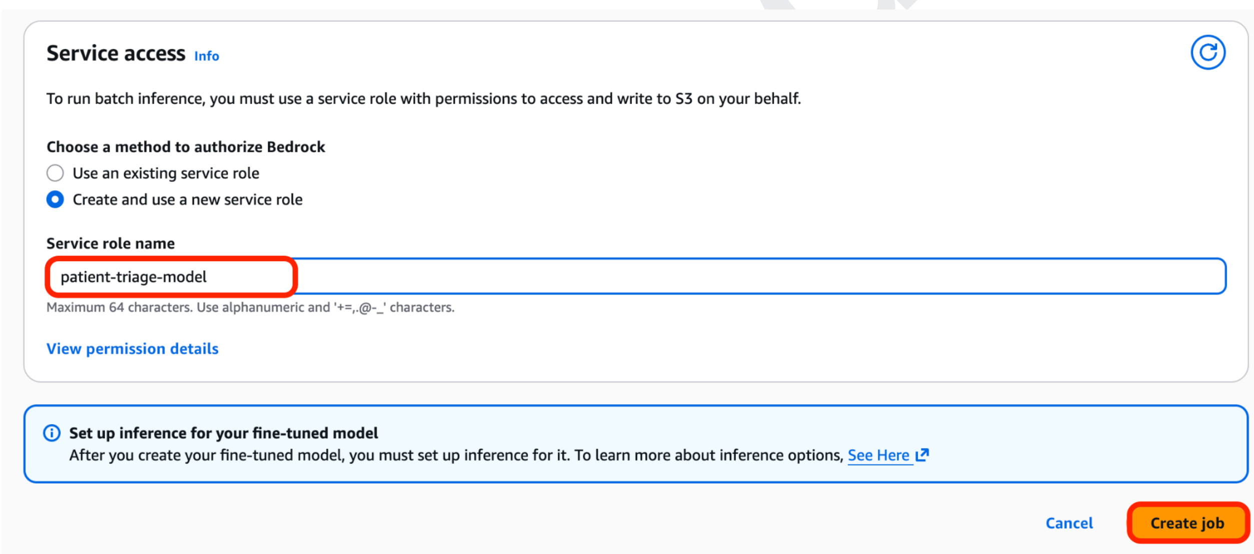
8. For the **Output data** location, click **Browse S3** and select the S3 folder where the fine-tuning outputs, logs, and model artifacts will be stored (for example, s3://k21-finetune-nova-enterprise/output/support-triage/).



9. Select the **output/** folder from the S3 bucket, then click **Choose** to use it as the destination for storing the fine-tuning job outputs.



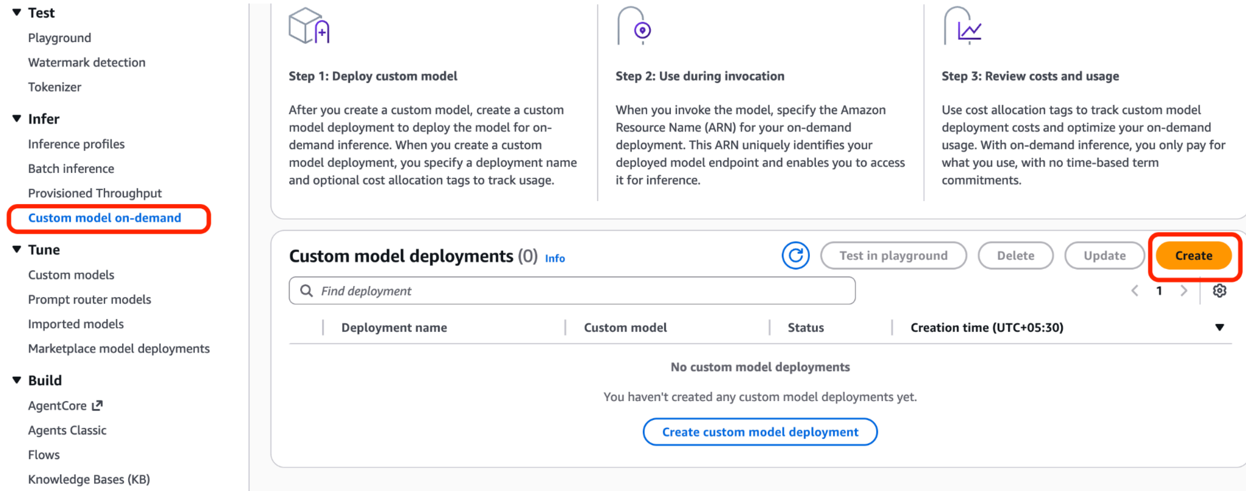
10. Under **Service access**, select **Create and use a new service role**, enter `patient-triage-model` as the service role name, then click **Create job** to start the fine-tuning process.



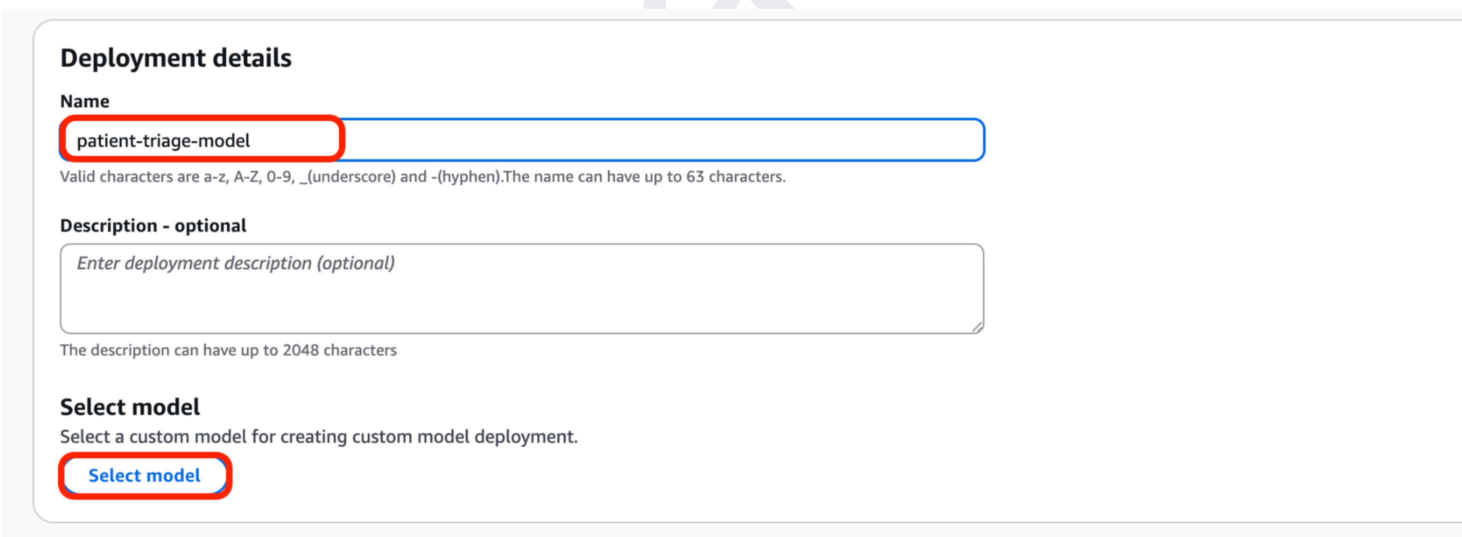
Note: For us, it took 4 hours to be in Active Status.

11. Deploy for on-demand inference

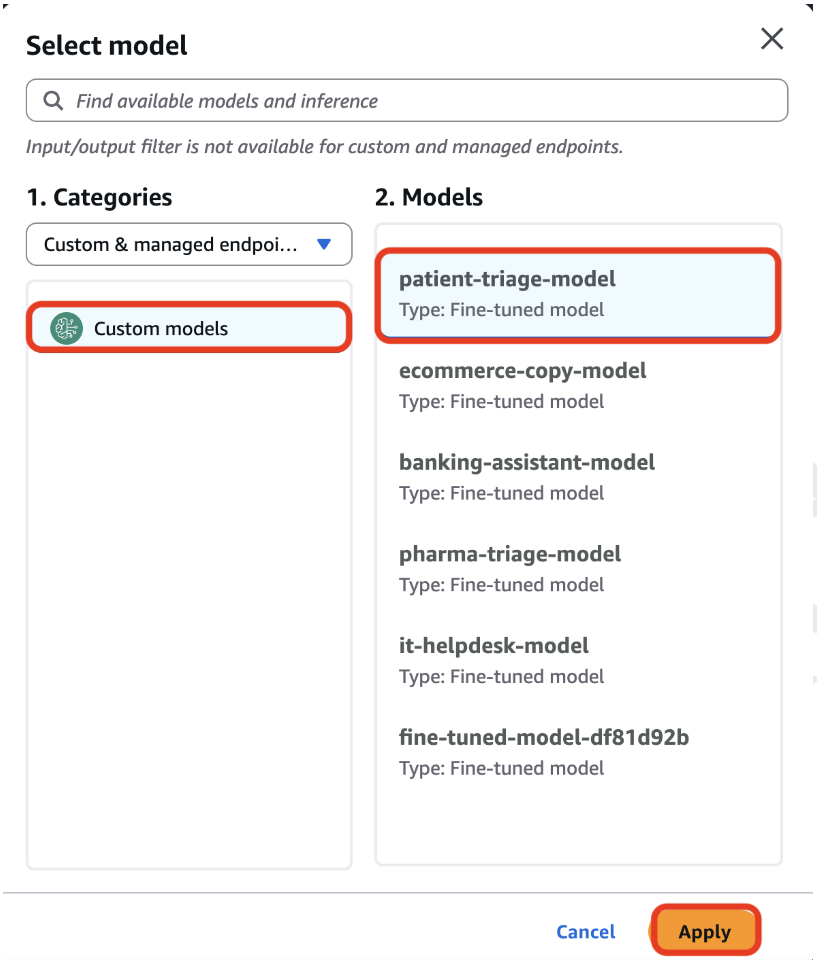
12. In the Bedrock dashboard, under **Infer**, choose **Custom model on demand**.



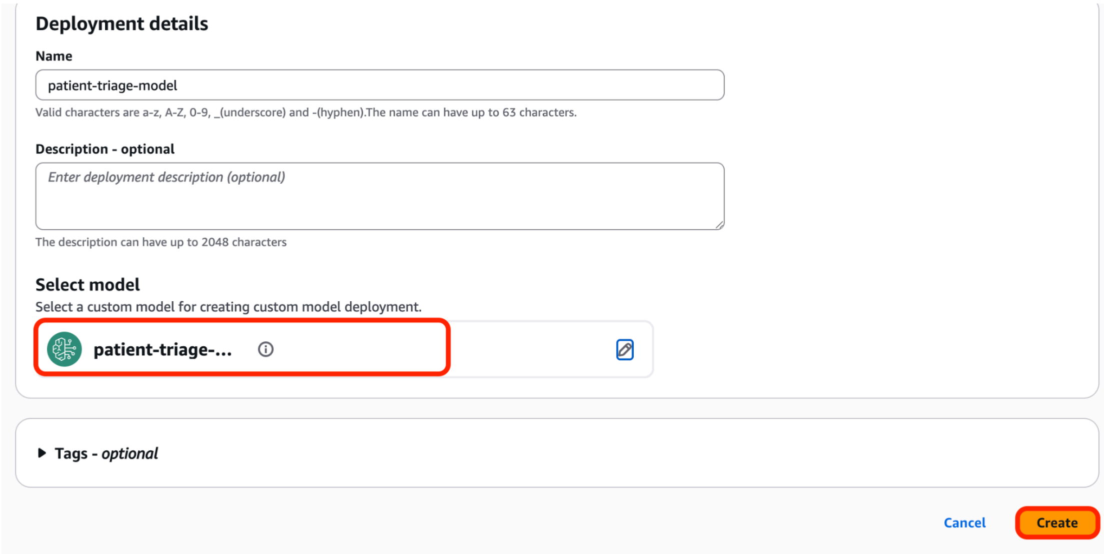
13. Navigate to **Custom model on-demand** under the **Infer** section. In **Deployment details**, enter `patient-triage-model` as the deployment name, then click **Select model** to choose the fine-tuned model for deployment.



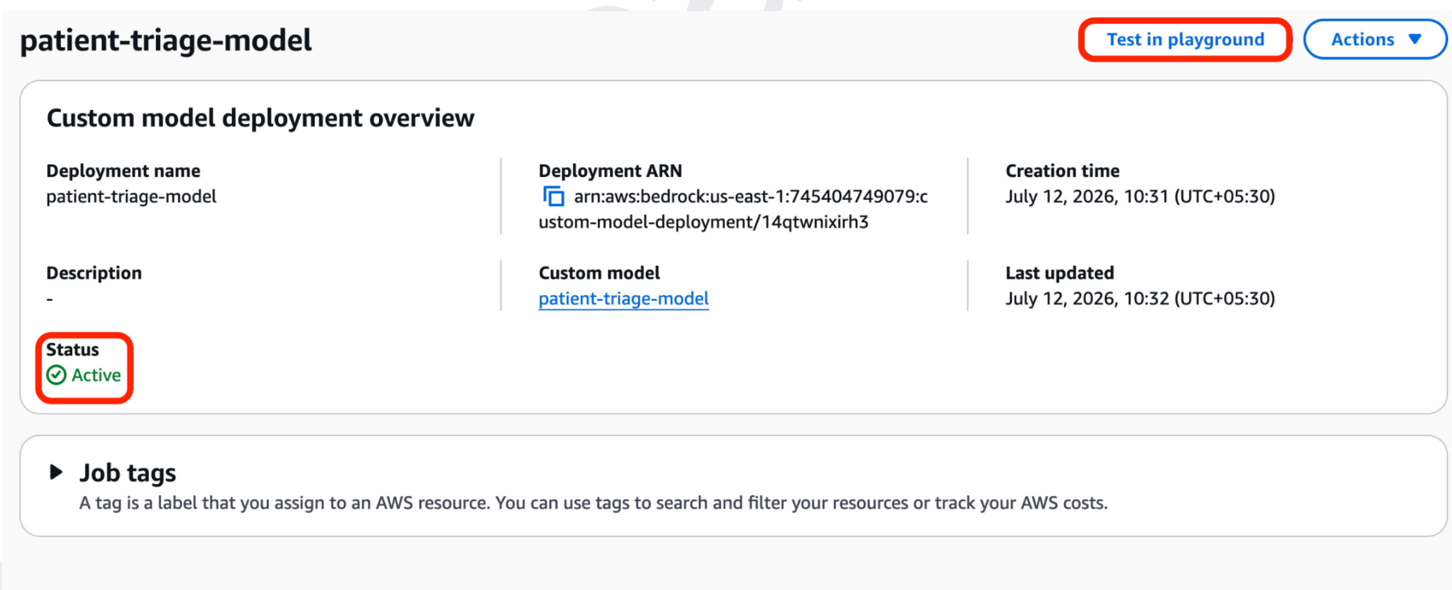
14. In the **Select model** dialog, choose **Custom models**, select `patient-triage-model` from the list of fine-tuned models, and click **Apply** to use it for deployment.



15. Verify that `patient-triage-model` is selected under **Select model**, then click **Create** to deploy the fine-tuned model for on-demand inference.

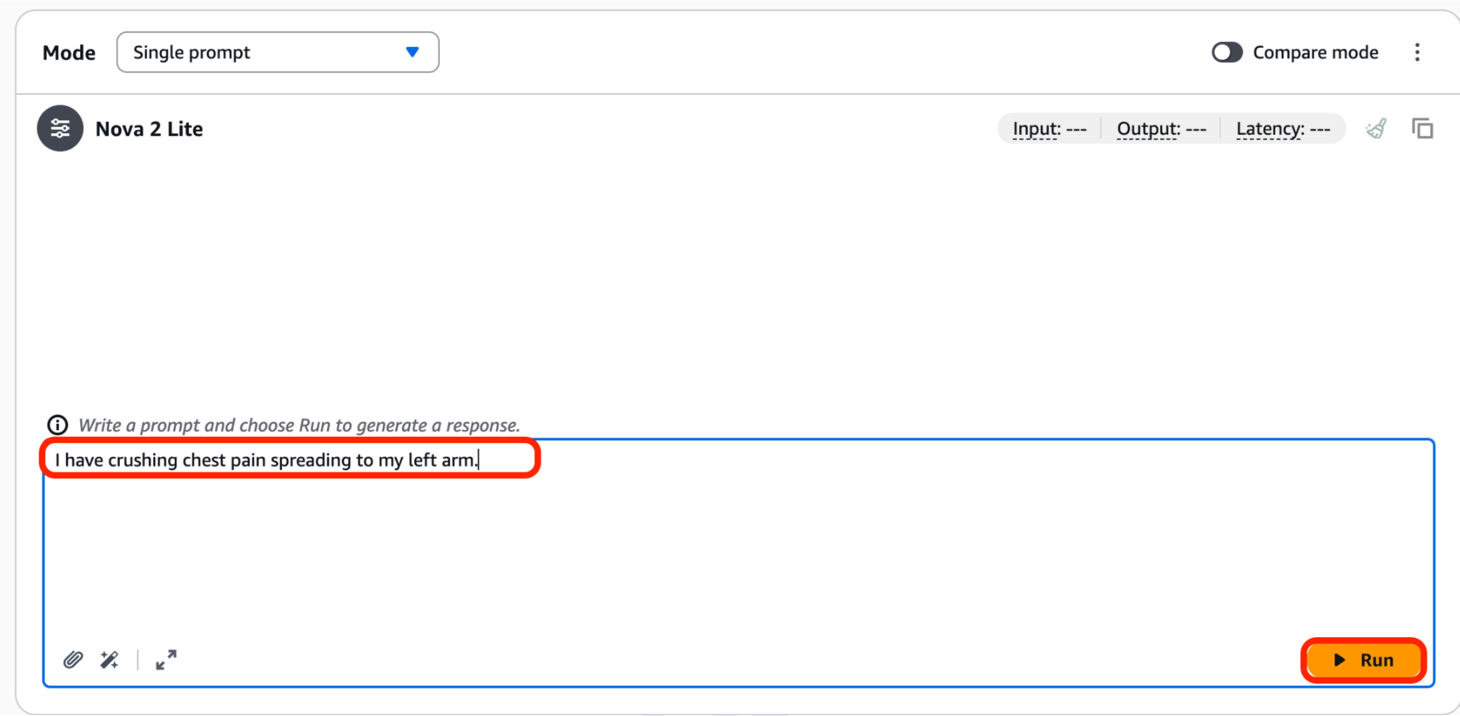


16. Test in the Playground
17. Wait until the deployment status changes to **Active**, then click **Test in playground** to verify that the deployed fine-tuned model is ready for inference.

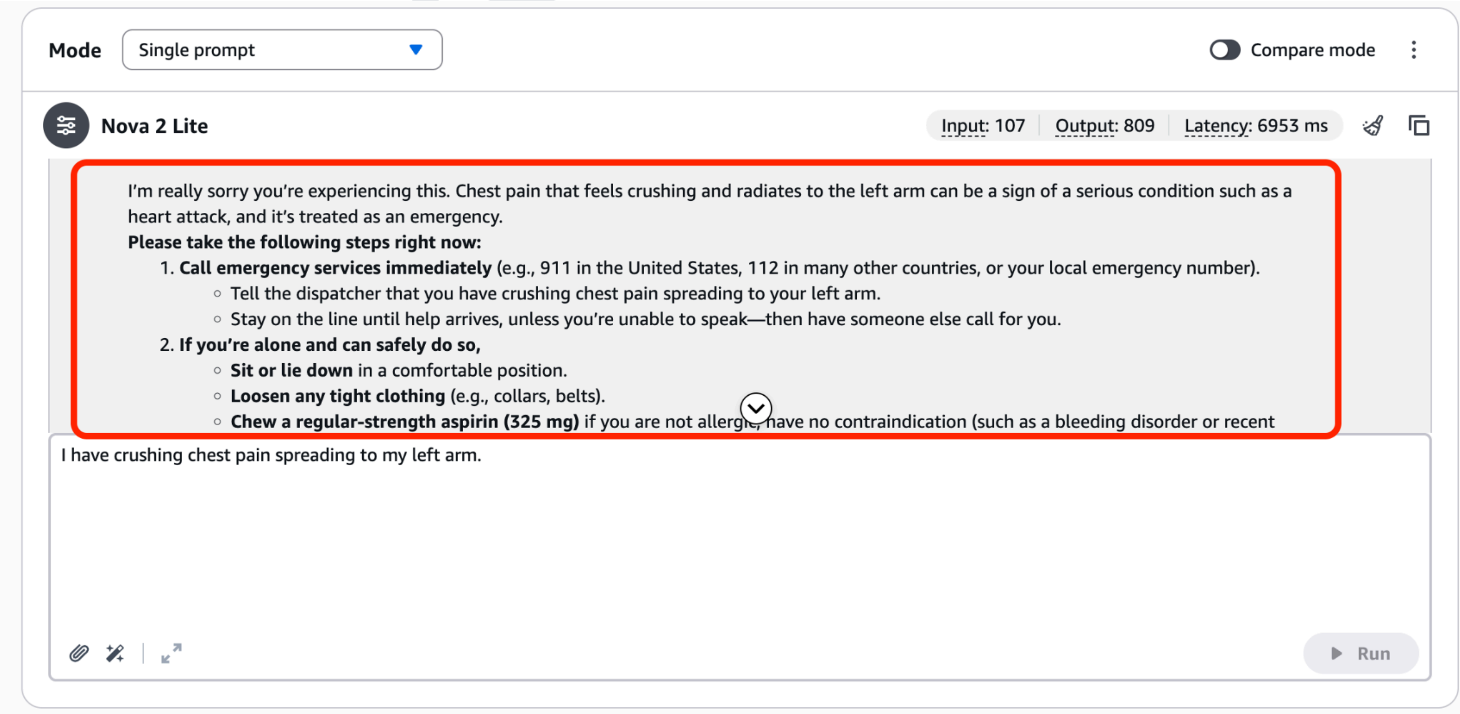


18. In the playground, enter the prompt **"I have crushing chest pain spreading to my left arm."** and click **Run** to verify that the fine-tuned model correctly recognizes the symptoms as a potential medical emergency and provides an appropriate triage

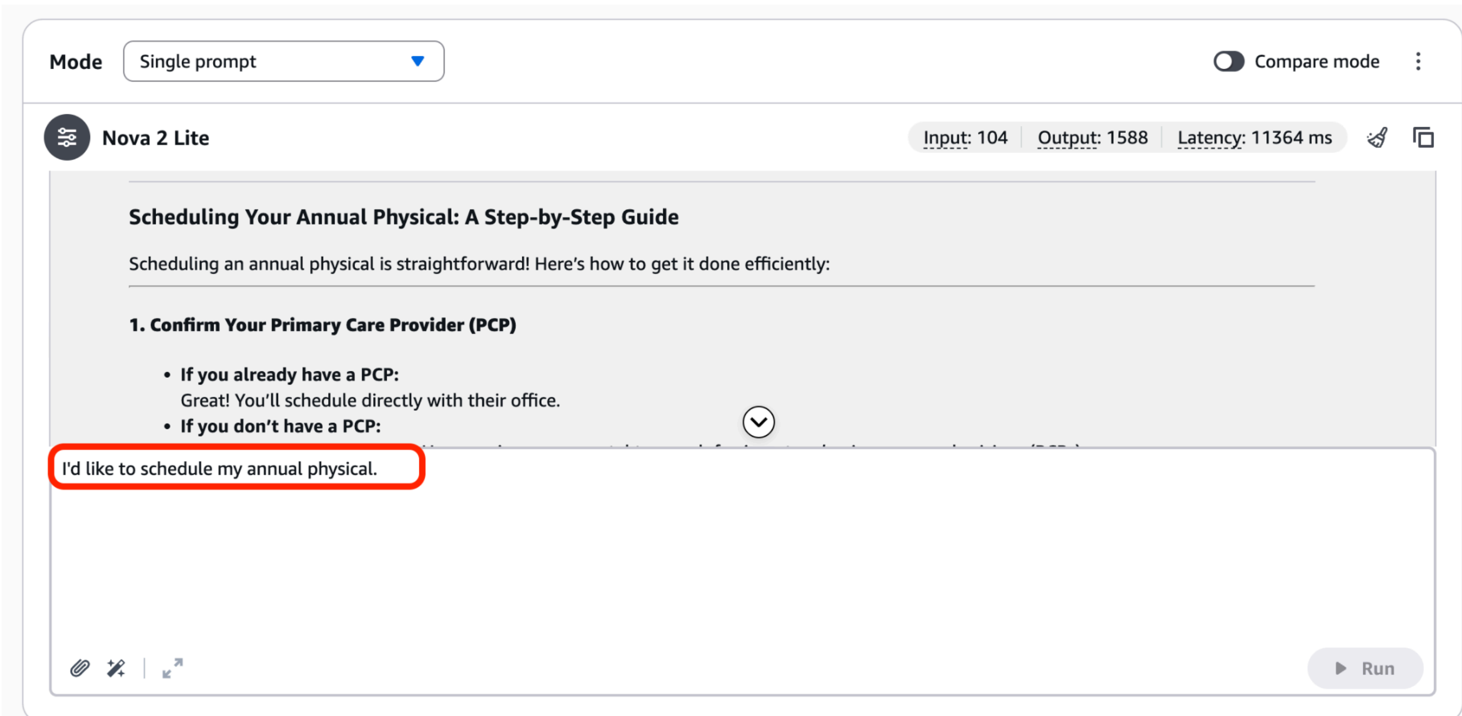
response.



19. Verify that the model identifies the symptoms as a **medical emergency**, recommends seeking **immediate emergency medical assistance**, and provides appropriate first-aid guidance while waiting for help.



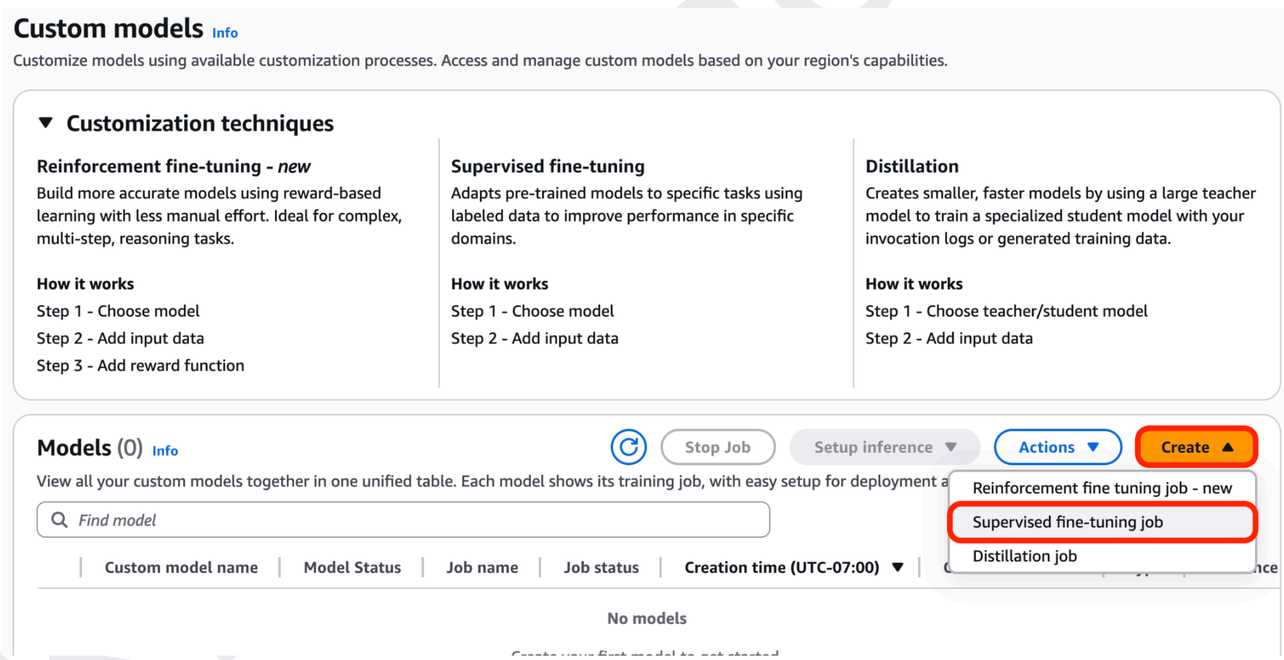
20. Verify that the model provides clear, step-by-step guidance for scheduling an annual physical, including contacting a primary care provider, confirming insurance coverage, and preparing for the appointment.



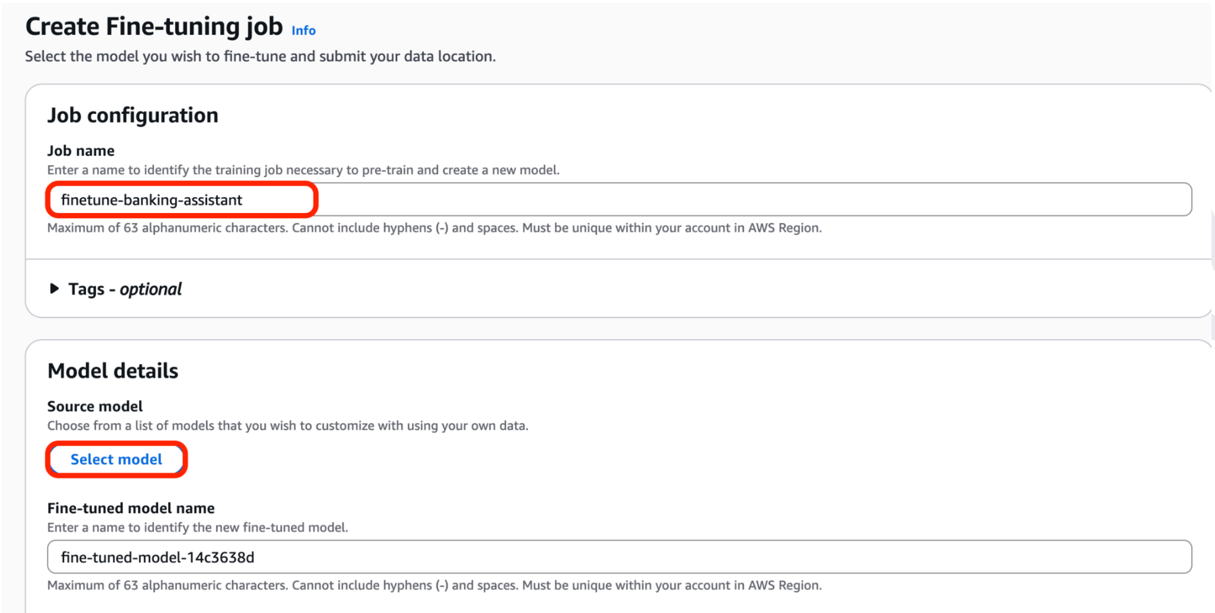
16. Scenario 5: Banking Virtual Assistant

This scenario customizes the model to handle common retail banking queries - such as fund transfers, account management, transaction disputes, and general investment guidance - in a way that is accurate, compliant, and customer-friendly. Banking customers expect fast, reliable answers at any time of day, and contact centers handling these queries at scale are expensive to operate. A fine-tuned banking assistant can resolve the majority of routine questions without agent involvement, while also being trained to handle sensitive topics like investment advice responsibly — providing educational guidance rather than specific recommendations, and always directing users to consult a qualified financial advisor when appropriate.

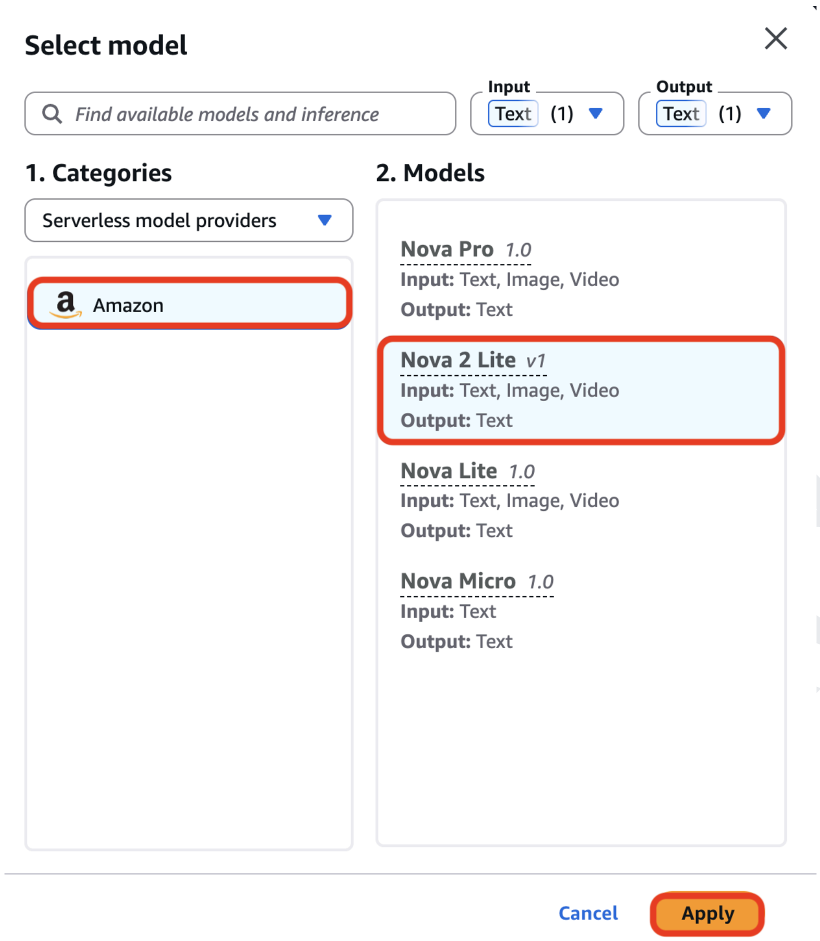
1. Click **Create**, then select **Supervised fine-tuning job** to create a new custom model training job.



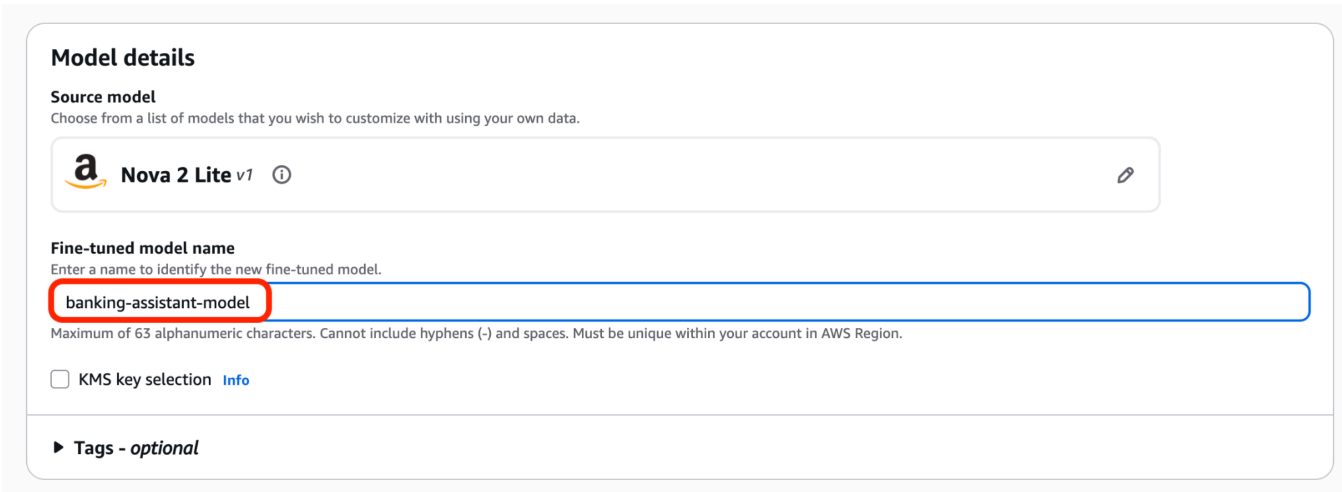
2. Enter the job name `finetune-banking-assistant`, then click **Select model** to choose the base model that will be fine-tuned for the banking assistant use case.


Create Fine-tuning job Info
Select the model you wish to fine-tune and submit your data location.
Job configuration
Job name
Enter a name to identify the training job necessary to pre-train and create a new model.
finetune-banking-assistant
Maximum of 63 alphanumeric characters. Cannot include hyphens (-) and spaces. Must be unique within your account in AWS Region.
Tags - optional
Model details
Source model
Choose from a list of models that you wish to customize with using your own data.
Select model
Fine-tuned model name
Enter a name to identify the new fine-tuned model.
fine-tuned-model-14c3638d
Maximum of 63 alphanumeric characters. Cannot include hyphens (-) and spaces. Must be unique within your account in AWS Region.

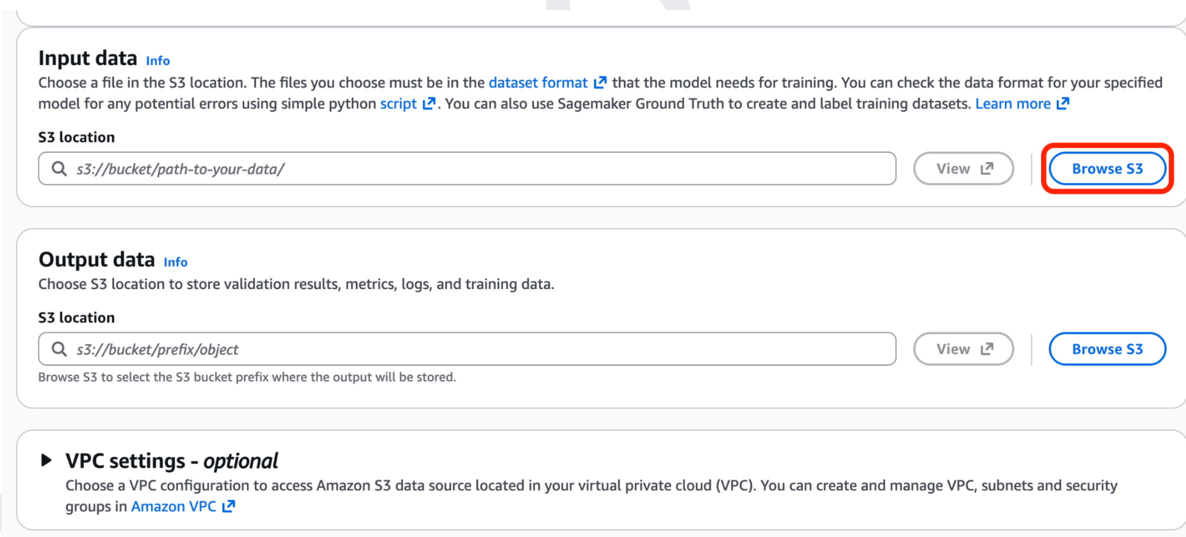
3. Select **Amazon** as the model provider, choose **Nova 2 Lite v1** as the source model, and click **Apply**



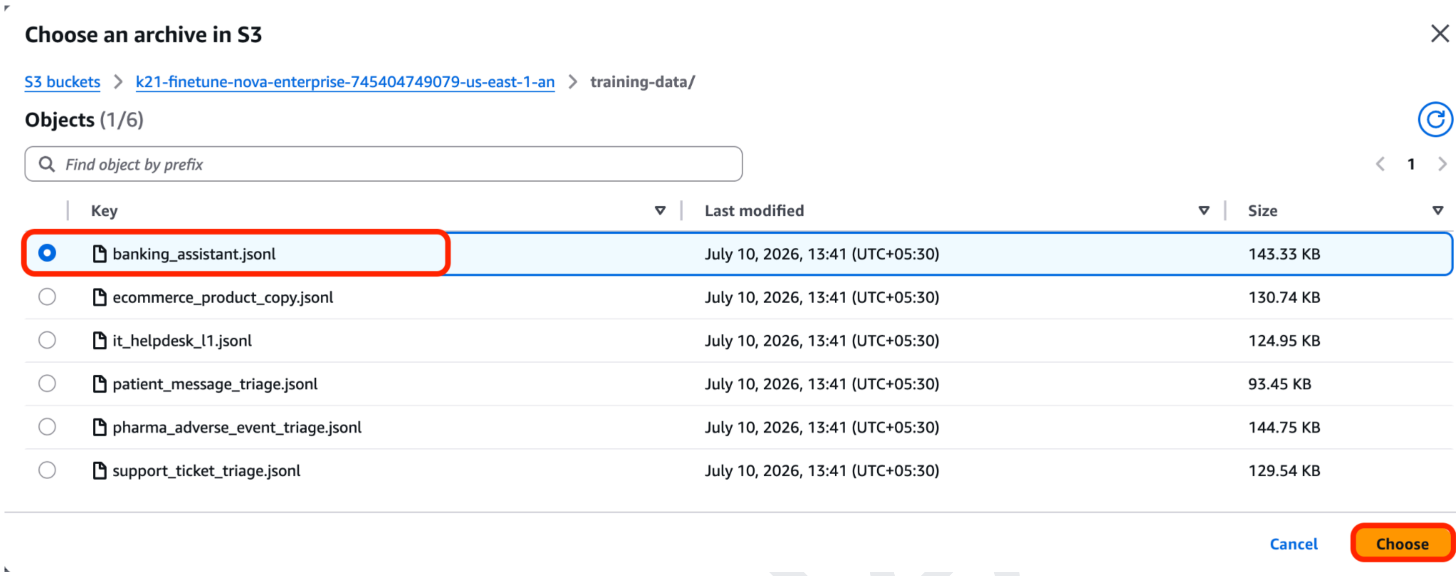
4. Verify that **Nova 2 Lite v1** is selected as the source model, enter `banking-assistant-model` as the fine-tuned model name, then proceed to the next step.



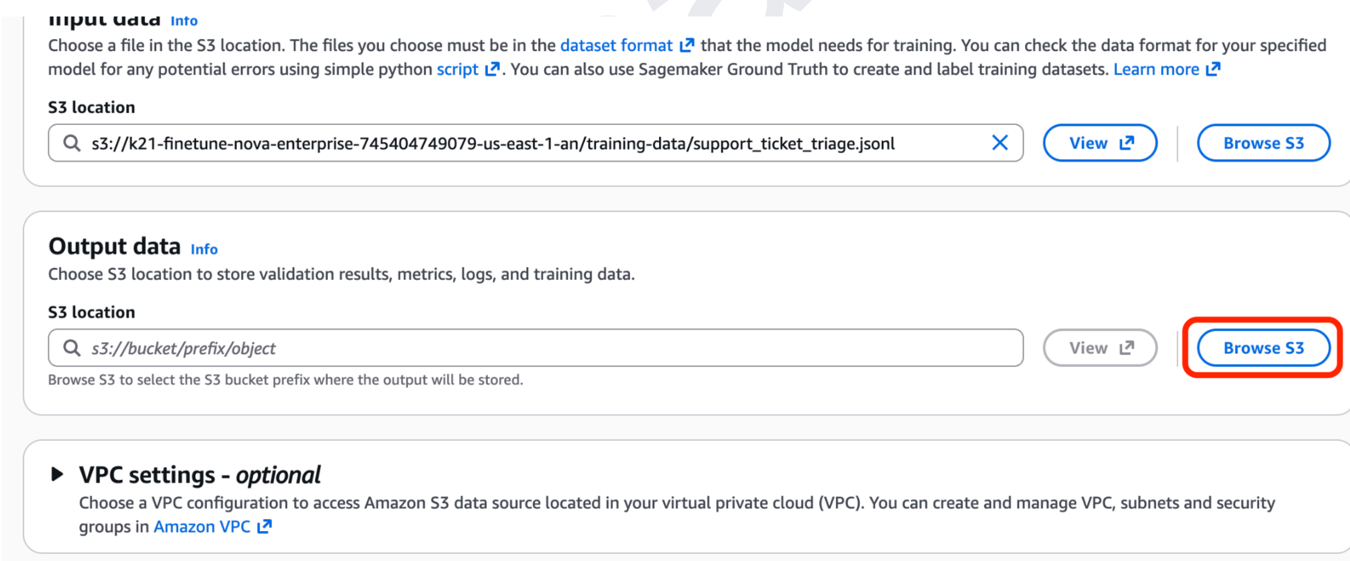
5. Hyperparameters: defaults.
6. Click **Browse S3** under **Input data** to select the training dataset from your Amazon S3 bucket. Then, choose an S3 location under **Output data** where the fine-tuning job results and model artifacts will be stored.



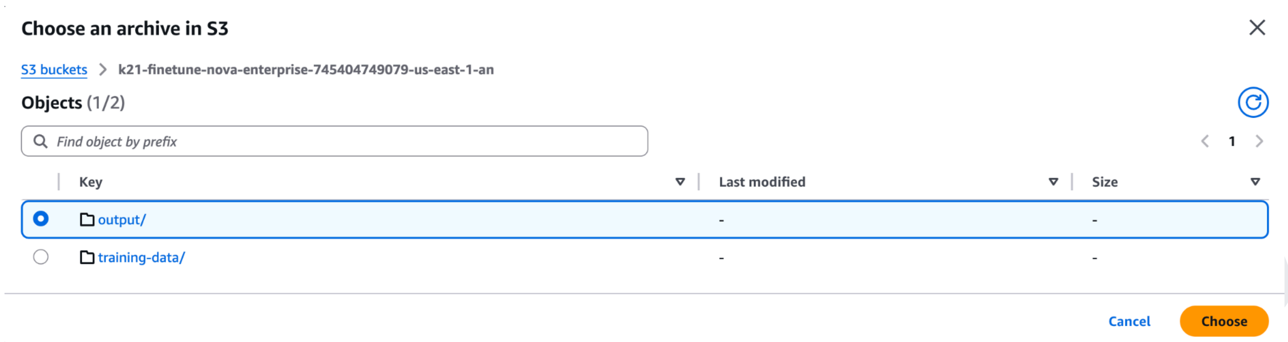
7. Select the `banking_assistant.jsonl` training dataset from the S3 bucket, then click **Choose** to use it as the training data for fine-tuning the banking assistant model.



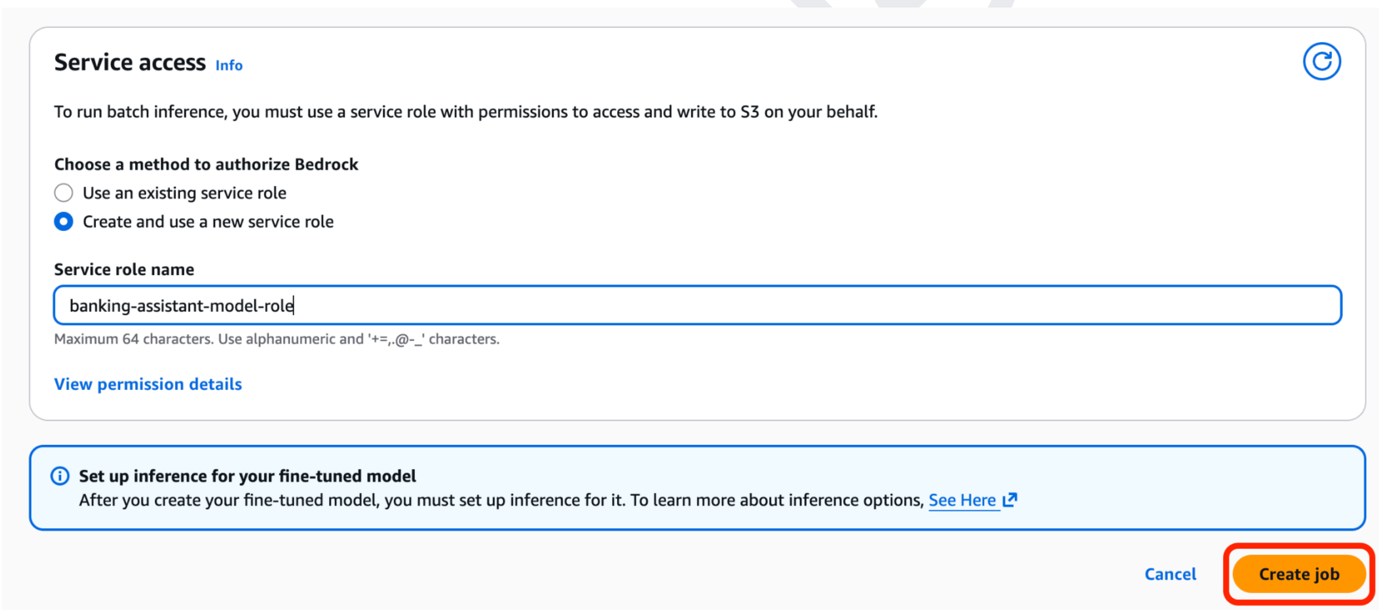
8. For the **Output data** location, click **Browse S3** and select the S3 folder where the fine-tuning outputs, logs, and model artifacts will be stored (for example, s3://k21-finetune-nova-enterprise/output/support-triage/).



9. Select the **output/** folder from the S3 bucket, then click **Choose** to use it as the destination for storing the fine-tuning job outputs.



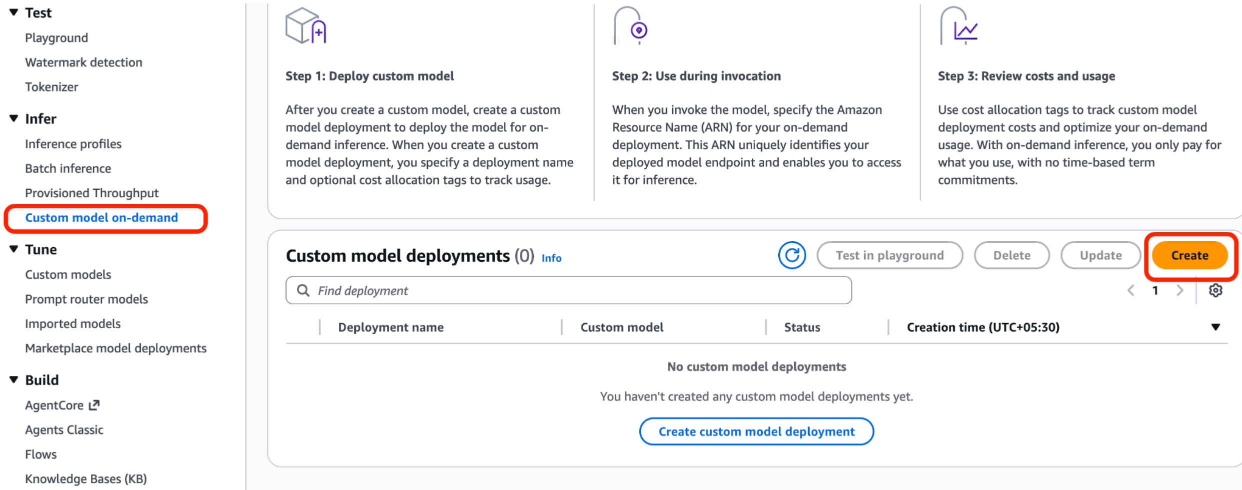
10. Create a new service role named **banking-assistant-model-role**, then click **Create job** to start the fine-tuning process for the banking assistant model.



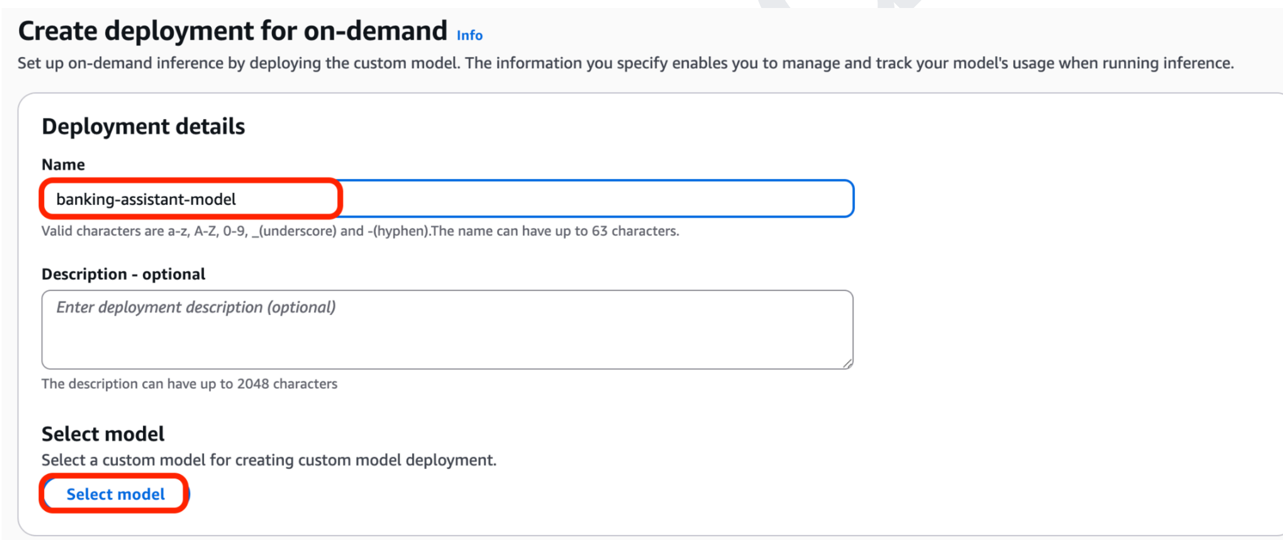
Note: For us, it took 4 hours to be in Active Status.

11. Deploy for on-demand inference

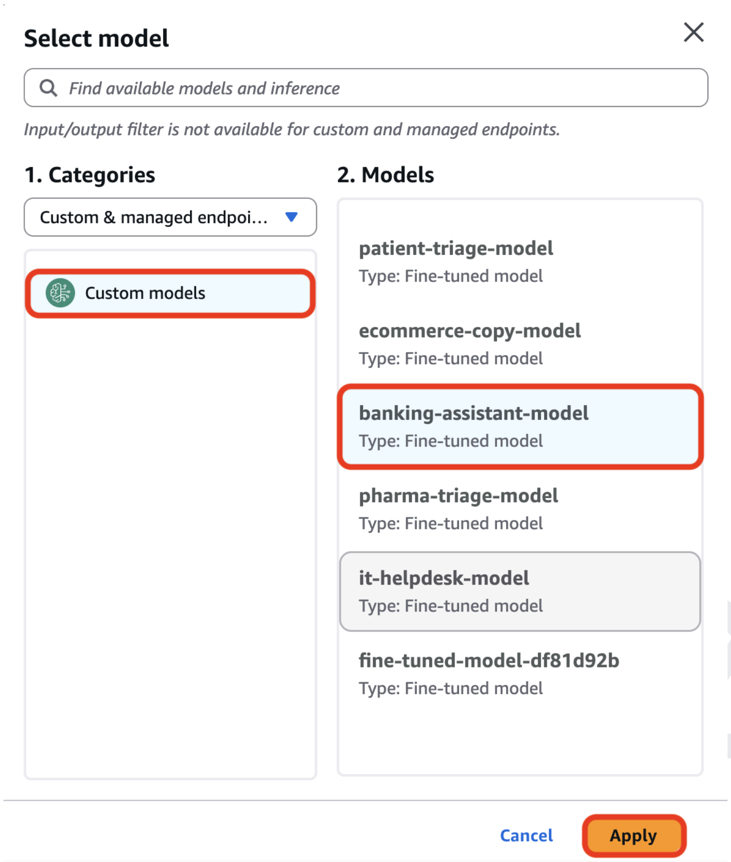
12. In the Bedrock dashboard, under **Infer**, choose **Custom model on demand**.



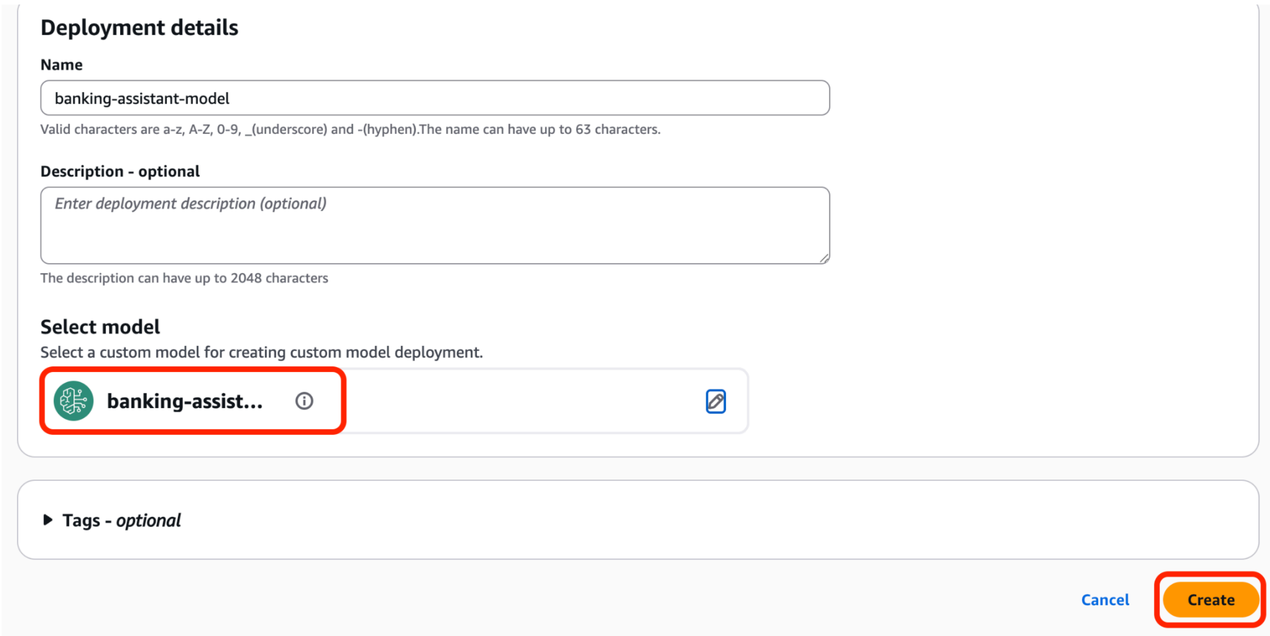
13. Enter `banking-assistant-model` as the deployment name, then click **Select model** to choose the fine-tuned banking assistant model for deployment.



14. Under **Custom models**, select `banking-assistant-model`, then click **Apply** to use the fine-tuned model for the deployment.

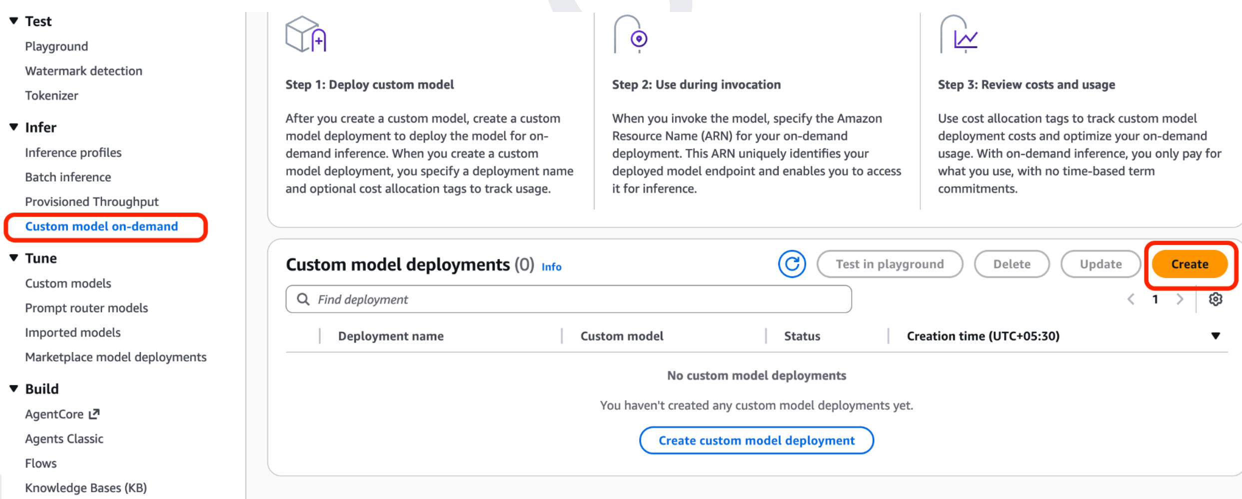


15. Verify that **`banking-assistant-model`** is selected as the deployment model, then click **Create** to deploy the fine-tuned banking assistant model for inference.



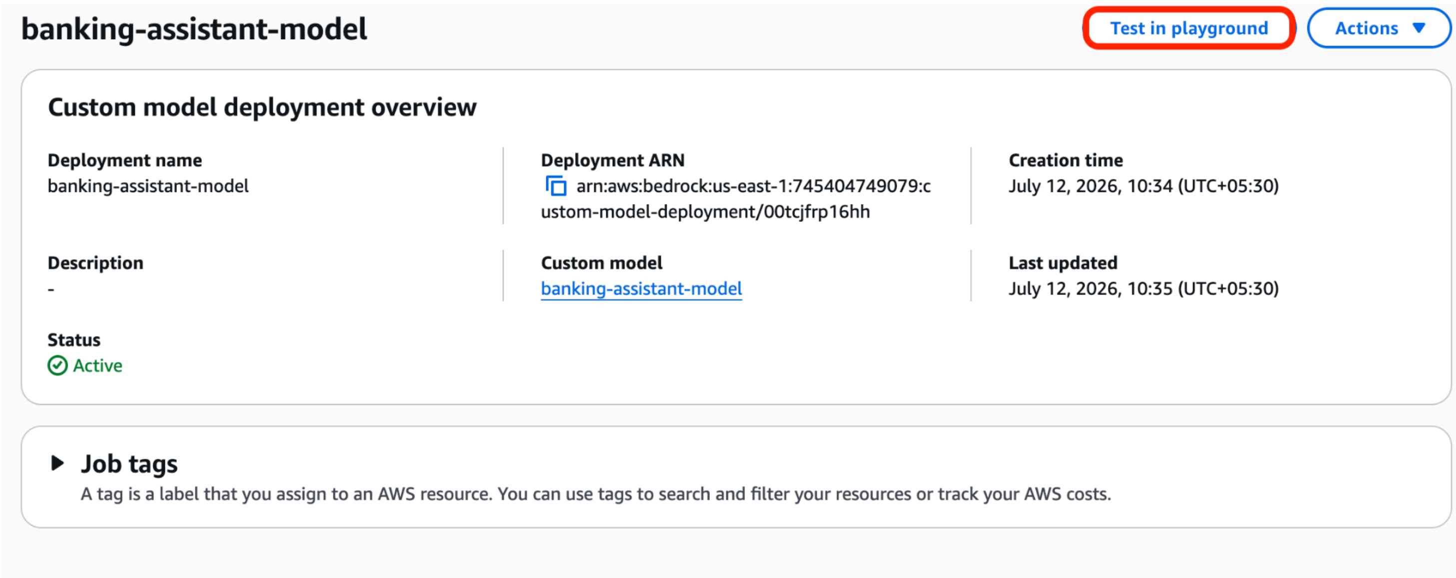
16. Test in the Playground

17. In the Bedrock dashboard, under **Infer**, choose **Custom model on demand**.

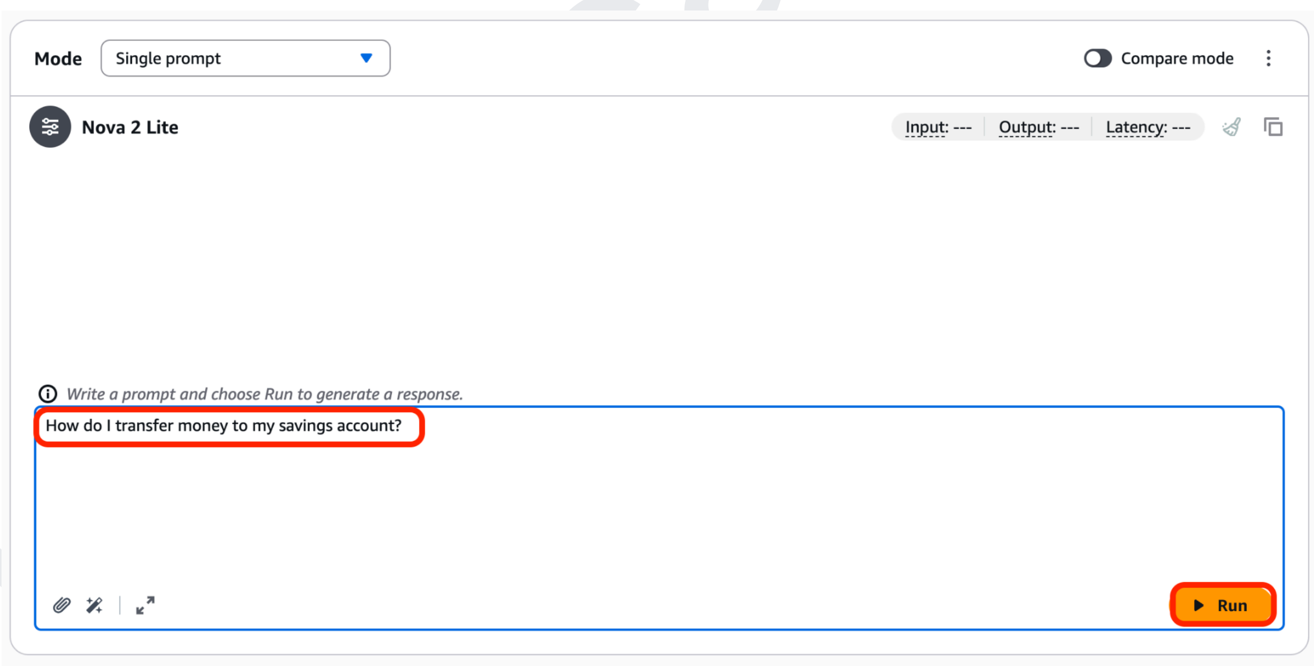


18. Verify that the deployment status is **Active**, then click **Test in playground** to validate the fine-tuned banking assistant model by sending sample banking-related queries and reviewing its

responses.

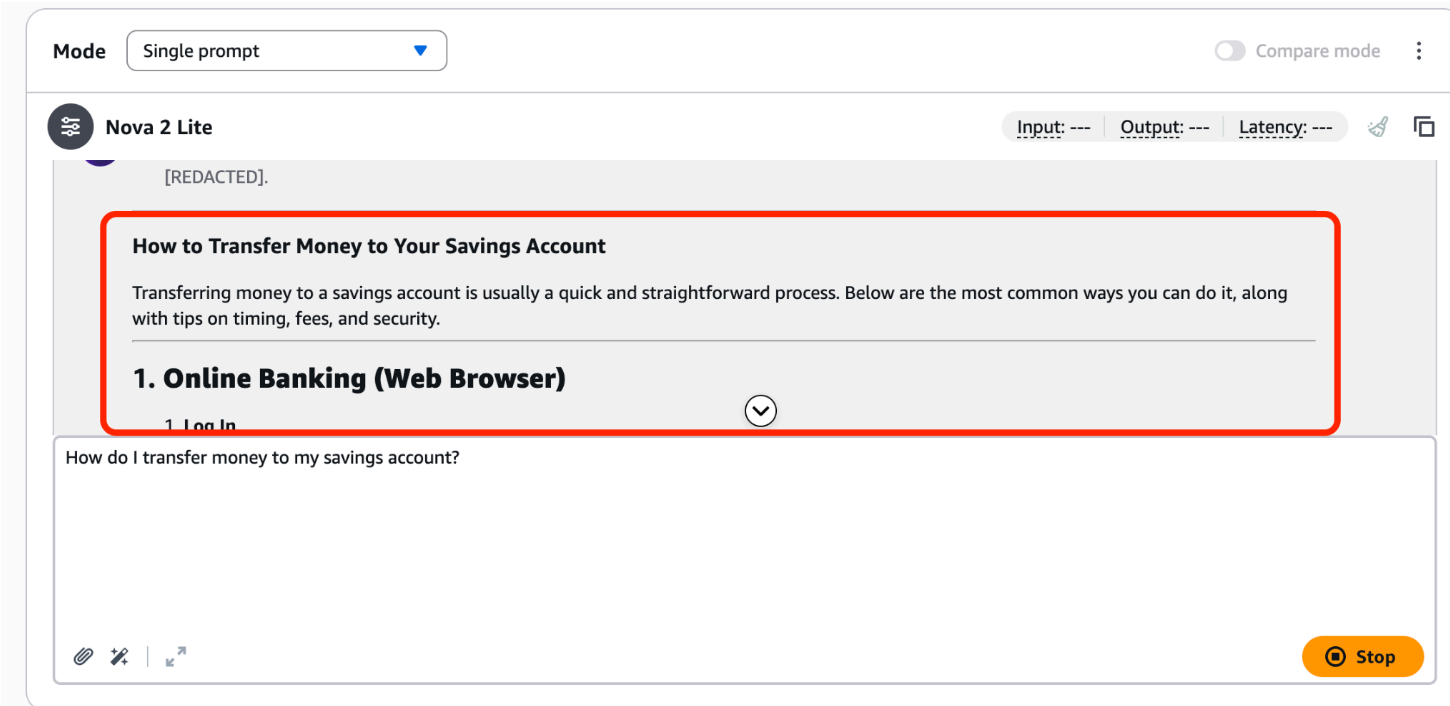


19. Enter the prompt **"How do I transfer money to my savings account?"**, then click **Run** to verify that the fine-tuned banking assistant model provides clear, accurate, and customer-friendly guidance for transferring funds between accounts.

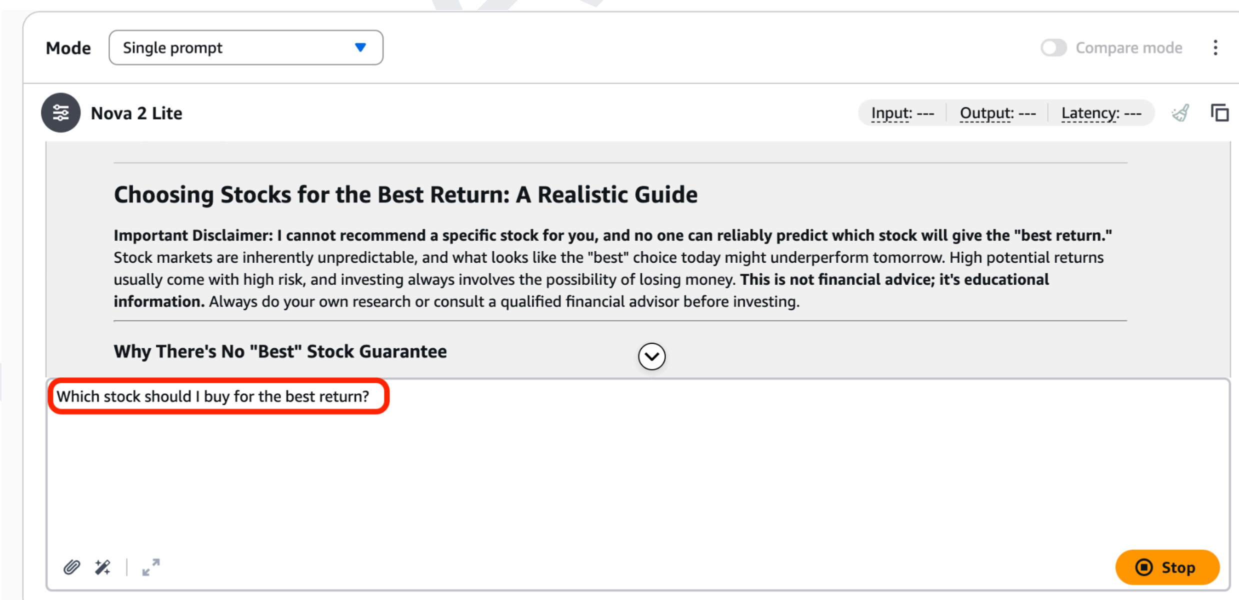


20. Verify that the fine-tuned banking assistant model provides clear, step-by-step instructions for transferring money to a savings account, including common transfer methods, security

considerations, and any applicable fees or processing times.



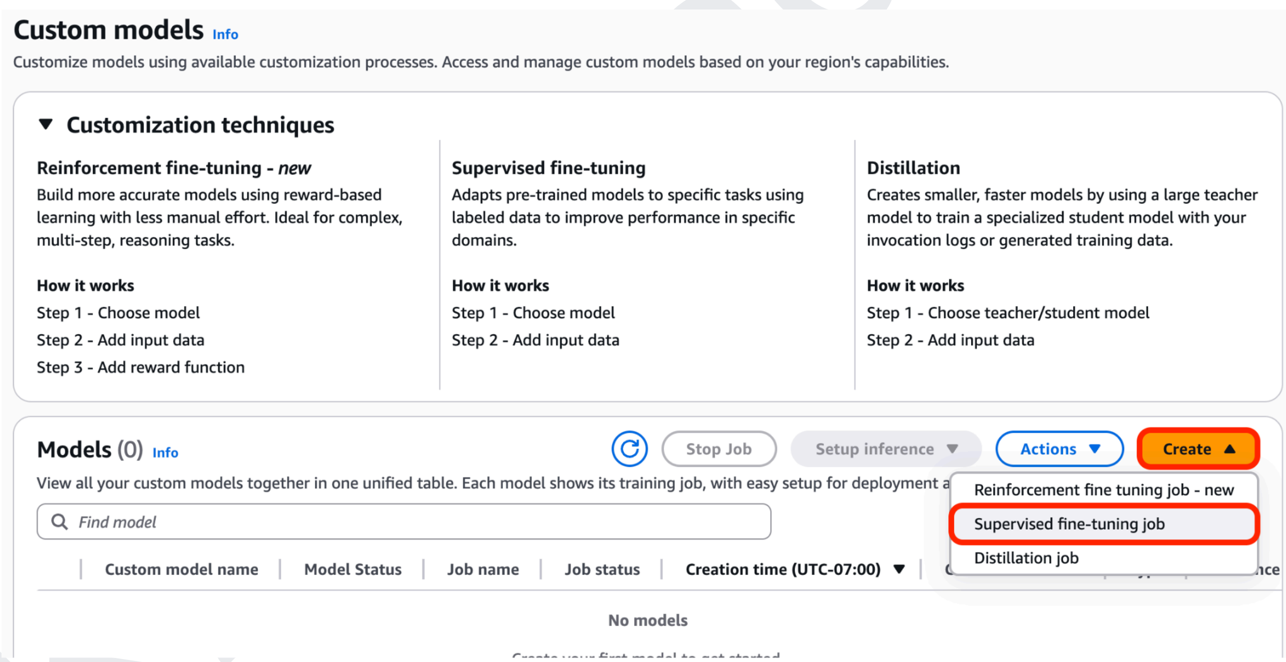
21. Verify that the fine-tuned banking assistant model responds to investment-related questions by providing balanced, educational guidance, highlighting investment risks, avoiding specific stock recommendations or guaranteed returns, and encouraging users to conduct their own research or consult a qualified financial advisor.



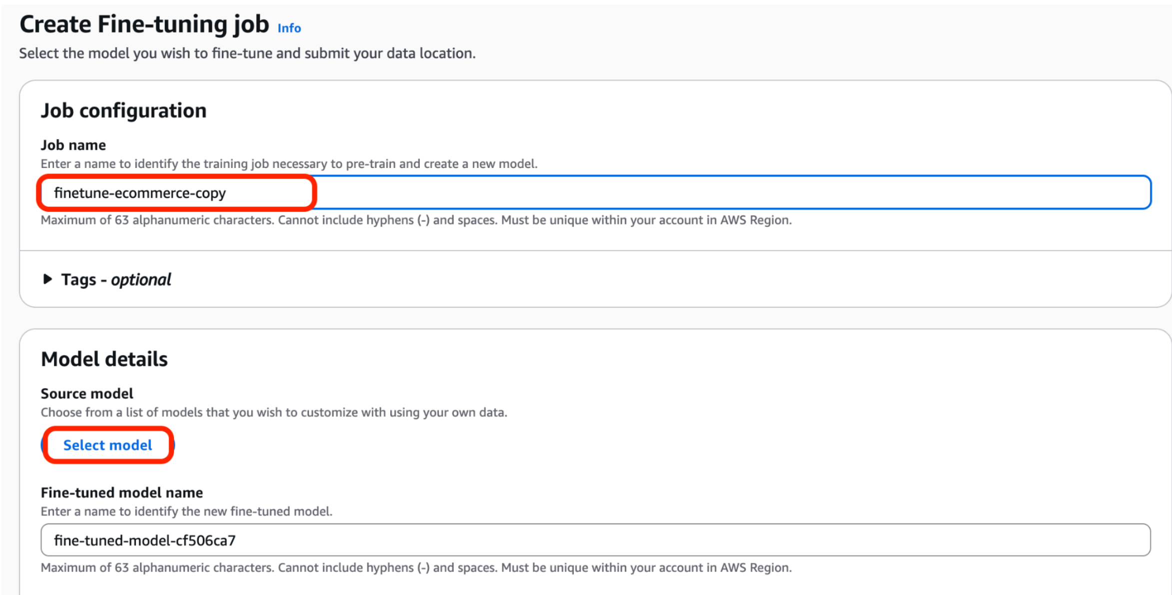
17. Scenario 6: E-Commerce Product Description Generator

This scenario trains the model to generate structured, conversion-focused product descriptions from a set of raw product attributes - such as battery life, features, and certifications. Writing compelling product copy at scale is a significant challenge for e-commerce businesses managing thousands of SKUs across multiple categories. Copywriters can only produce content so fast, and generic or poorly written descriptions directly impact search visibility and sales conversion. By fine-tuning the model on high-quality product copy examples, it learns the brand's tone, structure, and persuasion style, enabling it to produce consistent, ready-to-publish descriptions in seconds from simple product data inputs.

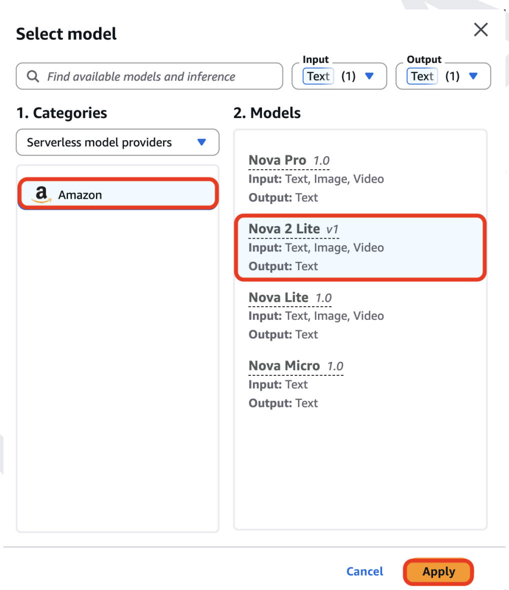
1. Click **Create**, then select **Supervised fine-tuning job** to create a new custom model training job.



2. In the **Create Fine-tuning job** page, enter **`finetune-ecommerce-copy`** as the **Job name**, then click **Select model** under **Source model** to choose the foundation model for fine-tuning.

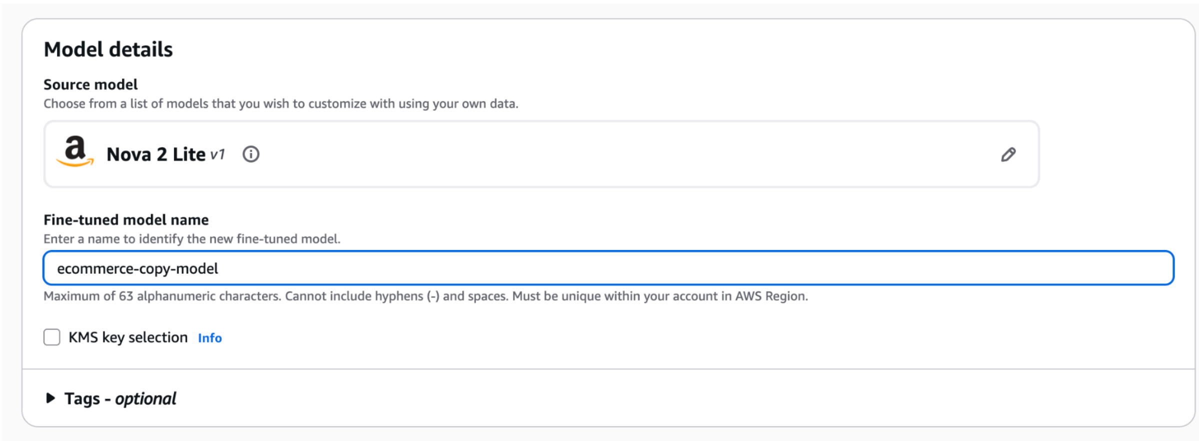


3. Select **Amazon** as the model provider, choose **Nova 2 Lite v1** as the source model, and click **Apply**.

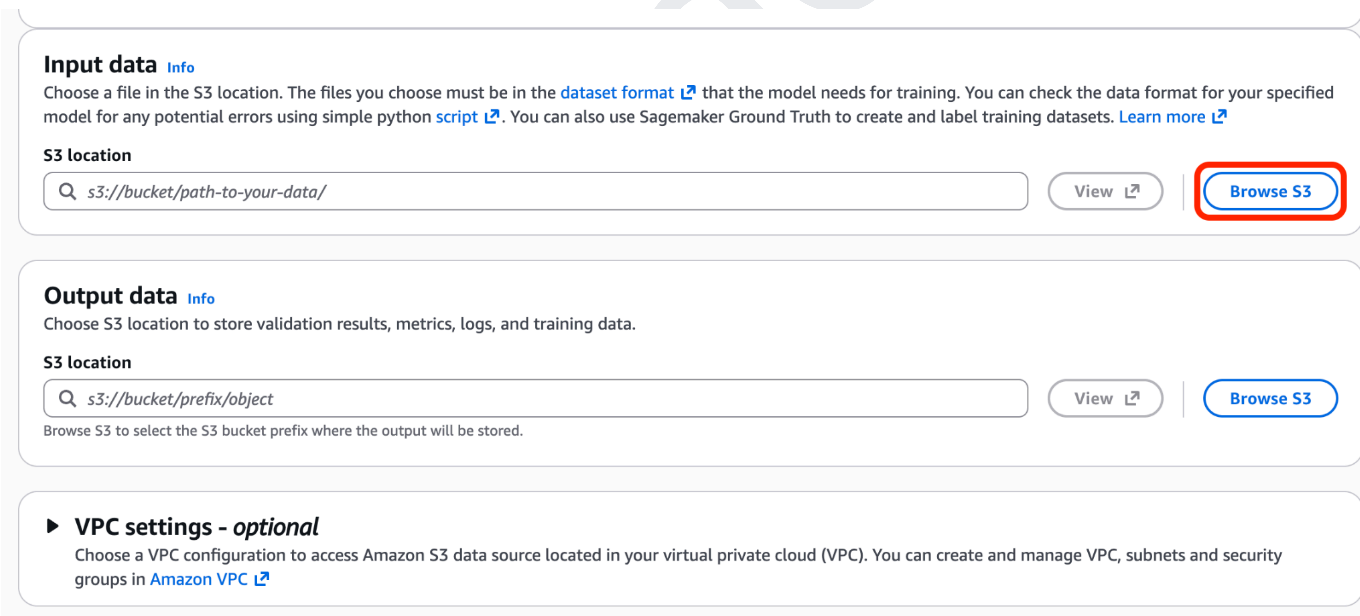


4. Verify that **Nova 2 Lite v1** is selected as the **Source model**, then enter **`ecommerce-copy-model`** as the **Fine-tuned model name**. Leave the remaining settings at

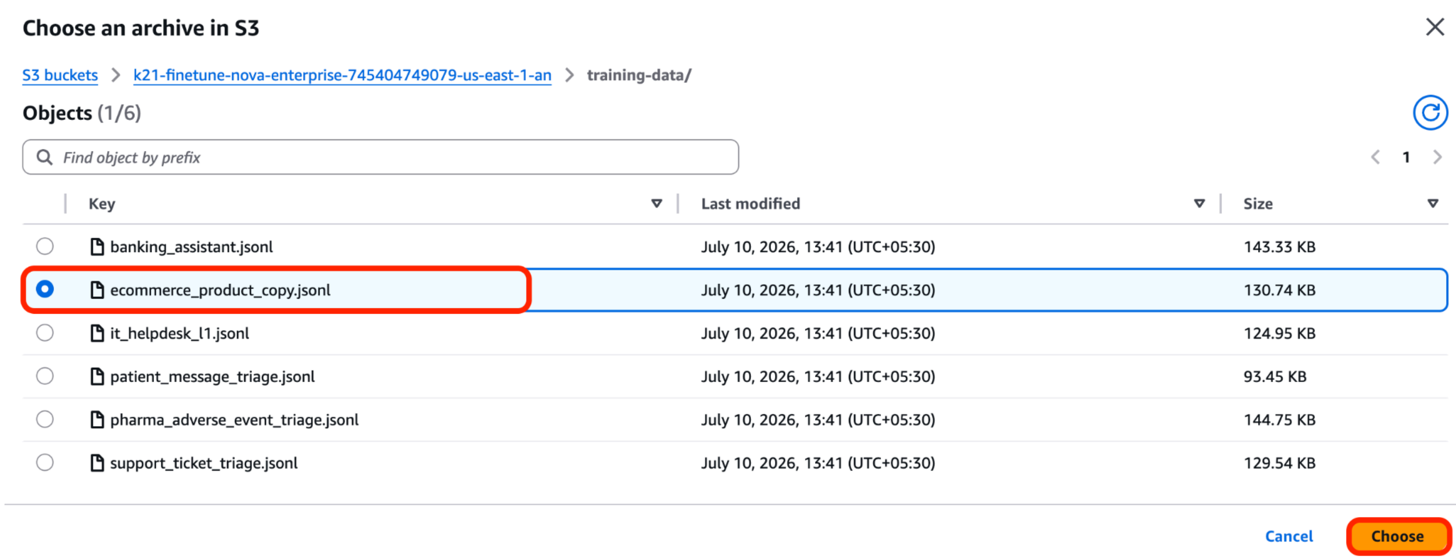
their default values and continue to the next step.



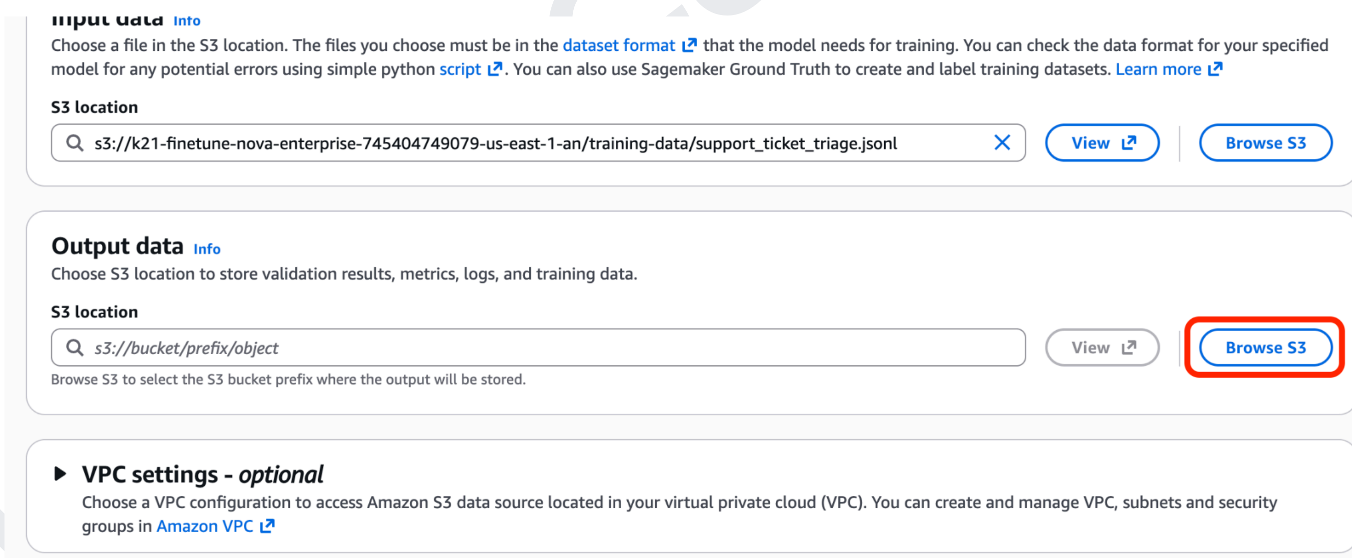
5. Hyperparameters: defaults.
6. Click **Browse S3** under **Input data** to select the training dataset from your Amazon S3 bucket. Then, choose an S3 location under **Output data** where the fine-tuning job results and model artifacts will be stored.



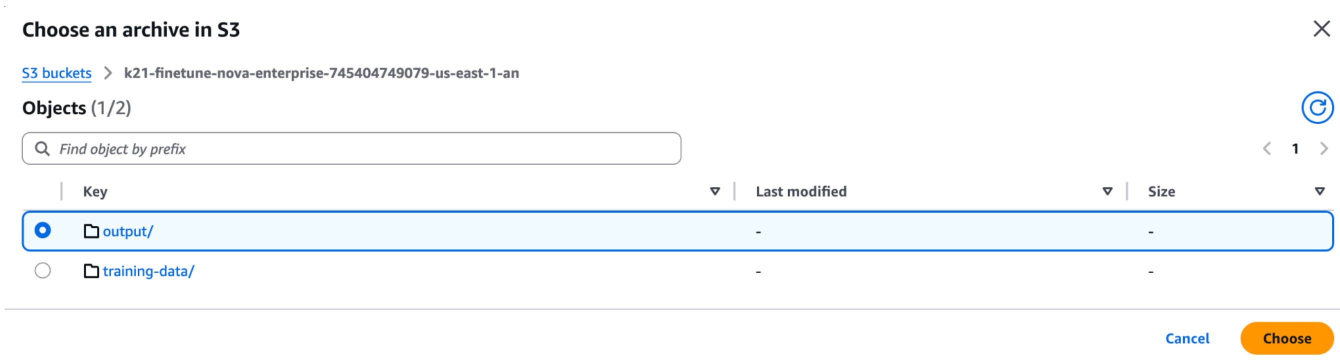
7. In the **Choose an archive in S3** dialog, select `ecommerce_product_copy.jsonl` from the **training-data** folder, then click **Choose** to use it as the training dataset for the fine-tuning job.



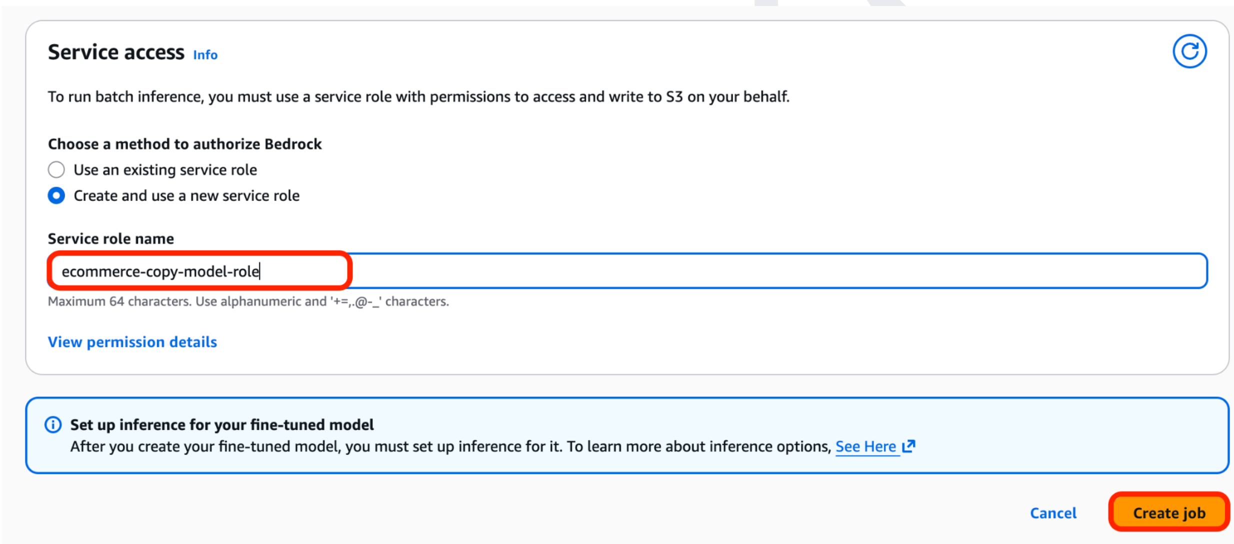
8. • • For the **Output data** location, click **Browse S3** and select the S3 folder where the fine-tuning outputs, logs, and model artifacts will be stored (for example, s3://k21-finetune-nova-enterprise/output/support-triage/).



9. Select the **output/** folder from the S3 bucket, then click **Choose** to use it as the destination for storing the fine-tuning job outputs.



10. Under **Service access**, select **Create and use a new service role**, enter `ecommerce-copy-model-role` as the **Service role name**, and click **Create job**. Wait for the fine-tuning job to complete and the model status to change to **Active** before proceeding.

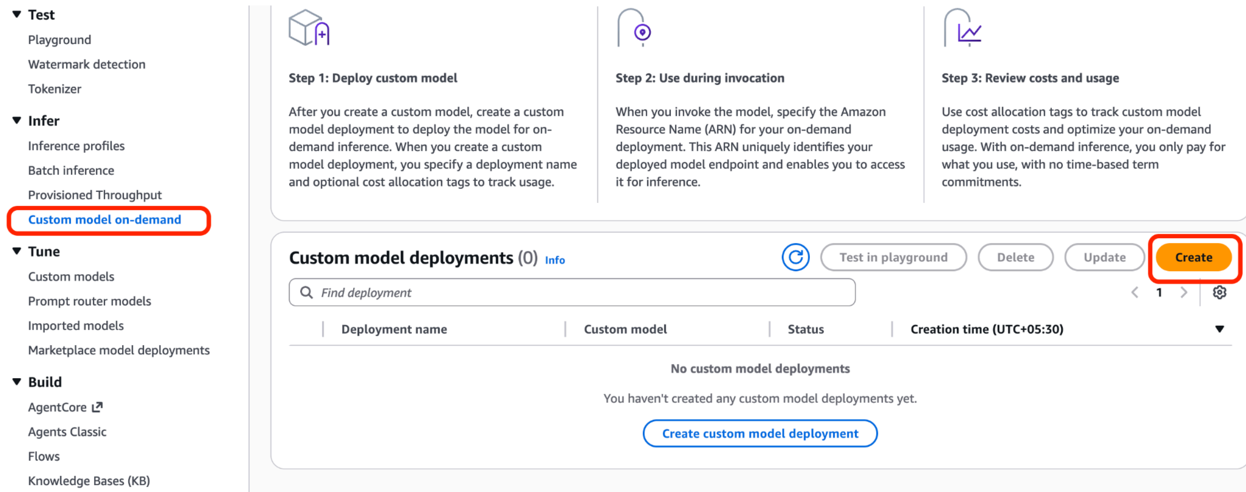


Note: For us, it took 4 hours to be in Active Status.

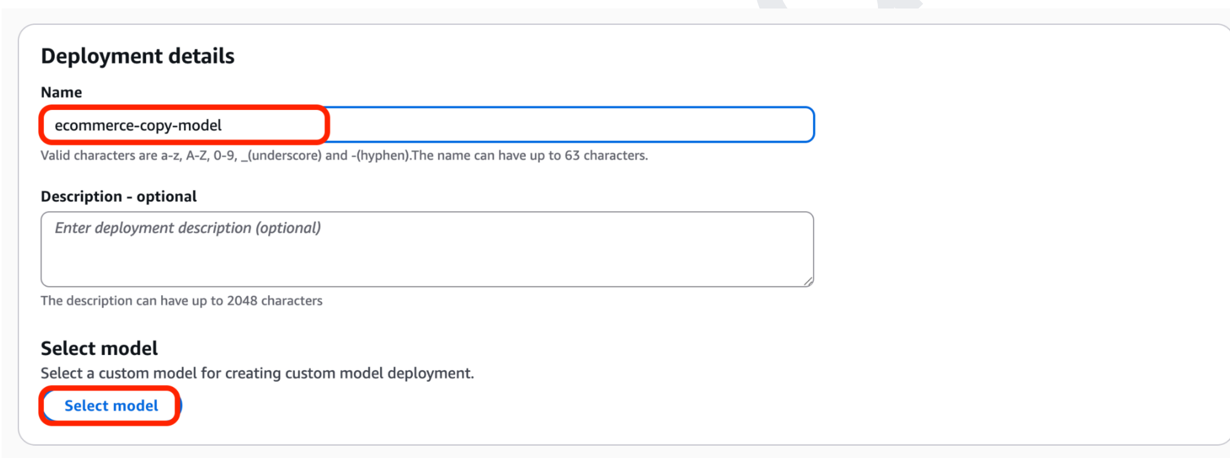
11. Deploy for on-demand inference

12. After the fine-tuning job completes and the model status becomes **Active**, navigate to **Custom model on-demand** under **Infer**, then click **Create** to deploy the fine-tuned model for on-demand

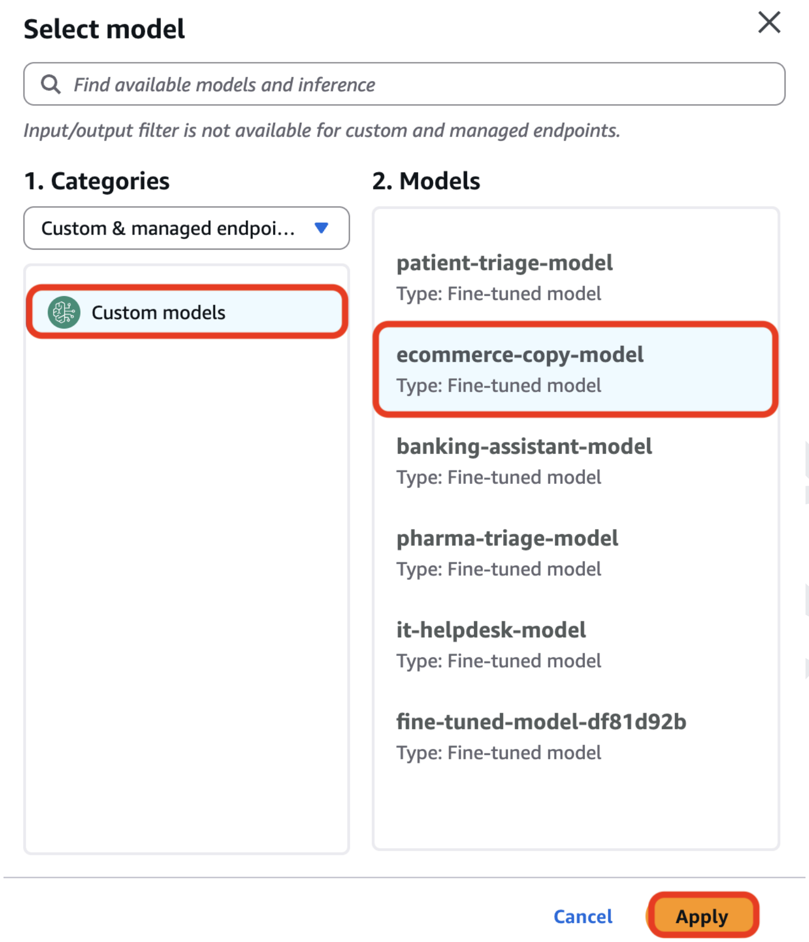
inference.



13. Enter **`ecommerce-copy-model`** as the **Deployment name**, then click **Select model** to choose the fine-tuned model that you want to deploy for on-demand inference.

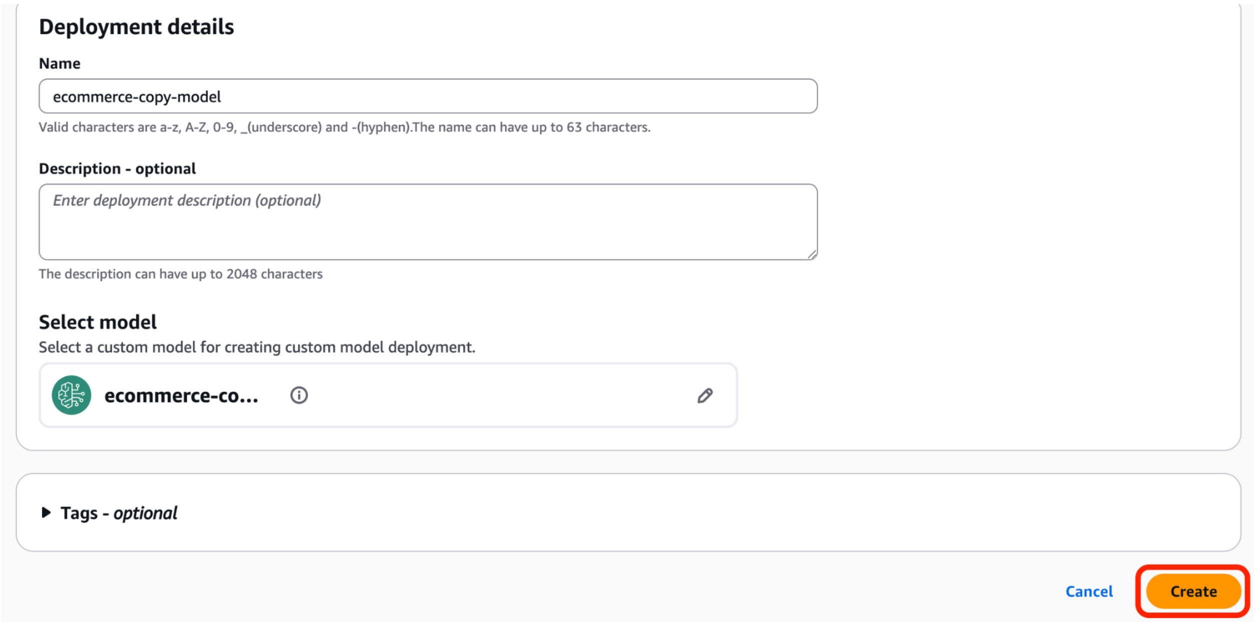


14. In the **Select model** dialog, choose **Custom models** under **Categories**, select `ecommerce-copy-model` from the list of fine-tuned models, and click **Apply**.

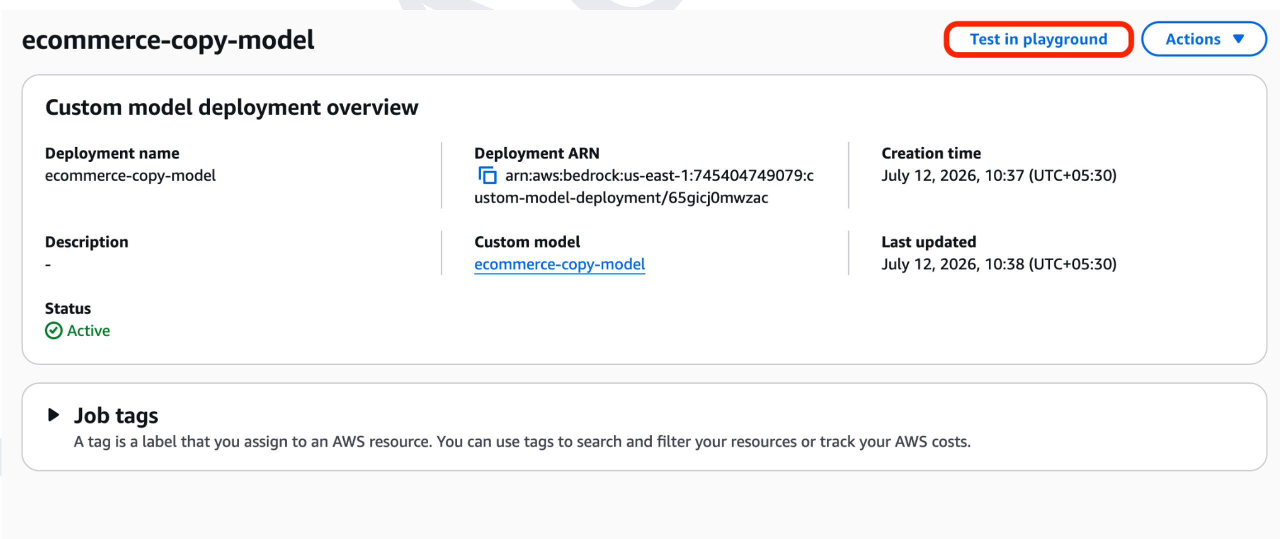


15. Verify that `ecommerce-copy-model` is selected as the deployment name and the corresponding fine-tuned model is listed under **Select model**, then click **Create**. Wait for the

deployment status to change to **Active** before testing the model in the Playground.

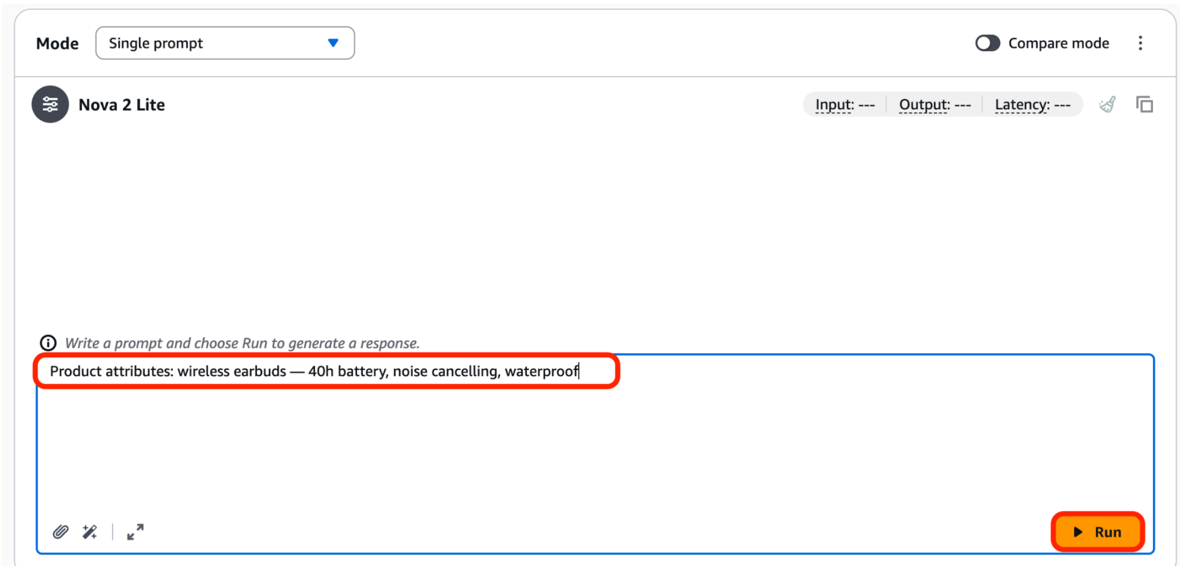


16. Test in the Playground
17. Once the deployment status changes to **Active**, click **Test in playground** to open the Amazon Bedrock Playground and interact with your fine-tuned `ecommerce-copy-model` using custom prompts.

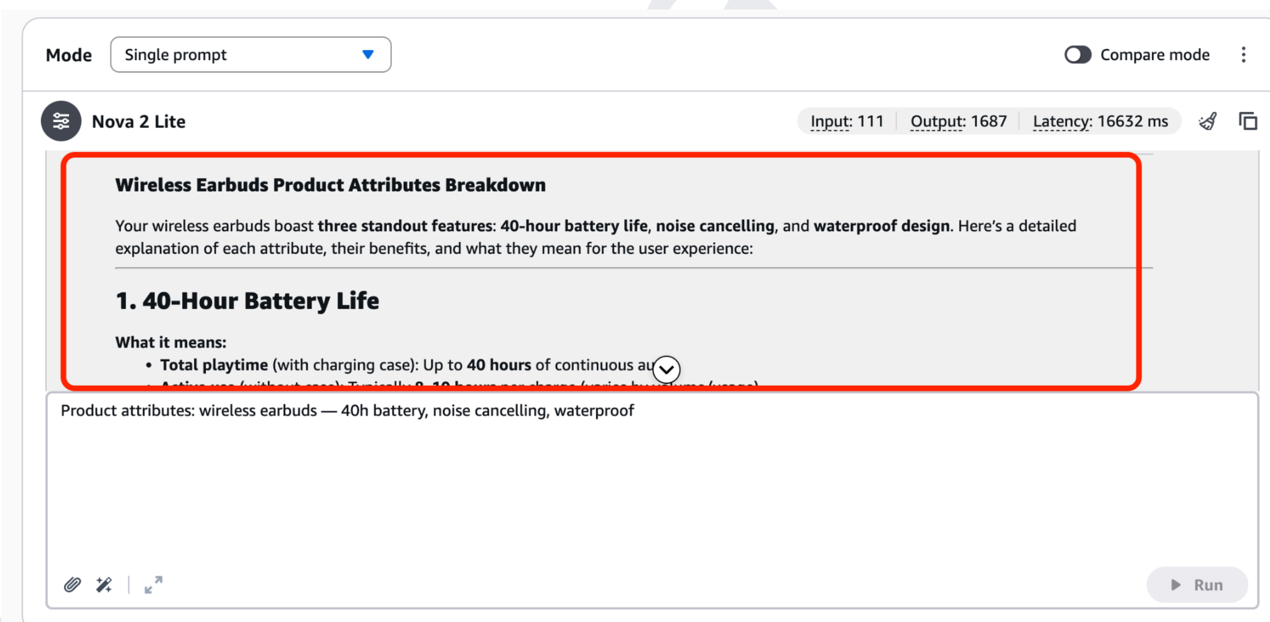


18. In the **Playground**, enter the following prompt to test your fine-tuned e-commerce copy model: Product attributes: **“ wireless earbuds — 40h battery, noise cancelling, waterproof”** . Then

click **Run** to generate marketing copy based on the provided product attributes.



19. Verify that the fine-tuned model generates relevant and structured product copy based on the provided product attributes, confirming that the deployment is working successfully.



📝 Note on Additional Scenarios

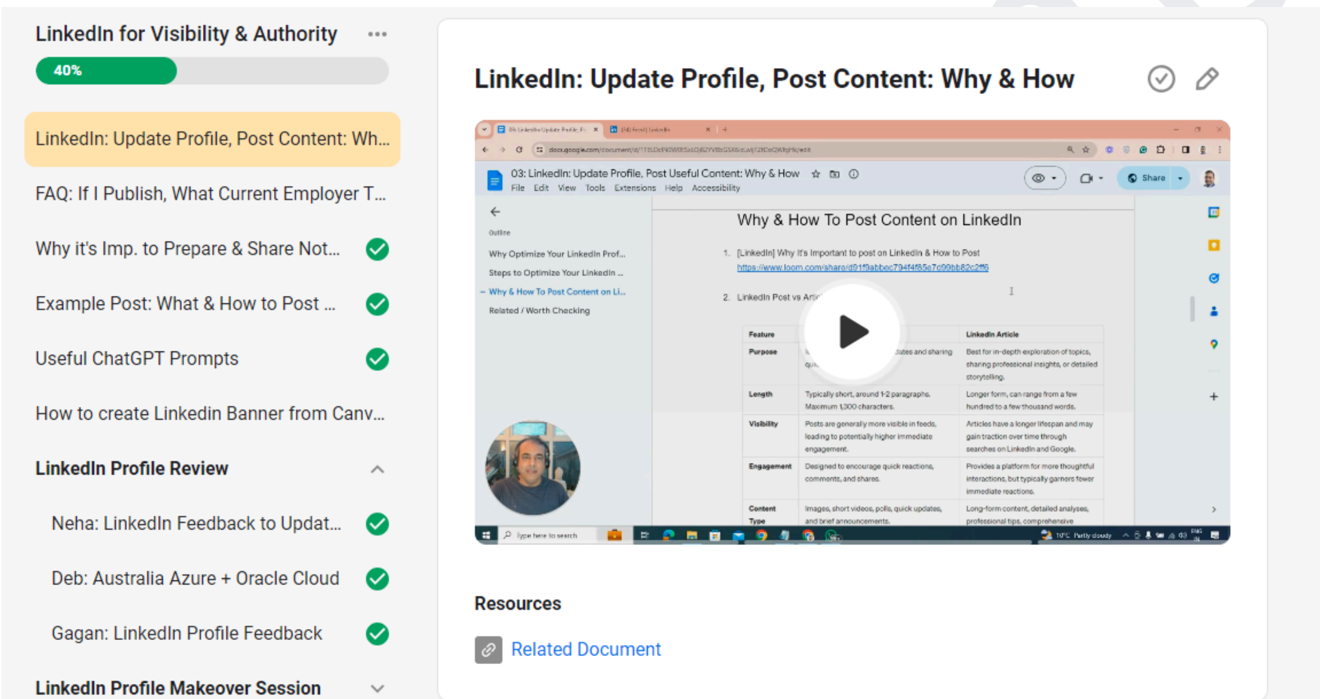
The six scenarios above have been added to this lab based on direct requests from many of you in our community. We heard that you wanted to see how the same fine-tuning workflow applies beyond a single use case — and across the industries you actually work in or are preparing for. Each scenario follows the exact same process covered in the core lab, applied to a different domain and dataset, so the steps will feel familiar as you work through them.

We hope these additional scenarios give you a broader view of what is possible with Amazon Bedrock fine-tuning and help you connect this skill to your own professional context. We will continue updating this lab as more use cases are requested - so keep the feedback coming in the community.

18. Share your learnings

In this section, you are going to share whatever you learned during this lab on LinkedIn and the Community.

Please watch the video below to understand why it's important to post & how to post: https://www.skool.com/k21academy/classroom/0c24b6af?md=f7975c8e525b4d5d8191e131bdef5740



As you know, building a strong professional profile is essential in today's job market. One way to showcase your skills and knowledge is by sharing your labs and projects on LinkedIn.

Your LinkedIn profile is a powerful tool in your job search. Sharing your labs and projects is just one way to demonstrate your expertise and stand out to potential employers. If you don't have LinkedIn, we strongly recommend that you create one for yourself.

By doing so, you can demonstrate to potential employers or connections that you have hands-on experience in your field and are actively engaged in learning and growing your skills.

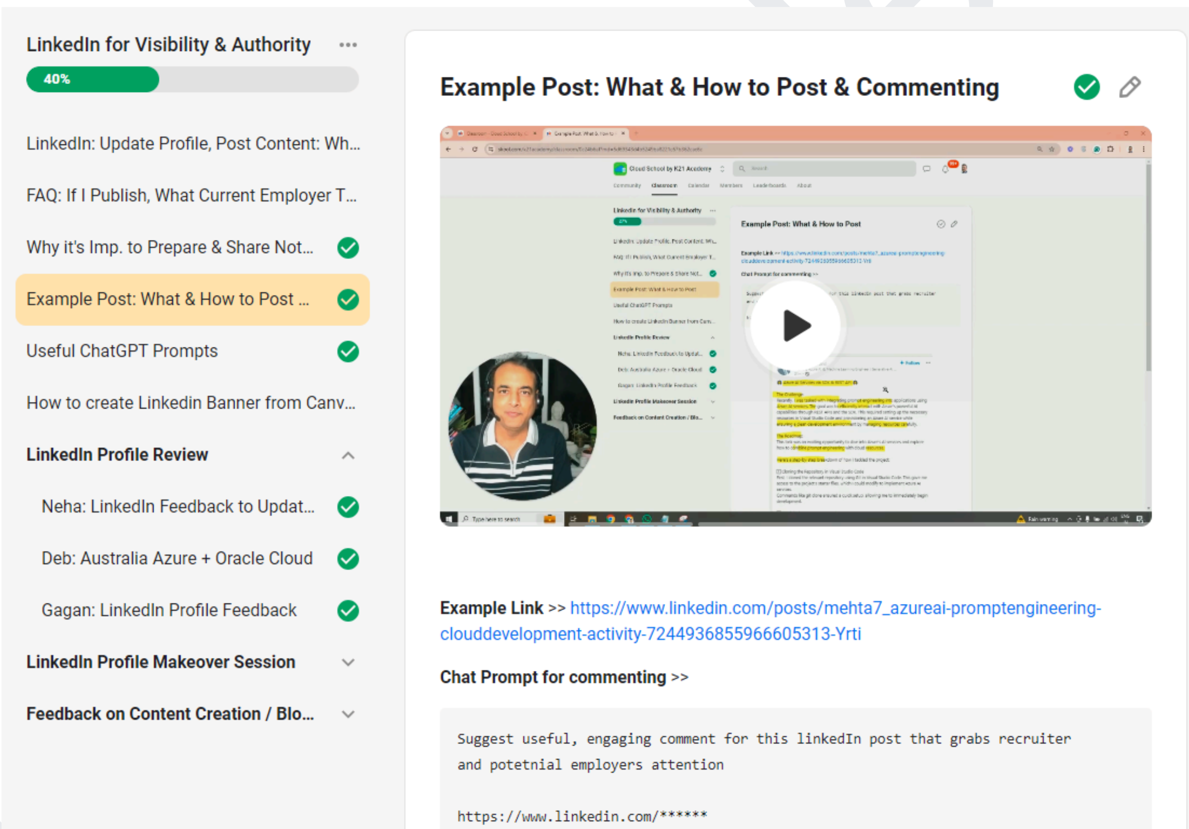
a. On LinkedIn

Take a screenshot and share it on your LinkedIn. So, this will attract recruiters and employers to your profile and increase your reach. **Do remember to tag**

K21Academy (https://www.linkedin.com/company/k21academy) &
Atul Kumar (https://www.linkedin.com/in/atulk21academy/), as we will circulate in our network too, to increase your reach.

Example Video & Post

- https://www.skool.com/k21academy/classroom/0c24b6af



Here is a sample that includes steps inside a post followed by a screenshot:

Adelakun Joshua • 3rd+

--

1d • Edited •

+ Follow

AWS CLOUDWATCH BILLING ALARM

Problem:
Today, one of my clients reached out to me. They have AWS billing challenges. They just got an email alert from AWS stating that, they have to pay certain amount of bills. When they evaluated their end of the month service bills, they discovered that they are billed for a lot of services that they are actually not using on a day-to-day basis. But unfortunately, they didnt properly monitor and manage their aws resources, which incur charges.

My recommendation:
I recommended AWS Cloudwatch alarm to my client. And i configure the cloudwatch alarm for them.

AWS Cloudwatch:
Amazon CloudWatch is a monitoring and management service that provides data and actionable insights for AWS, on-premises, hybrid, and other cloud applications and infrastructure resources.

How I Set the Cloudwatch Billing Alarm Up:
1. I Opened the CloudWatch console at https://lnkd.in/gugv5tEE ✅.
2. In the navigation pane, I choose Alarms, and then choose All alarms.
I Choosed Create alarm.
3. Then, I Selected 'metric'. In Browse, I choose 'Billing', and then 'Total Estimated Charge'.
4. I Selected the box for the 'EstimatedCharges metric', and then I choose 'Select metric'.
5. For 'Statistic', I choose Maximum.
6. For 'Period', I choose 6 hours.
7.For 'Threshold type', I choose Static.
8. For 'Whenever EstimatedCharges is' . . .,I selected Greater.
For 'than' . . ., I defined the value that i want to cause the alarm to trigger. For example, $10 USD

SNS topic included the email address by which my client will recieve email when the billing amount crosses the billing threshold specified.
Note:
You can select an existing Amazon SNS topic, create a 'new Amazon SNS topic', or use a topic ARN to notify other account. If you want your alarm to send multiple notifications for the same alarm state or for different alarm states, choose 'Add notification'.
11. I choosed Next.
12. Under Name and description, I entered a name for the alarm. (Optional) Enter a description of the alarm.
13. Then I Choosed 'Next'.
14. Under 'Preview and create', after confirming that the configuration is correct, and then i choosed 'Create alarm'.

Conclusion: Organization that properly set up a cloudwatch billing alarm, will be able to properly monitor their aws resources usage. The metrics provided by the cloudwatch alarm through the dashboard will also help the organization to effectively control their spending.

#aws
#awscloud
#JTechconsult
#K21Academy: Learn Cloud From Experts
#AtulKumar
#SumtiMehta

Cloudwatch billing alarm architectural diagram:

Other examples:

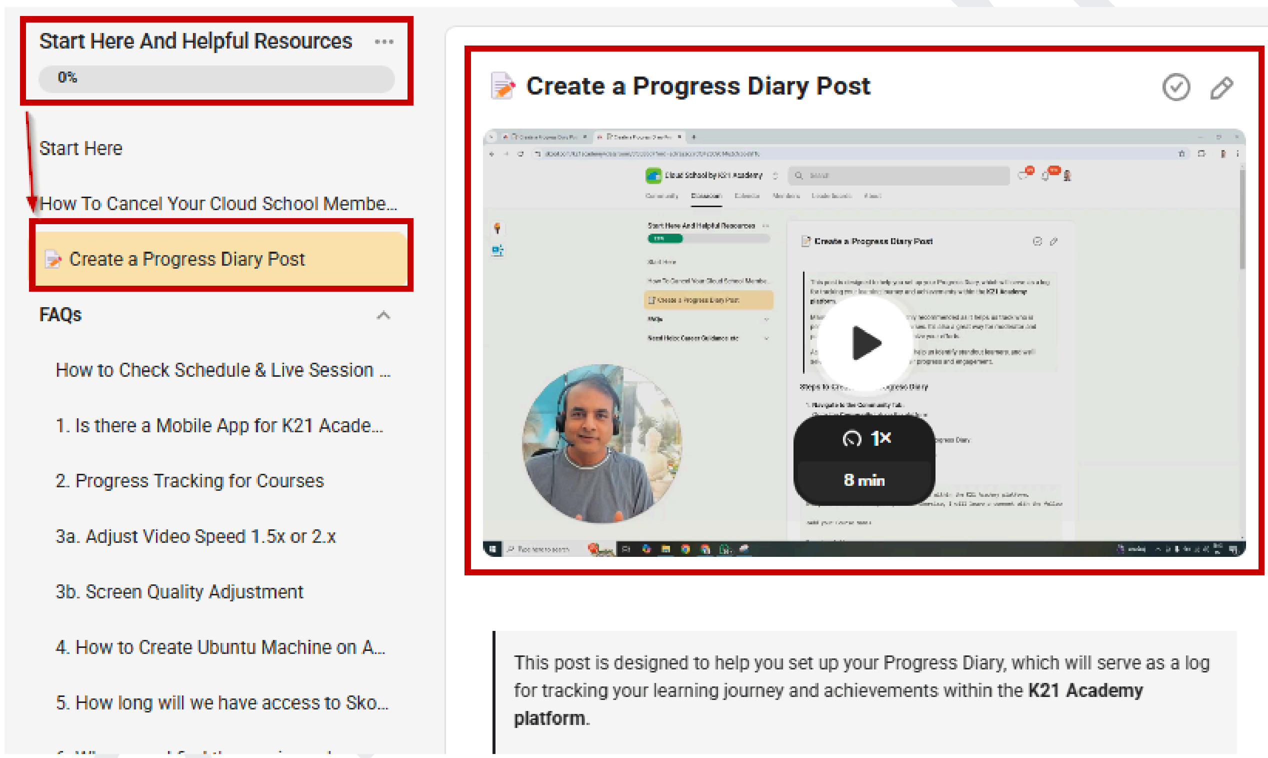
http://go.k21academy.com/4lSBFcz

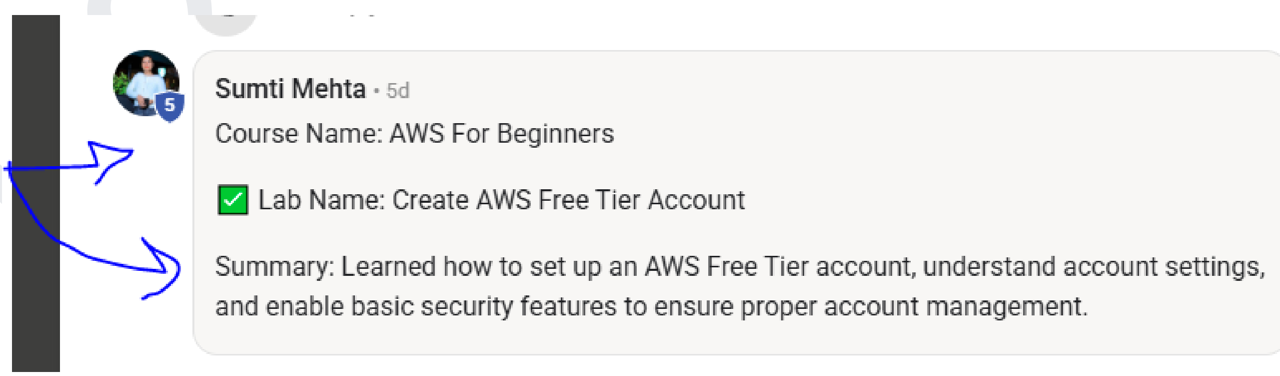
http://go.k21academy.com/4lS1GbD

b. Share Wins on Community

Share screenshots in Cloud School Community under your **'Progress Diary'** (If you have not created a Progress Diary yet, then check at https://www.skool.com/k21academy/classroom/8f3838d7?md=ad7a2acc7df34280901462cfc65e9f16).

This will boost your confidence in the progressive path you are following and encourage/inspire others to perform these hands-on labs.





19. Delete And Clean Up

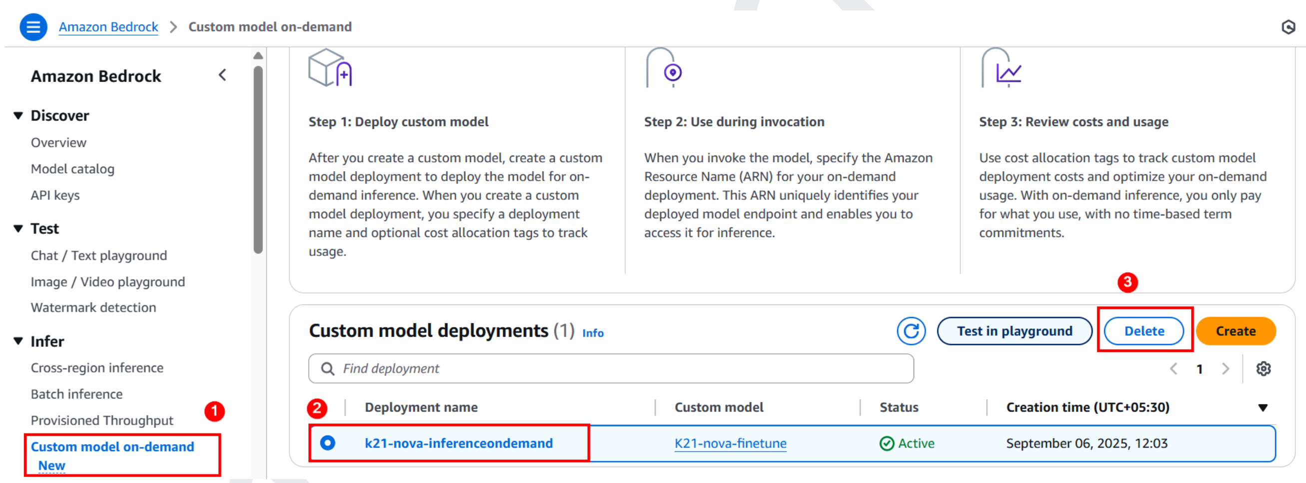
Deleting your custom model after the project is completed is important to avoid incurring unnecessary costs. Even with the **AWS Free Tier**, exceeding the usage limits or leaving unused resources running can result in unexpected charges, especially for services like **Amazon Bedrock**.

By deleting the custom model and related resources, you ensure that you're only billed for the services you actively use, maintaining cost efficiency and preventing unwanted charges.

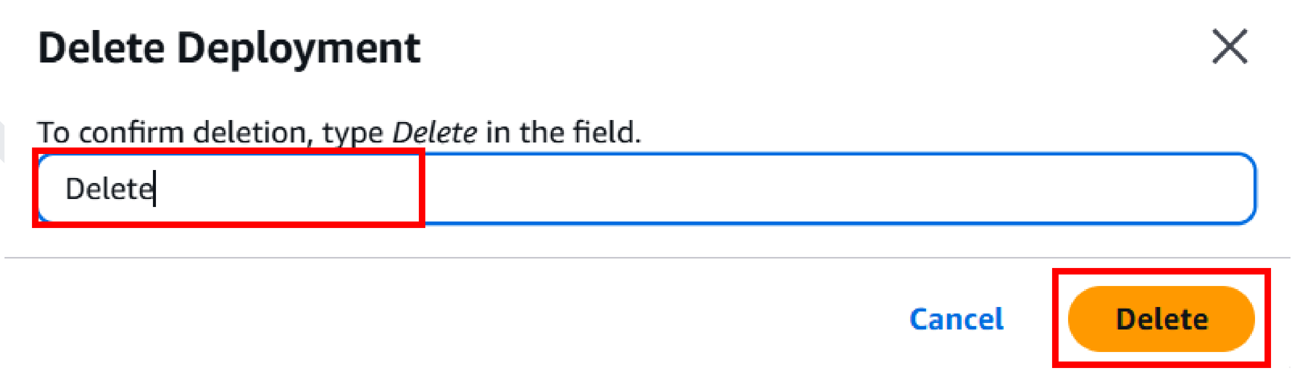
a. Terminating the Custom Model

In this section, we shall see the steps to delete the custom Model in Bedrock.

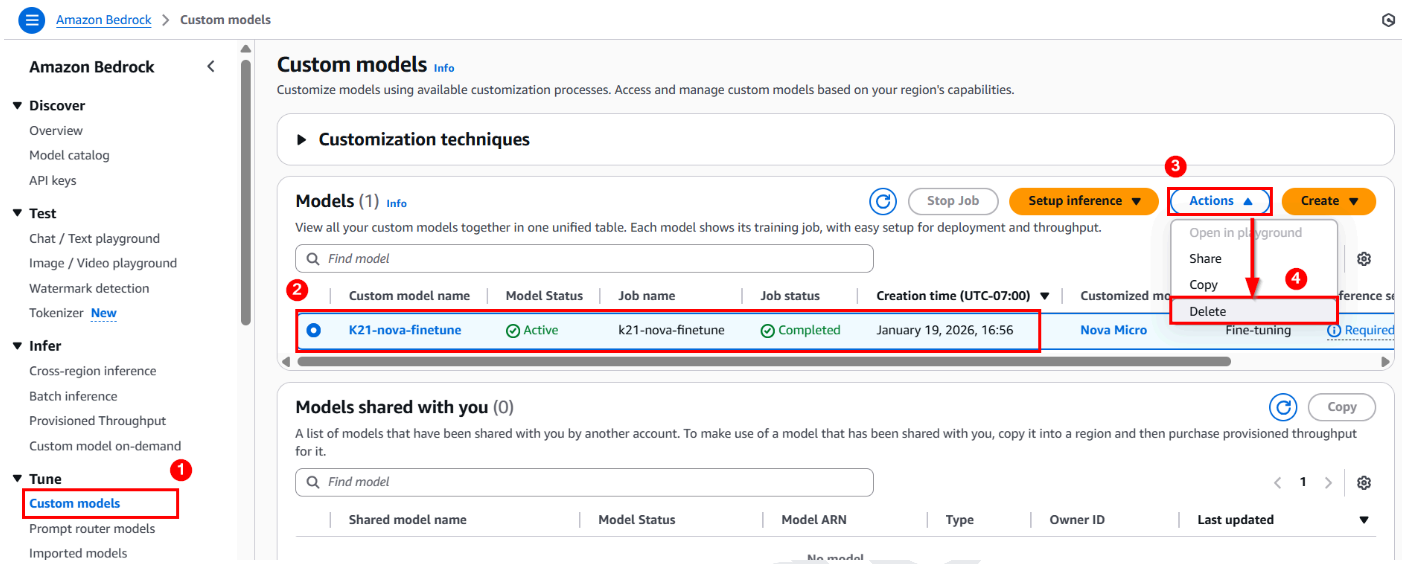
1. In the Bedrock dashboard, select the custom model and click on Delete.



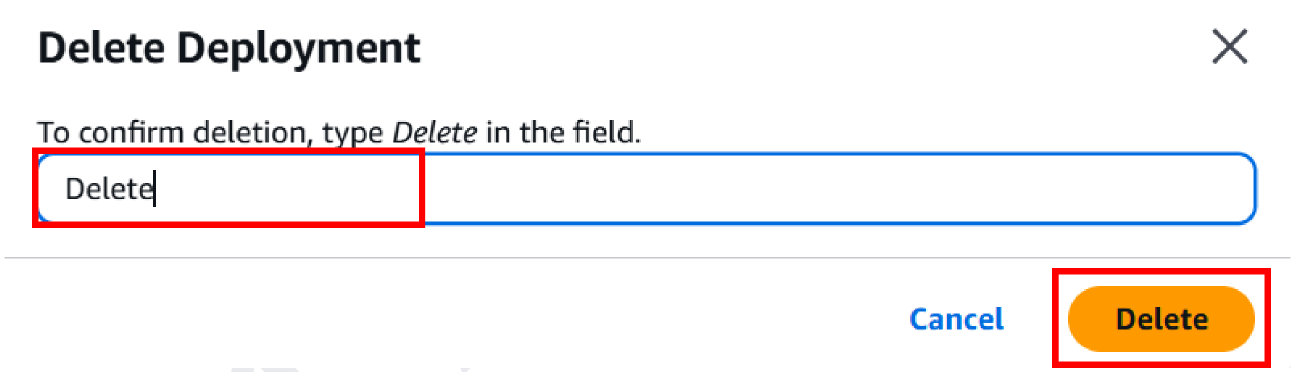
2. Now, to confirm deletion, type Delete and click on it.



3. Now, under the tune, go to Custom Model, select the active, and click on delete.



4. Now, to confirm deletion, type Delete and click on it.

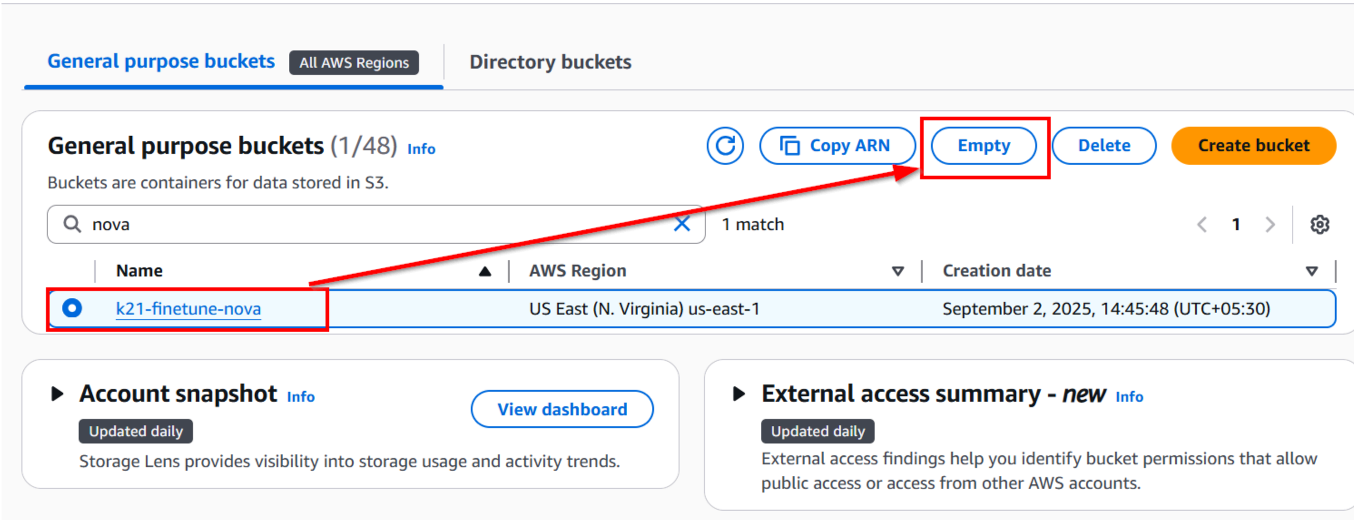


Thus, we have successfully deleted the Bedrock Custom Model.

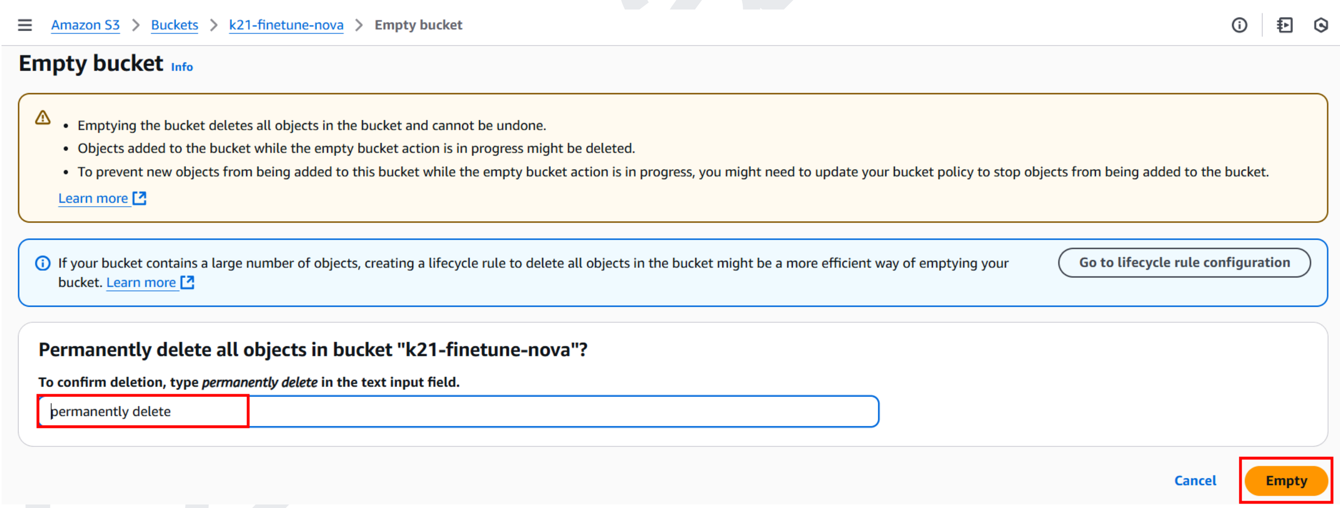
Note: Follow the same steps to delete all the remaining models, one by one.

b. Delete the S3 Bucket

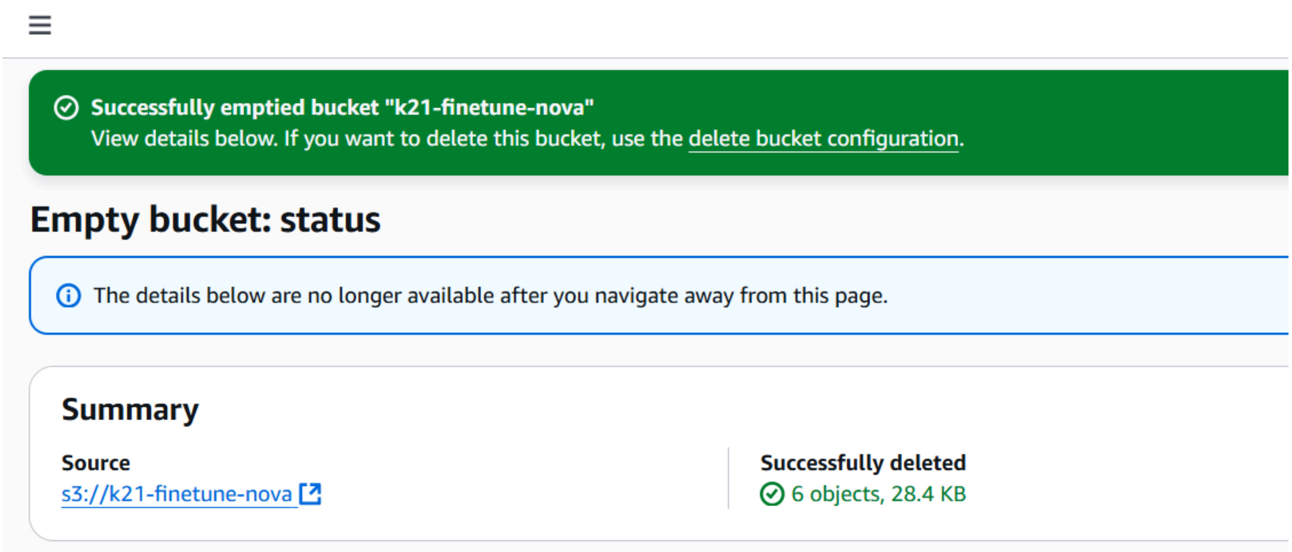
1. Go to S3 and select our bucket, and click on **Empty**.



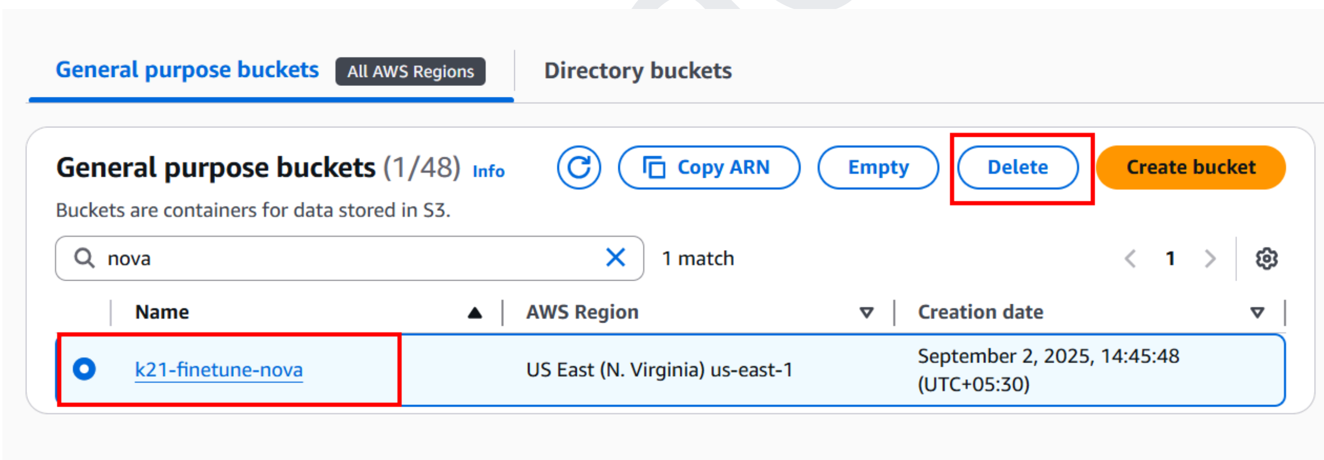
2. To confirm deletion, type **permanently delete** and click on **Empty.**



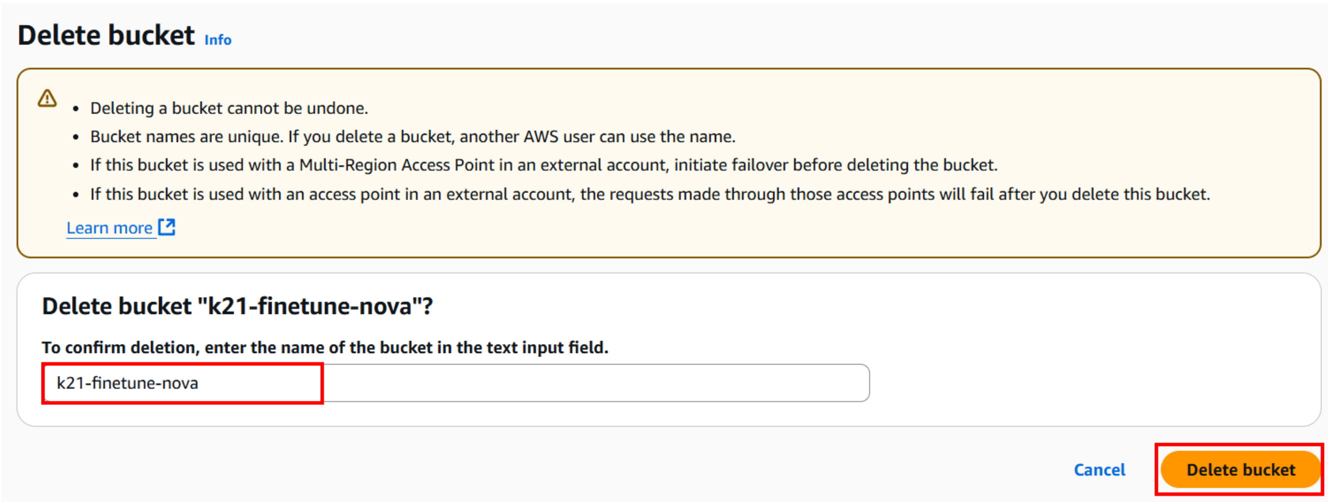
3. Now we have successfully emptied the entire bucket.



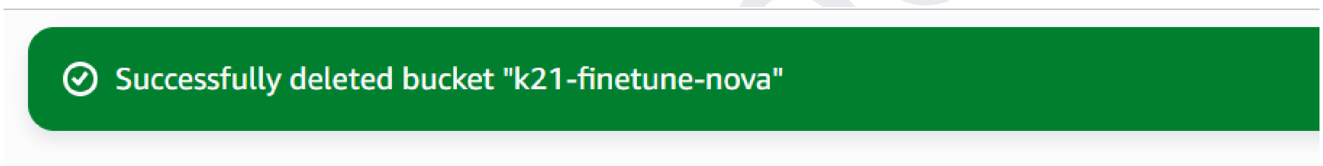
4. Once the bucket is empty, we can now delete it. Select our bucket and click on **Delete**.



5. Now, enter the bucket name and click on **Delete Bucket**.



6. Now you shall see our bucket is **deleted** successfully.



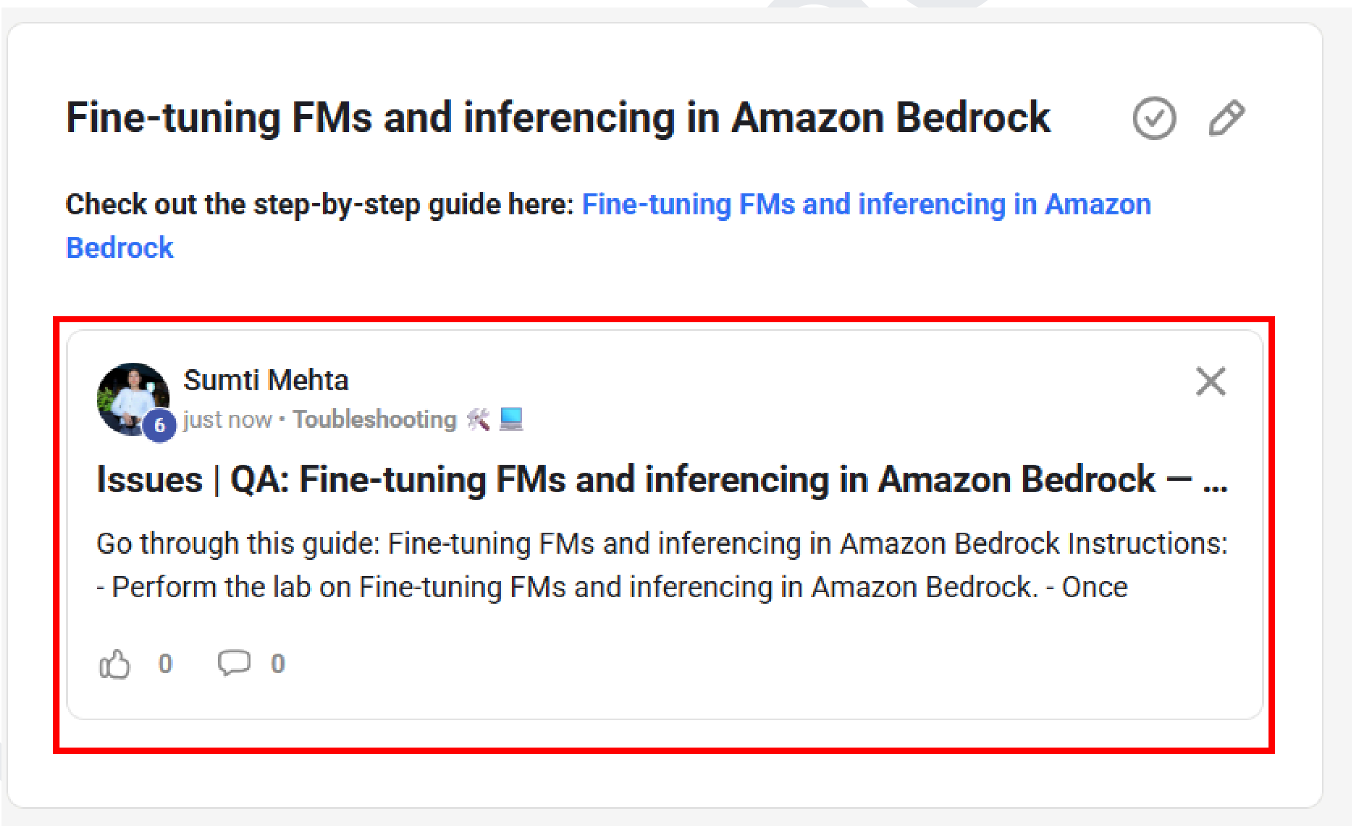
Thus, we successfully emptied and deleted the bucket.

20. TROUBLESHOOTING

In this section, we outline some common issues you may encounter during the process, along with explanations of why these problems may occur. This will help you quickly identify and resolve any challenges you face.

Note: *If you hit any issue in performing this lab, then add your issue in the thread below this lab (linked to this thread in the community)*

https://www.skool.com/k21academy/issues-qa-fine-tuning-fms-and-inferencing-in-amazon-bedrock-post-here



21. Summary

In this lab, we covered the following key concepts and hands-on exercises to give you a complete understanding of working with Amazon Bedrock:

1. Created an S3 Bucket and Organized Training Data
2. Built a Custom Model with Fine-Tuning
3. Created and Used Custom Models on Demand with Inference
4. Tested the Custom Model in the Bedrock Playground
5. Explored Six Additional Industry Scenarios
    - Scenario 1: Support Ticket Triage
    - Scenario 2: IT / DevOps L1 Helpdesk
    - Scenario 3: Pharmacovigilance Adverse-Event Triage
    - Scenario 4: Patient Message Triage
    - Scenario 5: Banking Virtual Assistant
    - Scenario 6: E-Commerce Product Description Generator
6. Cleaned Up Resources

By completing this lab, you've gained practical experience in fine-tuning models and using Amazon Bedrock to build, test, and deploy AI models in a scalable cloud environment.